



OSR Standard eHSM

技术参考手册

OSR IP

June 23, 2026

Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2026 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

版权声明和专有信息声明

版权所有 © 2014 - 2026 深圳市纽创信安科技开发有限公司保留所有权利。本文档包含机密和专有信息，属于深圳市纽创信安科技开发有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市纽创信安科技开发有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

Contents

1	前言	15
1.1	关于本手册	16
1.1.1	目标受众	16
1.1.2	约定	16
1.1.3	术语表	16
1.2	客户支持	19
2	eHSM 介绍	20
2.1	功能列表	21
2.2	系统架构	22
2.3	时钟和复位架构	25
2.3.1	子模块	25
2.3.2	CGU 架构	27
2.3.3	RGU 架构	28
2.4	存储资源需求	29
2.5	IO 信号	30
2.5.1	时钟与复位接口	31
2.5.2	状态与错误接口	31
2.5.3	128 Bit Debug Enable	36
2.5.4	Mailbox AHB Slave 接口	36
2.5.5	AHB Master 接口	36
2.5.6	DMA AXI Master 接口	37
2.5.7	NVM AHB Master 接口	39
2.5.8	CFG AHB Master 接口	40
2.5.9	OTP AHB Master 接口	40
2.5.10	IROM 接口	40
2.5.11	IRAM 接口	41
2.5.12	DRAM 接口	42
2.5.13	KMU RAM 接口	42
2.5.14	PKE RAM 接口	42
2.5.15	调试接口	44
2.5.16	UART 接口	44
2.5.17	TRNG 测试接口	44
2.5.18	DFT 接口	44
2.5.19	Mailbox 中断接口	45
2.6	内存映射	45
2.7	CPU 中断	47
3	安全存储	48
3.1	OTP	48
3.1.1	OTP 内存映射	49
3.2	OTP 硬件控制字段	50
3.3	传感器响应	51
3.4	密钥存储	51
3.4.1	eHSM 密钥	51
3.4.2	OTP 密钥字段	52
3.4.3	eHSM 密钥属性	53
3.4.4	OTP 密钥安装	56
3.5	敏感信息存储	59
3.6	固件镜像存储	59

4	安全生命周期	60
4.1	生命周期介绍	60
4.2	生命周期定义	61
4.2.1	生命周期中的功能控制	61
4.2.2	生命周期中的 OTP 密钥解密	62
4.2.3	生命周期中的 KMU 密钥访问权限	63
4.2.4	生命周期中的 eHSM 硬件启动错误	63
5	安全启动	65
5.1	启动需求	65
5.2	硬件启动流程	65
6	安全调试	68
6.1	安全调试鉴权流程	68
7	密码服务	69
7.1	TRNG	69
7.2	HASH	69
7.3	SKE	69
7.4	PKE	70
8	自检	71
8.1	硬件自检	71
8.2	固件自检	71
8.2.1	算法自检	71
9	系统 CSR	73
9.1	系统 CSR 介绍	73
9.2	系统 CSR 特性	73
9.3	系统 CSR 寄存器	74
9.3.1	系统 CSR 寄存器映射	74
9.3.2	系统 CSR SYS_HW_INF0	76
9.3.3	系统 CSR SYS_HW_INF1	76
9.3.4	系统 CSR SYS_HW_INF2	77
9.3.5	系统 CSR SYS_HW_INF3	77
9.3.6	系统 CSR SYS_HW_INF4	77
9.3.7	系统 CSR SYS_HW_INF5	78
9.3.8	系统 CSR SYS_VER0	78
9.3.9	系统 CSR SYS_VER1	78
9.3.10	系统 CSR SYS_HSM_STA0	79
9.3.11	系统 CSR SYS_HSM_STA1	79
9.3.12	系统 CSR SYS_HSM_STA2	80
9.3.13	系统 CSR SYS_HSM_STA3	80
9.3.14	系统 CSR SYS_WR_HSM_DBG_EN	80
9.3.15	系统 CSR SYS_WR_SOC_DBG_EN	80
9.3.16	System CSR SYS_WR_SOC_DBG_EN_128B0	81
9.3.17	System CSR SYS_WR_SOC_DBG_EN_128B1	81
9.3.18	System CSR SYS_WR_SOC_DBG_EN_128B2	81
9.3.19	System CSR SYS_WR_SOC_DBG_EN_128B3	81
9.3.20	系统 CSR SYS_WR_SOC_CPU_RELEASE	81
9.3.21	系统 CSR SYS_WR_SOC_CPU_RESET	82
9.3.22	系统 CSR SYS_WR_SOC_RESET	82
9.3.23	System CSR SYS_SOC_STA	82
9.3.24	System CSR SYS_SOC_STA_INT	82

9.3.25	System CSR SYS_SOC_STA_INT_EN	82
9.3.26	系统 CSR SYS_CLK_CTRL0	83
9.3.27	系统 CSR SYS_CLK_CTRL1	83
9.3.28	系统 CSR SYS_ALG_CLK_CTRL0	84
9.3.29	系统 CSR SYS_ALG_CLK_CTRL1	84
9.3.30	系统 CSR SYS_ALG_CLK_CTRL2	84
9.3.31	系统 CSR SYS_ALG_CLK_CTRL3	84
9.3.32	系统 CSR SYS_ALG_CLK_CTRL4	85
9.3.33	系统 CSR SYS_RST_CTRL0	85
9.3.34	系统 CSR SYS_RST_CTRL1	86
9.3.35	系统 CSR SYS_ALG_RST_CTRL0	86
9.3.36	系统 CSR SYS_ALG_RST_CTRL1	86
9.3.37	系统 CSR SYS_ALG_RST_CTRL2	87
9.3.38	系统 CSR SYS_ALG_RST_CTRL3	87
9.3.39	系统 CSR SYS_ALG_RST_CTRL4	87
9.3.40	系统 CSR SYS_CIPHER_CTRL	87
9.3.41	系统 CSR SYS_SOC_MEM_BAL	88
9.3.42	系统 CSR SYS_SOC_MEM_BAH	88
9.3.43	系统 CSR SYS_TRNG_DRBG_ALG	88
9.3.44	系统 CSR SYS_CPU_CFG	88
9.3.45	系统 CSR SYS_BUS_ERR_CFG	89
9.3.46	系统 CSR SYS_BUS_PR_CFG	89
9.3.47	系统 CSR SYS_AHB_DMA_CFG	89
9.3.48	系统 CSR SYS_AXI_DMA_CFG0	90
9.3.49	系统 CSR SYS_AXI_DMA_CFG1	90
9.3.50	系统 CSR SYS_AXI_DMA_CH0_CFG0	90
9.3.51	系统 CSR SYS_AXI_DMA_CH0_CFG1	91
9.3.52	系统 CSR SYS_AXI_DMA_CH0_CFG2	91
9.3.53	系统 CSR SYS_AXI_DMA_CH1_CFG0	91
9.3.54	系统 CSR SYS_AXI_DMA_CH1_CFG1	92
9.3.55	系统 CSR SYS_AXI_DMA_CH1_CFG2	92
9.3.56	系统 CSR SYS_AXI_DMA_CH2_CFG0	92
9.3.57	系统 CSR SYS_AXI_DMA_CH2_CFG1	93
9.3.58	系统 CSR SYS_AXI_DMA_CH2_CFG2	93
9.3.59	系统 CSR SYS_AXI_DMA_CH3_CFG0	93
9.3.60	系统 CSR SYS_AXI_DMA_CH3_CFG1	94
9.3.61	系统 CSR SYS_AXI_DMA_CH3_CFG2	94
9.3.62	系统 CSR SYS_CPU_ECC_CTRL0(CPU IROM)	95
9.3.63	系统 CSR SYS_CPU_ECC_CTRL1(CPU IRAM)	95
9.3.64	系统 CSR SYS_CPU_ECC_CTRL2(CPU DRAM)	95
9.3.65	系统 CSR SYS_KMU_ECC_CTRL(KMU RAM)	96
9.3.66	系统 CSR SYS_PKE_ECC_CTRL0(PKE RAM0)	96
9.3.67	系统 CSR SYS_PKE_ECC_CTRL1(PKE RAM1)	96
9.3.68	系统 CSR SYS_PKE_ECC_CTRL2(PKE RAM2)	96
9.3.69	系统 CSR SYS_PKE_ECC_CTRL3(PKE RAM3)	97
9.3.70	系统 CSR LIFE_CYCLE	97
9.3.71	系统 CSR HW_CONTROL0	97
9.3.72	系统 CSR HW_CONTROL1	97
9.3.73	系统 CSR FW_CONTROL0	98
9.3.74	系统 CSR FW_CONTROL1	98
9.3.75	系统 CSR FW_CONTROL2	98
9.3.76	系统 CSR FW_CONTROL3	98
9.3.77	系统 CSR SYS_GEN_REG[N]	98

9.3.78 系统 CSR SYS_RUN_STEP	98
10 错误管理单元	99
10.1 EMU 介绍	99
10.2 EMU 特性	99
10.3 EMU 寄存器	100
10.3.1 EMU 寄存器映射	100
10.3.2 EMU ERR_SENSOR 寄存器	102
10.3.3 EMU ERR_SOC_ERR 寄存器	102
10.3.4 EMU ERR_HW_0 寄存器	102
10.3.5 EMU ERR_HW_1 寄存器	102
10.3.6 EMU ERR_FW_0 寄存器	102
10.3.7 EMU ERR_FW_1 寄存器	102
10.3.8 EMU INT_SENSOR 寄存器	103
10.3.9 EMU INT_SOC_ERR 寄存器	103
10.3.10 EMU INT_HW_0 寄存器	103
10.3.11 EMU INT_HW_1 寄存器	103
10.3.12 EMU INTEN_SENSOR 寄存器	103
10.3.13 EMU INTEN_SOC_ERR 寄存器	103
10.3.14 EMU INTEN_HW_0 寄存器	104
10.3.15 EMU INTEN_HW_1 寄存器	104
10.3.16 EMU MEMECC_ADDR_0 寄存器	104
10.3.17 EMU MEMECC_ADDR_1 寄存器	104
10.3.18 EMU MEMECC_ADDR_2 寄存器	104
10.3.19 EMU MEMECC_ADDR_3 寄存器	105
10.3.20 EMU MEMECC_ADDR_4 寄存器	105
10.3.21 EMU MEMECC_ADDR_5 寄存器	105
10.3.22 EMU MEMECC_ADDR_6 寄存器	105
10.3.23 EMU MEMECC_ADDR_7 寄存器	105
10.3.24 EMU RST_ALL_SENSOR 寄存器	106
10.3.25 EMU RST_ALL_SOC_ERR 寄存器	106
10.3.26 EMU RST_ALL_HW_0 寄存器	106
10.3.27 EMU RST_ALL_HW_1 寄存器	106
10.3.28 EMU RST_NBT_SENSOR 寄存器	106
10.3.29 EMU RST_NBT_SOC_ERR 寄存器	107
10.3.30 EMU RST_NBT_HW_0 寄存器	107
10.3.31 EMU RST_NBT_HW_1 寄存器	107
10.3.32 EMU RST_CPU_SENSOR 寄存器	107
10.3.33 EMU RST_CPU_SOC_ERR 寄存器	107
10.3.34 EMU RST_CPU_HW_0 寄存器	108
10.3.35 EMU RST_CPU_HW_1 寄存器	108
10.3.36 EMU RST_ENABLE 寄存器	108
10.3.37 EMU RST_MODE_FLAG 寄存器	109
10.3.38 EMU ERR_HW_TRIG_0 寄存器	109
10.3.39 EMU ERR_HW_TRIG_1 寄存器	109
10.3.40 EMU ERR_TRIG_LOCK 寄存器	110
10.4 EMU 固件流程	110
10.4.1 示例：使能 WDT 复位 CPU	110

11 密钥管理单元	111
11.1 KMU 介绍	111
11.2 KMU 特性	111
11.3 KMU 密钥访问	111
11.4 KMU RAM 密钥槽	111
11.5 KMU 寄存器	111
11.5.1 KMU 寄存器映射	111
11.5.2 KMU KMU_CTRL	112
11.5.3 KMU KMU_EX_CH	112
11.5.4 KMU KMU_KEY_EN	113
11.5.5 KMU KMU_BUF[N]	113
11.5.6 KMU KMU_ATT[N]	113
12 Mailbox	114
12.1 Mailbox 介绍	114
12.2 Mailbox 特性	114
12.3 Mailbox 寄存器	115
12.3.1 Mailbox 寄存器映射	115
12.3.2 Mailbox MB_S2H_INFO_D[N] 寄存器	116
12.3.3 Mailbox MB_H2S_INFO_D[N] 寄存器	116
12.3.4 Mailbox MB_S2H_NOTE 寄存器	116
12.3.5 Mailbox MB_H2S_NOTE 寄存器	117
12.3.6 Mailbox MB_S2H_SOC_INT 寄存器	117
12.3.7 Mailbox MB_S2H_SOC_INT_EN 寄存器	117
12.3.8 Mailbox MB_S2H_HSM_INT 寄存器	117
12.3.9 Mailbox MB_S2H_HSM_INT_EN 寄存器	118
12.3.10 Mailbox MB_H2S_SOC_INT 寄存器	118
12.3.11 Mailbox MB_H2S_SOC_INT_EN 寄存器	118
12.3.12 Mailbox MB_H2S_HSM_INT 寄存器	118
12.3.13 Mailbox MB_H2S_HSM_INT_EN 寄存器	119
12.3.14 Mailbox eHSM_STATUS_0 寄存器	119
12.3.15 Mailbox eHSM_STATUS_1 寄存器	119
12.3.16 Mailbox MB_INT_ALL 寄存器	119
12.3.17 Mailbox MB_HSM_CFG0 寄存器	120
12.3.18 Mailbox MB_HSM_CFG1 寄存器	120
12.4 Mailbox 软件使用流程	120
12.4.1 主机向 eHSM 发送命令流程	120
12.4.2 eHSM 向主机发送响应流程	121
13 随机时钟	122
13.1 随机时钟寄存器	122
13.1.1 随机时钟寄存器映射	122
13.1.2 随机时钟种子寄存器	122
13.1.3 随机时钟模式寄存器	122
13.1.4 随机时钟中断寄存器	123
13.1.5 随机时钟中断使能寄存器	123
14 看门狗定时器	124
14.1 看门狗定时器用户工作流程	124
14.2 看门狗定时器寄存器	124
14.2.1 看门狗定时器寄存器映射	124
14.2.2 看门狗定时器 WDT_CTRL 寄存器	125
14.2.3 看门狗定时器 WDT_PSR 寄存器	125
14.2.4 看门狗定时器 WDT_LDR 寄存器	125

14.2.5 看门狗定时器 WDT_VDR 寄存器	125
14.2.6 看门狗定时器 WDT_ISR 寄存器	126
14.2.7 看门狗定时器 WDT_RCR 寄存器	126
14.2.8 看门狗定时器 WDT_WPT 寄存器	126
15 UART	127
15.1 UART 用户工作流程	127
15.2 UART 寄存器	127
15.2.1 UART 寄存器映射	127
15.2.2 UART 数据寄存器	127
15.2.3 UART 状态寄存器	128
15.2.4 UART 控制寄存器	128
15.2.5 UART 中断状态寄存器	128
15.2.6 UART 速率寄存器	128
16 CRC	129
16.1 CRC 用户工作流程	129
16.2 CRC 寄存器	129
16.2.1 CRC 寄存器映射	129
16.2.2 CRC 算法配置寄存器	130
16.2.3 CRC 数据 IQ 寄存器	130
16.2.4 LFSR 种子数据寄存器	130
16.2.5 LFSR KICK 寄存器	131
17 通用定时器	132
17.1 通用定时器工作流程	132
17.2 通用定时器寄存器	132
17.2.1 通用定时器寄存器	132
17.2.2 通用定时器 TIM_CTL0 寄存器	133
17.2.3 通用定时器 TIM_CTL1 寄存器	133
17.2.4 通用定时器 TIM_PRS0 寄存器	133
17.2.5 通用定时器 TIM_PRS1 寄存器	133
17.2.6 通用定时器 TIM_INIT0 寄存器	134
17.2.7 通用定时器 TIM_INIT1 寄存器	134
17.2.8 通用定时器 TIM_CNT0 寄存器	134
17.2.9 通用定时器 TIM_CNT1 寄存器	134
17.2.10 通用定时器 TIM_STA0 寄存器	134
17.2.11 通用定时器 TIM_STA1 寄存器	135
18 集成	136
18.1 发布数据库	136
18.2 eHSM 宏定义	137
18.2.1 eHSM RTL Key	137
18.2.2 eHSM 仿真宏	138
18.2.3 eHSM 配置宏	138
18.3 ROM Code	139
18.3.1 生成 ROM Code	139
18.4 标准单元库	140
18.4.1 存储器	140
18.4.2 标准单元	141
18.5 仿真	141
18.6 TRNG 仿真	141
18.7 Tape-out 检查清单	142

OSR CONFIDENTIAL

List of Figures

1	包含 eHSM 的 SoC 框图	20
2	eHSM 系统架构图	23
3	ICG 模块	25
4	ICG 时序图示例	25
5	简化的 ICG 模块	25
6	两级同步器	25
7	简化的两级同步器	26
8	异步复位同步模块	26
9	简化的异步复位同步模块	26
10	时钟生成模块	27
11	复位生成模块	28
12	eHSM 启动完成/错误时序图	34
13	eHSM 启动期间生命周期信号时序图	34
14	eHSM IROM 读时序图	41
15	eHSM IRAM 读时序图	41
16	eHSM IRAM 写时序图	42
17	eHSM PKE RAM 读写时序图	43
18	eHSM 密钥层级	52
19	eHSM 硬件启动流程图	66
20	eHSM 硬件获取随机数流程图	67
21	eHSM 安全调试架构图	68
22	KMU RAM 密钥槽	111
23	Mailbox 模块框图	114

List of Tables

1	约定	16
2	寄存器字段类型	16
3	不带 Memory ECC 的 eHSM 存储	29
4	带有 Memory ECC 的 eHSM 存储	29
5	时钟与复位接口	31
6	状态与错误接口	31
7	i_sensor 释义	32
8	i_soc_status Definition	32
9	i_soc_err Definition	32
10	o_hsm_status 释义	32
10	o_hsm_status 释义	33
11	o_hsm_err_hw 释义	35
12	o_hsm_err_fw 释义	35
13	eHSM IO Signals: 128 Bit Debug Enable	36
14	Mailbox AHB Slave 接口	36
15	AHB Master 接口	37
16	DMA AXI Master 写通道接口	38
17	DMA AXI Master 读通道接口	39
18	NVM AHB Master 接口	39
19	CFG AHB Master 接口	40
20	OTP AHB Master 接口	40
21	IROM 接口	41
22	IRAM 接口	41
23	DRAM 接口	42
24	KMU RAM 接口	42
25	PKE RAM 接口 (Dual Port RAM)	43
26	PKE RAM 接口 (Single Port RAM)	43
27	Debug 接口	44
28	UART 接口	44
29	TRNG 测试接口	44
30	DFT 接口	44
31	Mailbox 中断接口	45
32	块边界地址	46
33	eHSM CPU 的中断源	47
34	OTP 内存映射地址	49
35	OTP 硬件控制字段	50
36	OTP 传感器响应配置	51
37	eHSM Key	51
38	OTP Key 字段划分	53
39	eHSM 密钥属性 (OTP 位的默认值为 1 的情形)	54
40	eHSM 密钥属性 (OTP 位的默认值为 0 的情形)	55
41	eHSM 密钥的有效性	55
42	OTP 密钥安装	57
42	OTP 密钥安装	58
43	生命周期概述	60
44	生命周期 (不带 ECC) 定义 (OTP 位的默认值为 1 的情形)	60
45	生命周期 (不带 ECC) 定义 (OTP 位的默认值为 0 的情形)	61
46	生命周期中的功能控制	61
47	生命周期中的 OTP 密钥解密	62
48	生命周期中的 KMU 密钥访问权限	63
49	生命周期中的 eHSM 硬件启动错误 (由无效密钥触发)	64

50	生命周期中的 eHSM 硬件启动错误（由其他条件触发）	64
51	系统 CSR 寄存器映射	74
51	系统 CSR 寄存器映射	75
52	系统 CSR SYS_HW_INF0 (Register offset: 0x0)	76
53	系统 CSR SYS_HW_INF1 (Register offset: 0x4)	76
54	系统 CSR SYS_HW_INF2 (Register offset: 0x8)	77
55	系统 CSR SYS_HW_INF3 (Register offset: 0xC)	77
56	系统 CSR SYS_HW_INF4 (Register offset: 0x10)	77
57	系统 CSR SYS_HW_INF5 (Register offset: 0x14)	78
58	系统 CSR SYS_VER0 (Register offset: 0x80)	78
59	系统 CSR SYS_VER1 (Register offset: 0x84)	78
60	系统 CSR SYS_HSM_STA0 (Register offset: 0x1000)	79
61	系统 CSR SYS_HSM_STA1 (Register offset: 0x1004)	79
62	系统 CSR SYS_HSM_STA2 (Register offset: 0x1008)	80
63	系统 CSR SYS_HSM_STA3 (Register offset: 0x100C)	80
64	系统 CSR SYS_WR_HSM_DBG_EN (Register offset: 0x1080)	80
65	系统 CSR SYS_WR_SOC_DBG_EN (Register offset: 0x1100)	80
66	系统 CSR SYS_WR_SOC_DBG_EN_128B0 (Register offset: 0x1104)	81
67	系统 CSR SYS_WR_SOC_DBG_EN_128B1 (Register offset: 0x1108)	81
68	系统 CSR SYS_WR_SOC_DBG_EN_128B2 (Register offset: 0x110C)	81
69	系统 CSR SYS_WR_SOC_DBG_EN_128B3 (Register offset: 0x1110)	81
70	系统 CSR SYS_WR_SOC_CPU_RELEASE (Register offset: 0x1200)	81
71	系统 CSR SYS_WR_SOC_CPU_RESET (Register offset: 0x1240)	82
72	系统 CSR SYS_WR_SOC_RESET (Register offset: 0x1280)	82
73	System CSR SYS_SOC_STA (Register offset: 0x2000)	82
74	System CSR SYS_SOC_STA_INT (Register offset: 0x2800)	82
75	System CSR SYS_SOC_STA_INT_EN (Register offset: 0x2880)	82
76	系统 CSR SYS_CLK_CTRL0 (Register offset: 0x3000)	83
77	系统 CSR SYS_CLK_CTRL1 (Register offset: 0x3004)	83
78	系统 CSR SYS_ALG_CLK_CTRL0 (Register offset: 0x3100)	84
79	系统 CSR SYS_ALG_CLK_CTRL1 (Register offset: 0x3104)	84
80	系统 CSR SYS_ALG_CLK_CTRL2 (Register offset: 0x3108)	84
81	系统 CSR SYS_ALG_CLK_CTRL3 (Register offset: 0x310C)	84
82	系统 CSR SYS_ALG_CLK_CTRL4 (Register offset: 0x3110)	85
83	系统 CSR SYS_RST_CTRL0 (Register offset: 0x3200)	85
84	系统 CSR SYS_RST_CTRL1 (Register offset: 0x3204)	86
85	系统 CSR SYS_ALG_RST_CTRL0 (Register offset: 0x3300)	86
86	系统 CSR SYS_ALG_RST_CTRL1 (Register offset: 0x3304)	86
87	系统 CSR SYS_ALG_RST_CTRL2 (Register offset: 0x3308)	87
88	系统 CSR SYS_ALG_RST_CTRL3 (Register offset: 0x330C)	87
89	系统 CSR SYS_ALG_RST_CTRL4 (Register offset: 0x3310)	87
90	系统 CSR SYS_CIPHER_CTRL (Register offset: 0x3400)	87
91	系统 CSR SYS_SOC_MEM_BAL (Register offset: 0x3800)	88
92	系统 CSR SYS_SOC_MEM_BAH (Register offset: 0x3804)	88
93	系统 CSR SYS_TRNG_DRBG_ALG (Register offset: 0x3900)	88
94	系统 CSR SYS_CPU_CFG (Register offset: 0x4000)	88
95	系统 CSR SYS_BUS_ERR_CFG (Register offset: 0x5000)	89
96	系统 CSR SYS_BUS_PR_CFG (Register offset: 0x5100)	89
97	系统 CSR SYS_AHB_DMA_CFG (Register offset: 0x6000)	89
98	系统 CSR SYS_AXI_DMA_CFG0 (Register offset: 0x6800)	90
99	系统 CSR SYS_AXI_DMA_CFG1 (Register offset: 0x6804)	90
100	系统 CSR SYS_AXI_DMA_CH0_CFG0 (Register offset: 0x6880)	90
101	系统 CSR SYS_AXI_DMA_CH0_CFG1 (Register offset: 0x6884)	91

102	系统 CSR SYS_AXI_DMA_CH0_CFG2 (Register offset: 0x6888)	91
103	系统 CSR SYS_AXI_DMA_CH1_CFG0 (Register offset: 0x68A0)	91
104	系统 CSR SYS_AXI_DMA_CH1_CFG1 (Register offset: 0x68A4)	92
105	系统 CSR SYS_AXI_DMA_CH1_CFG2 (Register offset: 0x68A8)	92
106	系统 CSR SYS_AXI_DMA_CH2_CFG0 (Register offset: 0x68C0)	92
107	系统 CSR SYS_AXI_DMA_CH2_CFG1 (Register offset: 0x68C4)	93
108	系统 CSR SYS_AXI_DMA_CH2_CFG2 (Register offset: 0x68C8)	93
109	系统 CSR SYS_AXI_DMA_CH3_CFG0 (Register offset: 0x68E0)	93
110	系统 CSR SYS_AXI_DMA_CH3_CFG1 (Register offset: 0x68E4)	94
111	系统 CSR SYS_AXI_DMA_CH3_CFG2 (Register offset: 0x68E8)	94
112	系统 CSR SYS_CPU_ECC_CTRL0(CPU IROM) (Register offset: 0x7000)	95
113	系统 CSR SYS_CPU_ECC_CTRL1(CPU IRAM) (Register offset: 0x7004)	95
114	系统 CSR SYS_CPU_ECC_CTRL2(CPU DRAM) (Register offset: 0x7008)	95
115	系统 CSR SYS_KMU_ECC_CTRL(KMU RAM) (Register offset: 0x7100)	96
116	系统 CSR SYS_PKE_ECC_CTRL0(PKE RAM0) (Register offset: 0x7200)	96
117	系统 CSR SYS_PKE_ECC_CTRL1(PKE RAM1) (Register offset: 0x7204)	96
118	系统 CSR SYS_PKE_ECC_CTRL2(PKE RAM2) (Register offset: 0x7208)	96
119	系统 CSR SYS_PKE_ECC_CTRL3(PKE RAM3) (Register offset: 0x720C)	97
120	系统 CSR LIFE_CYCLE (Register offset: 0x8000)	97
121	系统 CSR HW_CONTROL0 (Register offset: 0x8080)	97
122	系统 CSR HW_CONTROL1 (Register offset: 0x8084)	97
123	系统 CSR FW_CONTROL0 (Register offset: 0x8100)	98
124	系统 CSR FW_CONTROL1 (Register offset: 0x8104)	98
125	系统 CSR FW_CONTROL2 (Register offset: 0x8108)	98
126	系统 CSR FW_CONTROL3 (Register offset: 0x810C)	98
127	系统 CSR SYS_GEN_REG[N] (Register offset: 0xF0800)	98
128	系统 CSR SYS_RUN_STEP (Register offset: 0xF0F00)	98
129	EMU 寄存器映射	101
130	EMU ERR_SENSOR 寄存器	102
131	EMU ERR_SOC_ERR 寄存器	102
132	EMU ERR_HW_0 寄存器	102
133	EMU ERR_HW_1 寄存器	102
134	EMU ERR_FW_0 寄存器	102
135	EMU ERR_FW_1 寄存器	102
136	EMU INT_SENSOR 寄存器	103
137	EMU INT_SOC_ERR 寄存器	103
138	EMU INT_HW_0 寄存器	103
139	EMU INT_HW_1 寄存器	103
140	EMU INTEN_SENSOR 寄存器	103
141	EMU INTEN_SOC_ERR 寄存器	103
142	EMU INTEN_HW_0 寄存器	104
143	EMU INTEN_HW_1 寄存器	104
144	EMU MEMECC_ADDR_0 寄存器	104
145	EMU MEMECC_ADDR_1 寄存器	104
146	EMU MEMECC_ADDR_2 寄存器	104
147	EMU MEMECC_ADDR_3 寄存器	105
148	EMU MEMECC_ADDR_4 寄存器	105
149	EMU MEMECC_ADDR_5 寄存器	105
150	EMU MEMECC_ADDR_6 寄存器	105
151	EMU MEMECC_ADDR_7 寄存器	105
152	EMU RST_ALL_SENSOR 寄存器	106
153	EMU RST_ALL_SOC_ERR 寄存器	106
154	EMU RST_ALL_HW_0 寄存器	106

155	EMU_RST_ALL_HW_1 寄存器	106
156	EMU_RST_NBT_SENSOR 寄存器	106
157	EMU_RST_NBT_SOC_ERR 寄存器	107
158	EMU_RST_NBT_HW_0 寄存器	107
159	EMU_RST_NBT_HW_1 寄存器	107
160	EMU_RST_CPU_SENSOR 寄存器	107
161	EMU_RST_CPU_SOC_ERR 寄存器	107
162	EMU_RST_CPU_HW_0 寄存器	108
163	EMU_RST_CPU_HW_1 寄存器	108
164	EMU_RST_ENABLE 寄存器	108
165	EMU_RST_MODE_FLAG 寄存器	109
166	EMU_ERR_HW_TRIG_0 寄存器	109
167	EMU_ERR_HW_TRIG_1 寄存器	109
168	EMU_ERR_TRIG_LOCK 寄存器	110
169	KMU 寄存器映射	112
170	KMU_KMU_CTRL	112
171	KMU_KMU_EX_CH	112
172	KMU_KMU_KEY_EN	113
173	KMU_KMU_BUF[N]	113
174	KMU_KMU_ATT[N]	113
175	Mailbox 选择	115
176	Mailbox 寄存器列表	116
177	Mailbox MB_S2H_INFO_D[N] 寄存器	116
178	Mailbox MB_H2S_INFO_D[N] 寄存器	116
179	Mailbox MB_S2H_NOTE 寄存器	117
180	Mailbox MB_H2S_NOTE 寄存器	117
181	Mailbox MB_S2H_SOC_INT 寄存器	117
182	Mailbox MB_S2H_SOC_INT_EN 寄存器	117
183	Mailbox MB_S2H_HSM_INT 寄存器	118
184	Mailbox MB_S2H_HSM_INT_EN 寄存器	118
185	Mailbox MB_H2S_SOC_INT 寄存器	118
186	Mailbox MB_H2S_SOC_INT_EN 寄存器	118
187	Mailbox MB_H2S_HSM_INT 寄存器	119
188	Mailbox MB_H2S_HSM_INT_EN 寄存器	119
189	Mailbox eHSM_STATUS_0 寄存器	119
190	Mailbox eHSM_STATUS_1 寄存器	119
191	Mailbox MB_INT_ALL 寄存器	119
192	Mailbox MB_HSM_CFG0 寄存器	120
193	Mailbox MB_HSM_CFG1 寄存器	120
194	随机时钟寄存器映射	122
195	随机时钟种子寄存器	122
196	随机时钟模式寄存器	122
197	随机时钟中断寄存器	123
198	随机时钟中断使能寄存器	123
199	看门狗定时器寄存器映射	124
200	看门狗定时器 WDT_CTRL 寄存器	125
201	看门狗定时器 WDT_PSR 寄存器	125
202	看门狗定时器 WDT_LDR 寄存器	125
203	看门狗定时器 WDT_VDR 寄存器	125
204	看门狗定时器 WDT_ISR 寄存器	126
205	看门狗定时器 WDT_RCR 寄存器	126
206	看门狗定时器 WDT_WPT 寄存器	126
207	UART 寄存器映射	127

208	UART_DATA 寄存器	127
209	UART_STATUS 寄存器	128
210	UART_CTRL 寄存器	128
211	UART_INT_STA 寄存器	128
212	UART_BAUD_DIV 寄存器	128
213	CRC 寄存器映射	129
214	CRC_Alg_Config 寄存器	130
215	CRC_IO_DATA 寄存器	130
216	LFSR_SEED_DATA 寄存器	130
217	LFSR_KICK 寄存器	131
218	通用定时器寄存器映射	132
219	通用定时器 TIM_CTL0 寄存器	133
220	通用定时器 TIM_CTL1 寄存器	133
221	通用定时器 TIM_PRS0 寄存器	133
222	通用定时器 TIM_PRS1 寄存器	133
223	通用定时器 TIM_INIT0 寄存器	134
224	通用定时器 TIM_INIT1 寄存器	134
225	通用定时器 TIM_CNT0 寄存器	134
226	通用定时器 TIM_CNT1 寄存器	134
227	通用定时器 TIM_STA0 寄存器	134
228	通用定时器 TIM_STA1 寄存器	135
229	eHSM Database	136
229	eHSM Database	137
230	eHSM RTL Key 宏	138
231	eHSM 仿真宏	138
232	eHSM 配置宏	139
233	Tape-out 检查清单	142
234	历史版本	143

1 前言

此前言包括以下几个部分：

- [关于本手册](#)
- [客户支持](#)

OSR CONFIDENTIAL

1.1 关于本手册

本手册介绍了 OSR 产品系列的成员 eHSM。通过使用本手册，您将了解它是什么以及如何将其集成到目标 SOC 中。

1.1.1 目标受众

本手册是为有经验的硬件和软件工程师编写的 SOC 安全系统白皮书。在使用本手册之前，您应该熟悉以下主题：

- 密码学标准，例如 GM/T 标准
- 安全系统，例如 ARM 平台安全架构

1.1.2 约定

Table 1: 约定

约定	描述
Table 1	超链接
<code>cd sim</code>	终端命令；表示进入到 <code>sim</code> 目录，在 C，Verilog，脚本等代码中使用
用户必须...	强调
w/	英文“with”的简写
w/o	英文“without”的简写

Table 2: 寄存器字段类型

寄存器字段类型	描述
-	无或未定义
Res	保留位，不可写入且读出为 0
RO	只读寄存器，不能写入
RW	可读可写寄存器
RWP	可读可写寄存器，但存在写保护
WO	只写寄存器，读出为 0
RW1C	可读可写寄存器，写入 1 清零，写入 0 不产生任何影响
RW0C	可读可写寄存器，写入 0 清零，写入 1 不产生任何影响
RW1S	可读可写寄存器，写入 1 置高，写入 0 不产生任何影响
RW1T	可读可写寄存器，写入 1 置高，写入 0 不产生任何影响，用于触发事件
W1T	只写寄存器，写入 1 产生一个周期的高电平脉冲，写入 0 对该寄存器值没有影响
W5AT	只写寄存器，写入 0x5A 触发。W65T、W93T、WA6T、WC9T 同理

1.1.3 术语表

AAD Additional Authenticated Data

ACE AXI Coherency Extensions

AES	Advanced Encryption Standard, FIPS197
AHB	Advanced High Performance Bus
AMBA	Advanced Microcontroller Bus Architecture
AP	Application Processor
AXI	Advanced eXtensible Interface
CBC	Cipher Block Chaining, NIST SP800-38A
CCM	Counter with Cipher Block Chaining-Message Authentication Code, NIST SP800-38C
CMAC	Cipher-based Message Authentication Code, NIST SP800-38B
CBC-MAC	Cipher Block Chaining-Message Authentication Code, NIST SP800-38C
CFB	Cipher Feedback, NIST SP800-38A, CFB128
CTR	Counter, defined by NIST SP800-38A
CPA	Correlation Power Analysis
CRC	Cyclic Redundancy Check
DES	Data Encryption Standard, defined by FIPS 46-3
3DES	Triple DES, defined by FIPS 46-3
DFA	Differential Fault-injection Attack
DPA	Differential Power Analysis
FIA	Fault Injection Attack
ECB	Electronic Code Book, defined by NIST SP800-38A
ECC	Error Check and Correction
EDE	Encrypt-decrypt-Encrypt
eHSM	embedded Hardware Security Module
EM	Emission
FA	Fault attack
FAE	Field Applications Engineer
FI	Fault inject
FIFO	First In First Out
GCM	Galois/Counter Mode, defined by NIST SP800-38D
HP	High Performance

IV	Initial Vector
KDEC	Key Decode
NIST SP800-90A	Recommendation for Random Number Generation Using Deterministic Random Bit Generators.
NIST SP800-90B	Recommendation for the Entropy Sources Used for Random Bit Generators.
NIST SP800-90C	Recommendation for Random Bit Generator (RBG) Constructions.
OFB	Output Feedback, define dy NIST SP800-38A
OSR	Open Security Research, Inc.
PKE	Public Key Engine
RISC-V	A Instruction-set architecture (ISA)
SEip	Secure Enclave IP
SKE	Symmetric Key Engine
SM	Safety Mechanism
SM4	A block cipher used in the Standard, GM/T 0002-2012
SOC	System on chip
TBC	To Be Confirmed
TBD	To Be Define
TRNG	True Random Number Generator
XTS	XEX encryption mode with tweak and ciphertext stealing, defined by IEEE1619-2007

1.2 客户支持

通过联系 OSR 的 FAE 团队可以获得客户支持。OSR 欢迎对该产品及其文档的反馈。如果您有任何问题或建议，请随时与我们联系。

公司网址

<https://www.osr-tech.com>

OSR CONFIDENTIAL

2 eHSM 介绍

eHSM 是设备上的信任根，并为关键安全功能提供了与系统其余部分的硬件隔离。

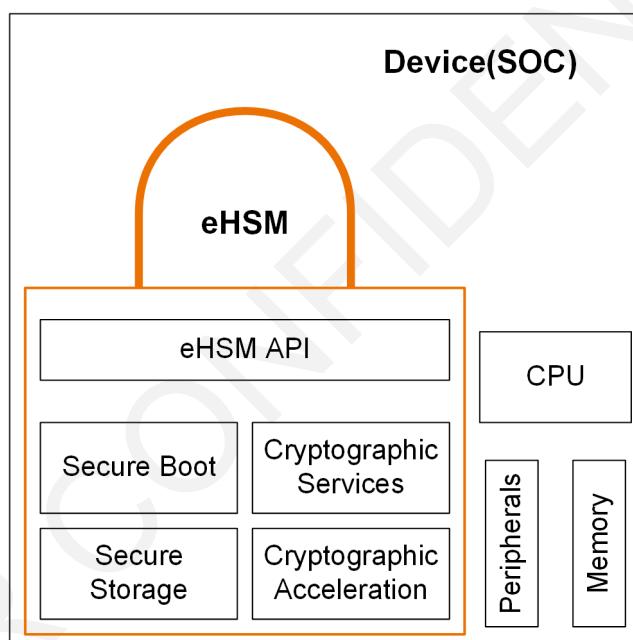
eHSM 拥有完整的硬件单元，如时钟、复位、启动和总线系统，一个 CPU 核及其关联内存，一套完整的高性能加密算法引擎系统，以及一些通用外设。复位释放后，eHSM 将完成自身硬件系统的自检。

eHSM 为设备（SoC）提供与信息安全相关的服务，如安全存储、安全启动、安全生命周期和密钥管理。

设备（SoC）通过 Mailbox 与 eHSM 通信，然后使用 eHSM 提供的密码服务（TRNG、HASH、SKE 和 PKE）。

除了 Mailbox 外，eHSM 提供了丰富的硬件接口，用于传输传感器，状态（警告和错误）信号以及设备（SoC）密钥（通过安全端口）。

Figure 1: 包含 eHSM 的 SoC 框图



2.1 功能列表

eHSM 拥有以下功能:

- 安全存储
 - 密钥存储
 - 敏感信息存储
 - 固件镜像存储
- 安全生命周期
- 安全启动
- 安全调试
- 密码服务
 - *TRNG*
 - *HASH*
 - *SKE*
 - *PKE*
- 自检
- 系统 *CSR*
- 错误管理单元
- 密钥管理单元
- *Mailbox*
- 外设
 - 随机时钟
 - *UART*
 - 看门狗定时器
 - 通用定时器
 - *CRC*

2.2 系统架构

主系统由 32 位多层 AHB 总线矩阵组成，连接如下：

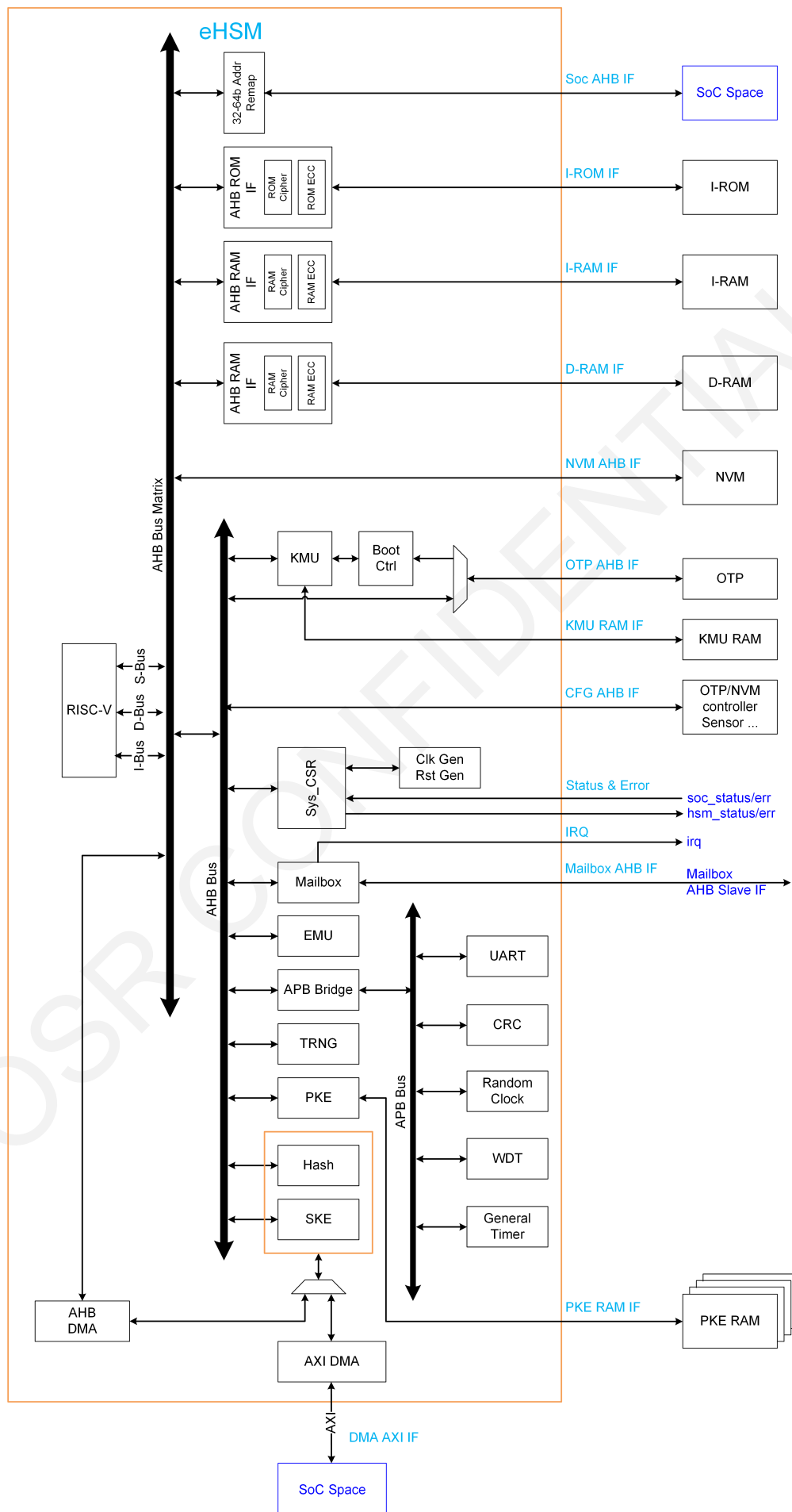
主设备

- CPU I-Bus (指令总线)
- CPU D-Bus (数据总线)
- CPU S-Bus (系统总线)
- AHB DMA

从设备

- IROM
- IRAM
- SoC 内存
- DRAM
- NVM
- AHB 总线
 - OTP
 - KMU
 - 系统寄存器
 - Mailbox
 - TRNG
 - PKE
 - SKE
 - HASH
 - AHB-APB 转换桥
 - 随机时钟
 - UART
 - 看门狗定时器
 - 通用定时器
 - CRC

Figure 2: eHSM 系统架构图



Note: 功能列表 和 系统架构部分中的架构是 eHSM 的完整版本。实际架构取决于客户的需求。

- 以下功能是国密二级要求的:
 - 固件 应不可升级: eHSM 的固件将直接存储在 IROM 中，也可以存储在设计上只读的 NVM 中。
 - 传感器
 - 温度传感器
 - 电压传感器
 - 电压毛刺传感器
 - 频率检测传感器
 - 主动屏蔽传感器
 - 随机时钟
 - RAM 加密
 - ROM 加密
 - 总线加密
 - 针对侧信道分析和故障注入攻击的加密引擎

2.3 时钟和复位架构

2.3.1 子模块

2.3.1.1 ICG

ICG 标准单元用于 eHSM 中的时钟门控。ICG 的实现图如下。

Figure 3: ICG 模块

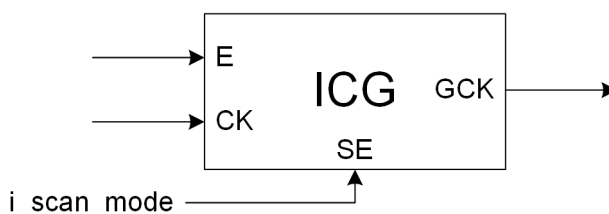
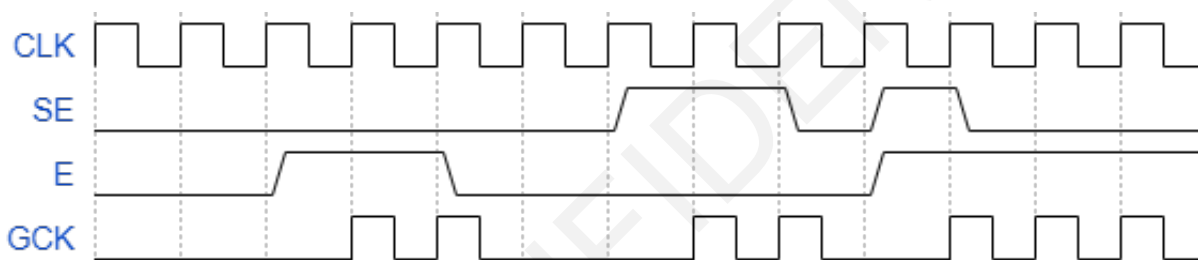


Figure 4: ICG 时序图示例



eHSM 中 ICG 的所有 SE 端口都连接到顶层 i_scan_mode。因此，ICG 的 SE 端口连接不再在下面的图中反映。

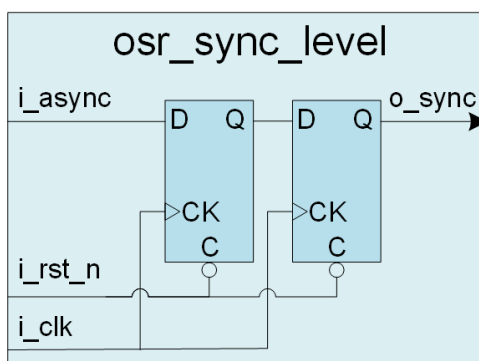
ICG 单元将被简化为下图。

Figure 5: 简化的 ICG 模块



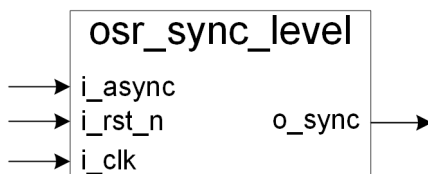
2.3.1.2 两级同步器

Figure 6: 两级同步器



osr_sync_level 模块将被简化为以下的单元图。

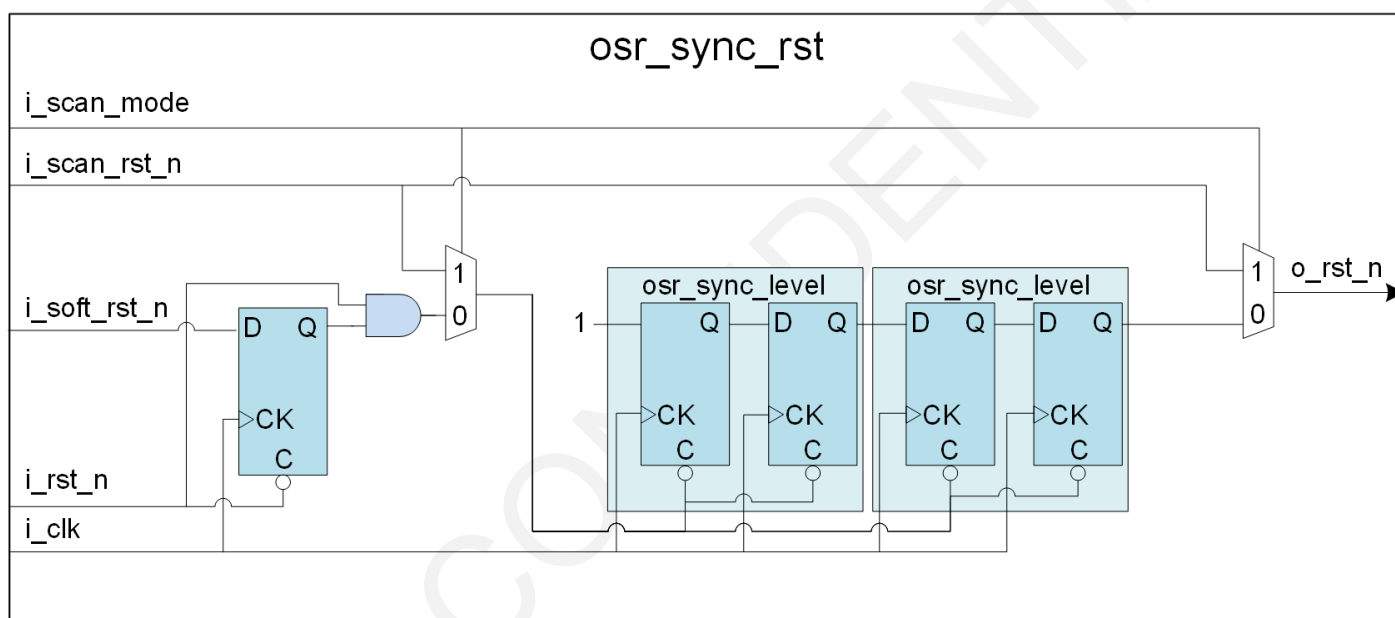
Figure 7: 简化的两级同步器



2.3.1.3 异步复位同步模块

异步复位同步模块用于 eHSM 中的异步复位信号。该模块的设计如下。

Figure 8: 异步复位同步模块

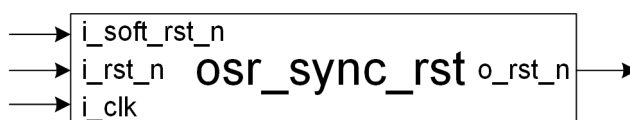


eHSM 中 osr_sync_rst 的所有 **i_scan_mode** 端口都连接到顶层 **i_scan_mode**。因此，osr_sync_rst 的 i_scan_mode 连接不再在下面的图中反映。

eHSM 中 osr_sync_rst 的所有 **i_scan_rst_n** 端口都连接到顶层 **i_rst_n**。因此，osr_sync_rst 的 i_scan_rst_n 连接不再在下面的图中反映。

osr_sync_rst 模块将被简化为以下单元图。

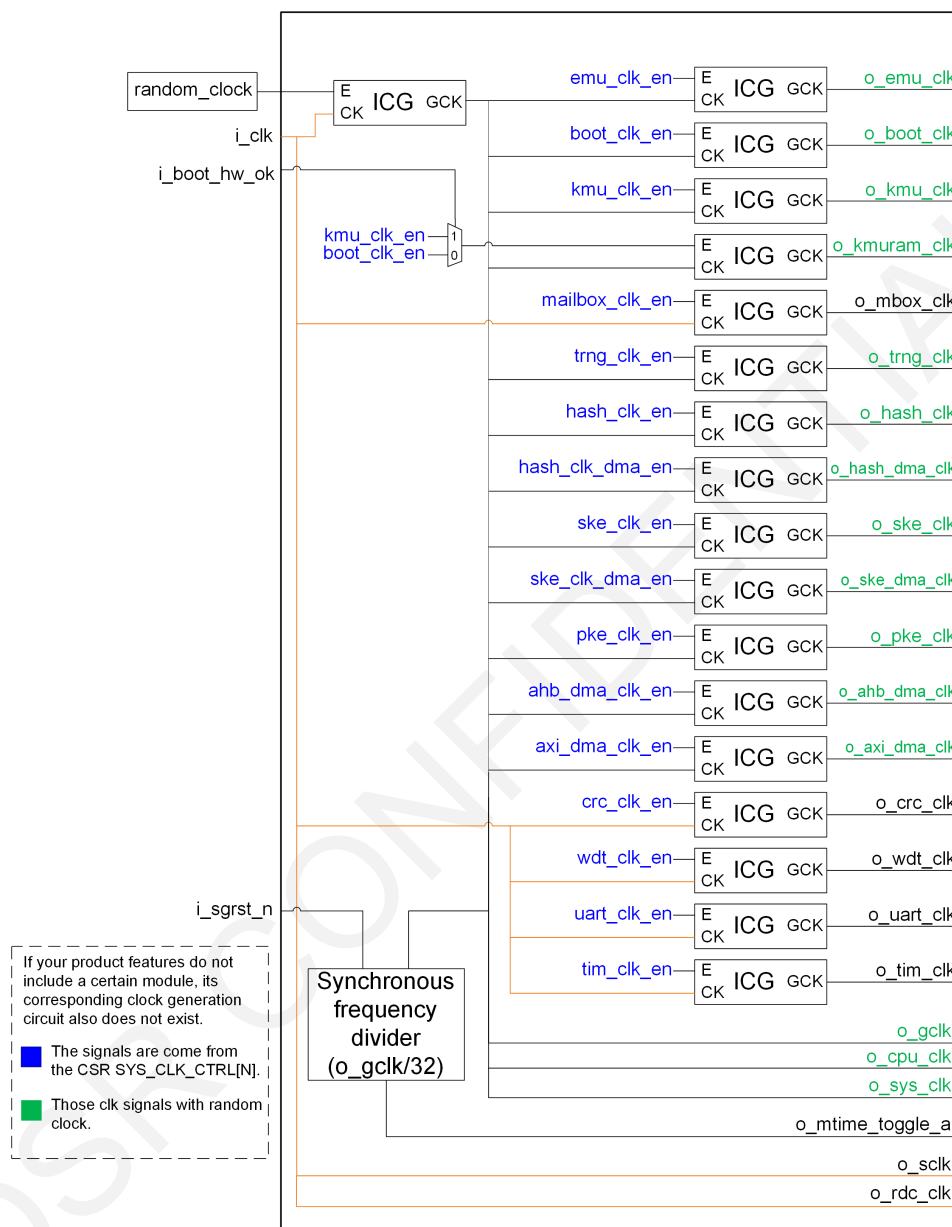
Figure 9: 简化的异步复位同步模块



2.3.2 CGU 架构

CGU 是 eHSM 的时钟生成模块，其架构如下图所示：

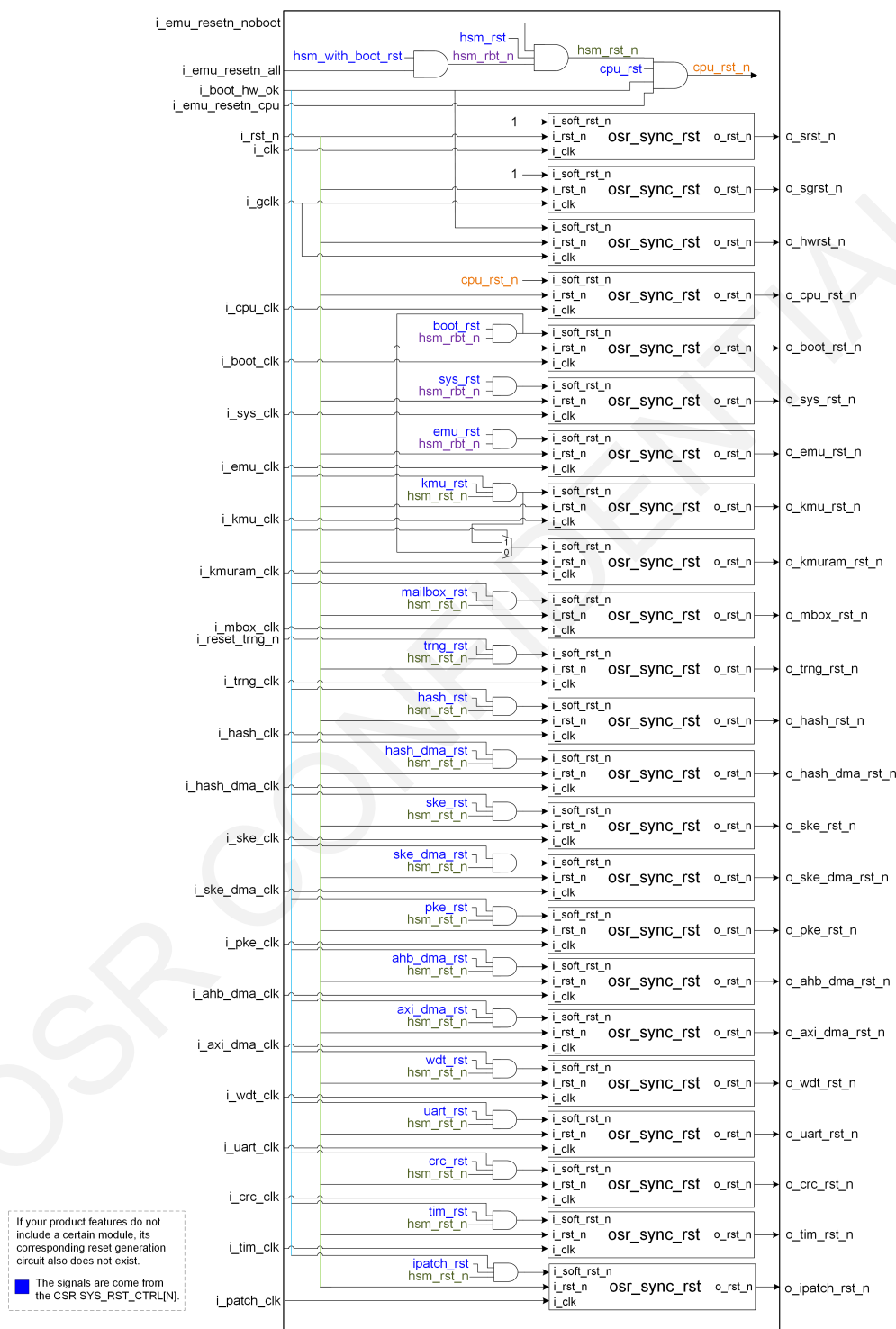
Figure 10: 时钟生成模块



2.3.3 RGU 架构

RGU 是 eHSM 的复位生成模块，其架构如下图所示：

Figure 11: 复位生成模块



2.4 存储资源需求

eHSM 需要以下存储资源：

Table 3: 不带 Memory ECC 的 eHSM 存储

Name	Type	Max Size	Data Width	Description
IROM	ROM	512KB	32	Bootloader（引导程序）
DRAM	Single Port RAM	512KB	32	CPU Data RAM（CPU 数据 RAM）
IRAM	Single Port RAM	512KB	32	CPU Instruction RAM（CPU 指令 RAM）
KMU_RAM	Single Port RAM	1KB	32	最大可存储 25 个密钥的 KMU RAM
PKE_RAM	Single or Dual Port RAM	PKE_UHP: 3KB x 4 PKE_HP: 3KB x 4 PKE_LP: 6KB x 2	PKE_UHP: 64 PKE_HP: 64 PKE_LP: 32	PKE 计算用 RAM
OTP	OTP	1KB	/	存储生命周期、版本信息、OTP 密钥等
NVM	NVM	4096KB	/	eHSM 固件代码和 eHSM 数据。如果支持 dual bank 模式空间将加倍

Table 4: 带有 Memory ECC 的 eHSM 存储

Name	Type	Max Size	Data Width	Description
IROM	ROM	512KB	32+7	Bootloader
DRAM	Single Port RAM	512KB	32+7	CPU Data RAM（CPU 数据 RAM）
IRAM	Single Port RAM	512KB	32+7	CPU Instruction RAM（CPU 指令 RAM）
KMU_RAM	Single Port RAM	1KB	32+7	最大可存储 25 个密钥的 KMU RAM
PKE_RAM	Single or Dual Port RAM	PKE_UHP: 3KB x 4 PKE_HP: 3KB x 4 PKE_LP: 6KB x 2	PKE_UHP: 64+8 PKE_HP: 64+8 PKE_LP: 32+7	PKE 计算用 RAM
OTP	OTP	1KB	/	存储生命周期、版本信息、OTP 密钥等
NVM	NVM	4096KB	/	eHSM 固件代码和 eHSM 数据。如果支持 dual bank 模式空间将加倍

Note:

1. 实际的存储大小取决于所支持的功能。
2. 由于 RAM Cipher 模块的限制，IRAM 的大小必须为 4KB 的整数倍，IROM 的大小必须为 8KB 的整数倍。
3. 如果支持 Memory ECC 功能，内存数据宽度包括数据（32 位）和可选的 ECC 校验位（7 位）。
4. 如果 eHSM 固件直接在 IROM 或 NVM 中运行，则不需要 IRAM。

5. KMU RAM 大小取决于存储的密钥数量，当前假设为 25 个 OTP 密钥。
6. PKE RAM 大小取决于支持的算法。
7. PKE 可以根据 RAM 类型选择和性能考虑，支持使用单端口 RAM 或双端口 RAM。当选择双端口 RAM 时，PKE_RAM 的一个端口仅用于写入，另一个端口仅用于读取。这两个端口不会同时访问同一地址。
8. 如果使用单端口 RAM，算法性能将减少约双端口 RAM 版本的 1/3。

2.5 IO 信号

eHSM 具有以下 IO 信号：

- 时钟与复位接口 [Table 5](#)
- 状态与错误接口 [Table 6](#)
- 128 位 Debug 使能接口 [Table 13](#)
- Mailbox AHB Slave 接口 [Table 14](#)
- AHB Master 接口 [Table 15](#)
- DMA AXI Master 接口 [Table 16](#) [Table 17](#)
- NVM AHB Master 接口 [Table 18](#)
- CFG AHB Master 接口 [Table 19](#)
- OTP AHB Master 接口 [Table 20](#)
- IROM 接口 [Table 21](#)
- IRAM 接口 [Table 22](#)
- DRAM 接口 [Table 23](#)
- KMU RAM 接口 [Table 24](#)
- PKE RAM 接口 [Table 26](#)
- 调试接口 [Table 27](#)
- UART 接口 [Table 28](#)
- TRNG 测试接口 [Table 29](#)
- DFT 接口 [Table 30](#)
- Mailbox 中断接口 [Table 31](#)

2.5.1 时钟与复位接口

Table 5: 时钟与复位接口

Signal	Direction	Width	Description
i_clk	In	1	SoC 时钟输入
i_rst_n	In	1	对 i_clk 的复位输入, 低有效

2.5.2 状态与错误接口

Table 6: 状态与错误接口

Signal	Direction	Width	Description
SoC to eHSM			
i_sensor	In	32	传感器告警输入, 参见 Table 7
i_soc_status	In	32	SoC 到 eHSM 的状态输入, 参见 Table 8
i_soc_err	In	32	SoC 到 eHSM 的错误信号输入, 参见 Table 9
eHSM to SoC			
o_hsm_status	Out	64	eHSM to SoC : 状态, 参见 Table 10
o_hsm_err_sensor	Out	32	eHSM to SoC : 传感器检测到的错误, 每一位与 i_sensor 有相同的定义
o_hsm_err_hw	Out	64	eHSM to SoC : 硬件检测到的错误, 参见 Table 11
o_hsm_err_fw	Out	64	eHSM to SoC : 固件检测到的错误, 参见 Table 12

Note: i_sensor 和 o_hsm_err_sensor 仅适用于满足国密二级认证的安全版本。

i_sensor 用于将传感器因攻击而触发的报警信息通知给 eHSM。位的顺序和定义取决于 SoC 实现。下表为一个示例。

Table 7: i_sensor 释义

Bits	Field	Description
<i>reserved</i>		
31:16	<i>reserved</i>	-
Active Shield Sensor		
15:13	<i>reserved</i>	-
12	active_shield	主动屏蔽失败
Frequency Detect Sensor		
11:10	<i>reserved</i>	-
9	fd_high	频率过高
8	fd_low	频率过低
Voltage Sensor		
7	<i>reserved</i>	-
6	volt_glitch	电压异常
5	volt_high	电压过高
4	volt_low	电压过低
Temperature Sensor		
3:2	<i>reserved</i>	-
1	temp_high	温度过高
0	temp_low	温度过低

Note:

i_sensor 输入信号的每一位都应该为电平信号。

下表 (Table 8, Table 9) 详述了 SoC 到 eHSM 状态和错误信号的定义。

Table 8: i_soc_status Definition

Bits	Field	Description
31:0	<i>reserved</i>	-

Table 9: i_soc_err Definition

Bits	Field	Description
31:0	<i>reserved</i>	-

下表 (Table 10, Table 11, Table 12) 详述了 eHSM 到 SoC 状态和错误信号的定义。

SYS_STA0 至 SYS_STA3 的值 (参考 Table 60 - Table 63) 通过 eHSM 顶层接口 o_hsm_status 输出至 SoC。

Table 10: o_hsm_status 释义

Bits	Field	Description
63:48	hsm_fw_sta3	SYS_STA3[31:16]
47:32	hsm_fw_sta2	SYS_STA2[31:16]

Table 10: o_hsm_status 释义

Bits	Field	Description
31:28	<i>reserved</i>	–
27	cpu_hart_halted	SYS_HSM_STA1[27]
26	<i>reserved</i>	–
25	cpu_wfi	SYS_HSM_STA1[25]
24:21	hsm_fw_sta1	SYS_HSM_STA1[24:21]
20	soc_cpu_release	SYS_HSM_STA1[20]
19	soc_cpu_reset	SYS_HSM_STA1[19]
18	soc_reset	SYS_HSM_STA1[18]
17	soc_dbg_en	SYS_HSM_STA1[17]
16	hsm_dbg_en	SYS_HSM_STA1[16]
15	hsm_fw_sta0	SYS_HSM_STA0[31]
14	hsm_lc_undef	SYS_HSM_STA0[30]
13	hsm_lc_destroy	SYS_HSM_STA0[29]
12	hsm_lc_debug	SYS_HSM_STA0[28]
11	hsm_lc_user	SYS_HSM_STA0[27]
10	hsm_lc_manu	SYS_HSM_STA0[26]
9	hsm_lc_dev	SYS_HSM_STA0[25]
8	hsm_lc_test	SYS_HSM_STA0[24]
7	soc_verify_err	SYS_HSM_STA0[23]
6	soc_verify_done	SYS_HSM_STA0[22]
5	firmware_err	SYS_HSM_STA0[21]
4	firmware_done	SYS_HSM_STA0[20]
3	bootloader_err	SYS_HSM_STA0[19]
2	bootloader_done	SYS_HSM_STA0[18]
1	hw_boot_err	SYS_HSM_STA0[17]
0	hw_boot_done	SYS_HSM_STA0[16]

Note: 其他位保留供将来使用。

Figure 12: eHSM 启动完成/错误时序图

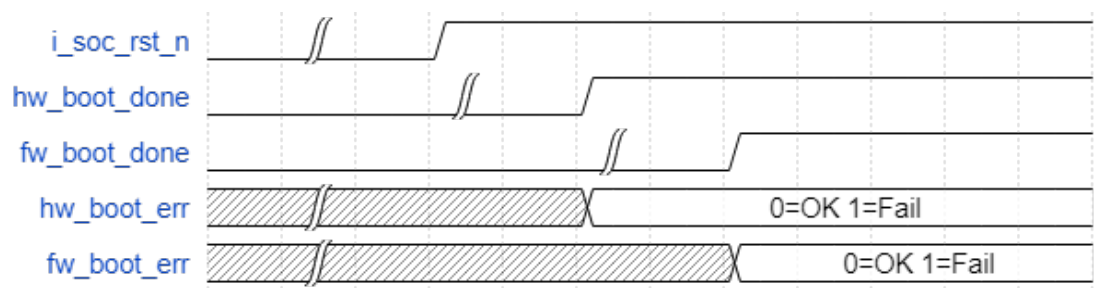


Figure 13: eHSM 启动期间生命周期信号时序图

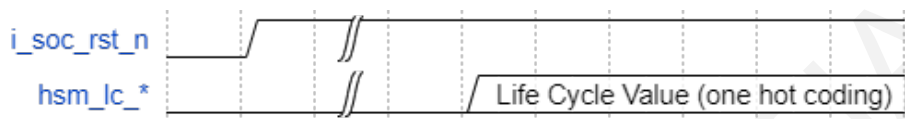


Table 11: o_hsm_err_hw 释义

Bits	Field	Type	Description
Misc Error			
63:41	reserved	—	—
40	wdt_timeout	Level	看门狗计时器超时
39:36	reserved	—	—
35	otp_key_crc_err	Level	OTP 密钥 CRC 错误
34	hw_trng_retry_warning	Level	启动期间 TRNG 健康检测失败。
33	hw_trng_retry_fail	Level	启动期间 TRNG 健康检测重启超时。
32	hw_trng_ht_fail	Level	TRNG 健康测试失败（在启动和功能阶段）
31:30	reserved	—	—
SoC Bus Error			
29	soc_err_ahb_cfg	Level	CFG AHB 总线响应错误
28	soc_err_ahb_nvm	Level	NVM AHB 总线响应错误
27	soc_err_ahb_otp	Level	OTP AHB 总线响应错误
26	soc_err_ahb_mem	Level	SoC Memory AHB 总线响应错误
25	soc_err_axi_dma_rd	Level	DMA AXI 总线读通道响应错误
24	soc_err_axi_dma_wr	Level	DMA AXI 总线写通道响应错误
Memory ECC Error			
23:20	reserved	—	—
19:16	mem_ecc_mb_pke3/2/1/0	Level	PKE3/PKE2/PKE1/PKE0 RAM 不可纠正错误，每个相应 RAM 各有 1 位
15	mem_ecc_mb_kmu	Level	KMU RAM 不可纠正错误
14	mem_ecc_mb_dram	Level	DRAM 不可纠正错误
13	mem_ecc_mb_iram	Level	IRAM 不可纠正错误
12	mem_ecc_mb_irom	Level	IROM 不可纠正错误
11:8	reserved	—	—
7:4	mem_ecc_1b_pke3/2/1/0	Level	PKE3/PKE2/PKE1/PKE0 RAM 可纠正错误，每个相应 RAM 各有 1 位。
3	mem_ecc_1b_kmu	Level	KMU RAM 可纠正错误
2	mem_ecc_1b_dram	Level	DRAM 可纠正错误
1	mem_ecc_1b_iram	Level	IRAM 可纠正错误
0	mem_ecc_1b_irom	Level	IROM 可纠正错误

Note:

- 当不存在 Memory ECC 检查时，Memory ECC Error 将作为保留位。
- 以上硬件错误信号的高电平标志可由 eHSM 固件依据实际需求进行清除，关于 eHSM 固件对以上硬件错误信号的处理方式说明，请参考最新的固件文档[OSR_eHSM_Firmware_TRM_vx.x.pdf](#)。

Table 12: o_hsm_err_fw 释义

Bits	Field	Description
该信号定义请参考最新的固件文档 OSR_eHSM_BootLoader_TRM_vx.x.pdf		

Note:

- 所有固件错误信号均为电平信号

2.5.3 128 Bit Debug Enable

128 位 SoC 调试使能。

Table 13: eHSM IO Signals: 128 Bit Debug Enable

Signal	Direction	Width	Description
o_soc_dbg_en_128b	Out	128	128 位 SoC 调试使能, 参见 Table 66 , Table 67 , Table 68 , Table 69

2.5.4 Mailbox AHB Slave 接口

用于 SoC 访问 eHSM mailbox。

Table 14: Mailbox AHB Slave 接口

Signal	Direction	Width	Description
i_s_hsel	In	1	参见 AMBA® 3 AHB-Lite 协议
i_s_haddr	In	32	同上
i_s_htrans	In	2	同上
i_s_hwrite	In	1	同上
i_s_hburst	In	3	同上
i_s_hsize	In	3	同上
i_s_hprot	In	4	同上
i_s_hmastlock	In	1	同上
i_s_hready	In	1	同上
i_s_hwdata	In	32	同上
o_s_hrdata	Out	32	同上
o_s_hresp	Out	2	同上
o_s_hreadyout	Out	1	同上

2.5.5 AHB Master 接口

用于 eHSM 访问 SoC memory。

Table 15: AHB Master 接口

Signal	Direction	Width	Description
o_m_hsel	Out	1	参见 AMBA® 3 AHB-Lite 协议
o_m_haddr	Out	64	同上
o_m_htrans	Out	2	同上
o_m_hwrite	Out	1	同上
o_m_hburst	Out	3	同上
o_m_hsize	Out	3	同上
o_m_hprot	Out	4	同上
o_m_hmastlock	Out	1	同上
o_m_hready	Out	1	同上
o_m_hwdata	Out	32	同上
i_m_hrdata	In	32	同上
i_m_hresp	In	2	同上
i_m_hreadyout	In	1	同上

2.5.6 DMA AXI Master 接口

2.5.6.1 DMA AXI 功能

- AXI 数据（地址）位宽：64 或 32 位（取决于 SoC 接口实际位宽）
- AXI 通道：2 个通道，HASH 通道 id0，SKE 通道 id1
- AXI FIFO 深度：每个通道 16
- AXI Burst：5 拍（最大）
- AXI 读 Outstanding：每个通道最大 4 个
- AXI 写 Outstanding：每个通道最大 4 个
- 支持读取数据乱序
- 支持读取数据交织

2.5.6.2 DMA AXI Master 接口

用于 eHSM DMA 访问 SoC memory。

Table 16: DMA AXI Master 写通道接口

Signal	Direction	Width	Description
write addr channel			
o_dma_m_awid	Out	4	参见 AMBA® AXI4™ 协议
o_dma_m_awaddr	Out	64	同上
o_dma_m_awlen	Out	8	同上
o_dma_m_awsize	Out	3	同上
o_dma_m_awburst	Out	2	同上
o_dma_m_awlock	Out	1	同上
o_dma_m_awcache	Out	4	同上
o_dma_m_awprot	Out	3	同上
o_dma_m_awqos	Out	4	同上
o_dma_m_awregion	Out	4	同上
o_dma_m_awsnoop	Out	3	同上
o_dma_m_awdomain	Out	2	同上
o_dma_m_awbar	Out	2	同上
o_dma_m_awvalid	Out	1	同上
i_dma_m_awready	In	1	同上
write data channel			
o_dma_m_wid	Out	4	同上
o_dma_m_wdata	Out	64	同上
o_dma_m_wstrb	Out	8	同上
o_dma_m_wlast	Out	1	同上
o_dma_m_wvalid	Out	1	同上
i_dma_m_wready	In	1	同上
write response channel			
i_dma_m_bid	In	4	同上
i_dma_m_bresp	In	2	同上
i_dma_m_bvalid	In	1	同上
o_dma_m_bready	Out	1	同上

Table 17: DMA AXI Master 读通道接口

Signal	Direction	Width	Description
read addr channel			
o_dma_m_arid	Out	4	参见 AMBA® AXI4™ 协议
o_dma_m_araddr	Out	64	同上
o_dma_m_arlen	Out	8	同上
o_dma_m_arsize	Out	3	同上
o_dma_m_arburst	Out	2	同上
o_dma_m_arlock	Out	1	同上
o_dma_m_arcache	Out	4	同上
o_dma_m_arprot	Out	3	同上
o_dma_m_arqos	Out	4	同上
o_dma_m_arregion	Out	4	同上
o_dma_m_arsnoop	Out	4	同上
o_dma_m_ardomain	Out	2	同上
o_dma_m_arbar	Out	2	同上
o_dma_m_arvalid	Out	1	同上
i_dma_m_arready	In	1	同上
read data channel			
i_dma_m_rid	In	4	同上
i_dma_m_rdata	In	64	同上
i_dma_m_rresp	In	2	同上
i_dma_m_rlast	In	1	同上
i_dma_m_rvalid	In	1	同上
o_dma_m_rready	Out	1	同上

2.5.7 NVM AHB Master 接口

Table 18: NVM AHB Master 接口

Signal	Direction	Width	Description
o_nvm_hsel	Out	1	参见 AMBA® 3 AHB-Lite 协议
o_nvm_haddr	Out	64	同上
o_nvm_htrans	Out	2	同上
o_nvm_hwrite	Out	1	同上
o_nvm_hburst	Out	3	同上
o_nvm_hsize	Out	3	同上
o_nvm_hprot	Out	4	同上
o_nvm_hmastlock	Out	1	同上
o_nvm_hready	Out	1	同上
o_nvm_hwdata	Out	32	同上
i_nvm_hrdata	In	32	同上
i_nvm_hresp	In	2	同上
i_nvm_hreadyout	In	1	同上

2.5.8 CFG AHB Master 接口

Table 19: CFG AHB Master 接口

Signal	Direction	Width	Description
o_cfg_hsel	Out	1	参见 AMBA® 3 AHB-Lite 协议
o_cfg_haddr	Out	64	同上
o_cfg_htrans	Out	2	同上
o_cfg_hwrite	Out	1	同上
o_cfg_hburst	Out	3	同上
o_cfg_hsize	Out	3	同上
o_cfg_hprot	Out	4	同上
o_cfg_hmastlock	Out	1	同上
o_cfg_hready	Out	1	同上
o_cfg_hwdata	Out	32	同上
i_cfg_hrdata	In	32	同上
i_cfg_hresp	In	2	同上
i_cfg_hreadyout	In	1	同上

2.5.9 OTP AHB Master 接口

Table 20: OTP AHB Master 接口

Signal	Direction	Width	Description
o_otp_hsel	Out	1	参见 AMBA® 3 AHB-Lite 协议
o_otp_haddr	Out	64	同上
o_otp_htrans	Out	2	同上
o_otp_hwrite	Out	1	同上
o_otp_hburst	Out	3	同上
o_otp_hsize	Out	3	同上
o_otp_hprot	Out	4	同上
o_otp_hmastlock	Out	1	同上
o_otp_hready	Out	1	同上
o_otp_hwdata	Out	32	同上
i_otp_hrdata	In	32	同上
i_otp_hresp	In	2	同上
i_otp_hreadyout	In	1	同上

2.5.10 IROM 接口

当宏定义 `OSR_FPGA_ROM` 启用时，WEB 和 DI 端口被启用，用于 FPGA 的 IROM 调试。

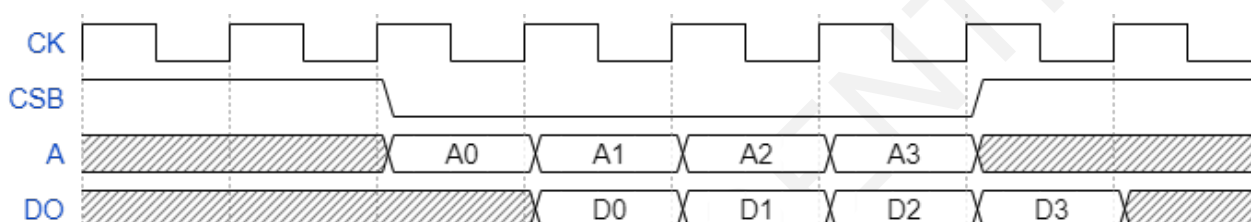
Table 21: IROM 接口

Signal	Direction	Width	Description
o_irom_CK	Out	1	时钟，上升沿
o_irom_A	Out	14	地址
o_irom_CSB	Out	1	片选，低电平有效
o_irom_WEB	Out	1/4	写使能，低电平有效
o_irom_DI	Out	39/32	写入 IROM 的数据
i_irom_DO	In	39/32	从 IROM 读回的数据

- 注意：如果有 ECC，DI/DO 宽度为 32 位数据 + 7 位 ECC，WEB 宽度为 1。

下图示例了 IROM 读时序

Figure 14: eHSM IROM 读时序图



2.5.11 IRAM 接口

Table 22: IRAM 接口

Signal	Direction	Width	Description
o_iram_CK	Out	1	时钟，上升沿
o_iram_A	Out	16	地址
o_iram_CSB	Out	1	片选，低电平有效
o_iram_WEB	Out	1/4	写使能，低电平有效
o_iram_DI	Out	39/32	写入 IRAM 的数据
i_iram_DO	In	39/32	从 IRAM 读回的数据

- 注意：如果有 ECC，DI/DO 宽度为 32 位数据 + 7 位 ECC，WEB 宽度为 1。

下图示例了 IRAM 的读写时序

Figure 15: eHSM IRAM 读时序图

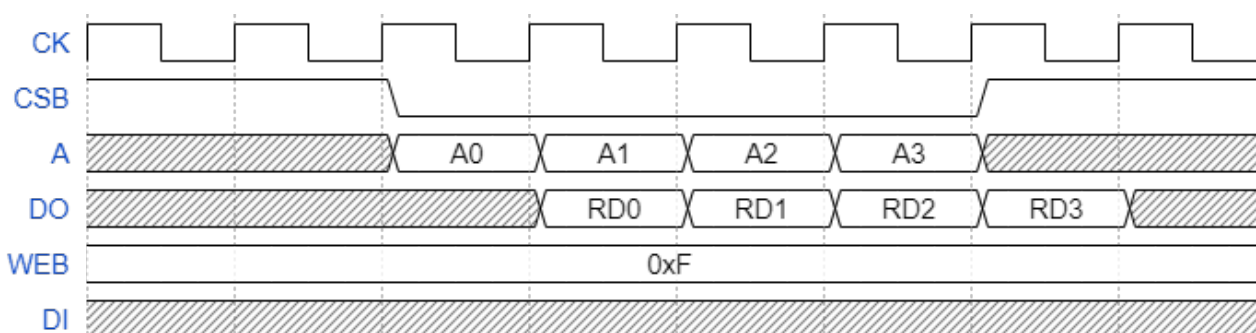
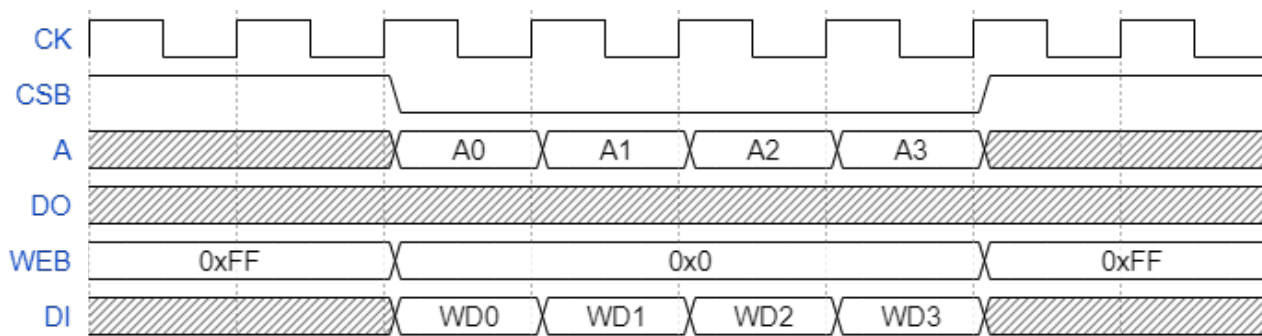


Figure 16: eHSM IRAM 写时序图



2.5.12 DRAM 接口

Table 23: DRAM 接口

Signal	Direction	Width	Description
o_dram_CK	Out	1	时钟，上升沿
o_dram_A	Out	14	地址
o_dram_CSB	Out	1	片选，低电平有效
o_dram_WEB	Out	1/4	写使能，低电平有效
o_dram_DI	Out	39/32	写入 DRAM 的数据
i_dram_DO	In	39/32	从 DRAM 读回的数据

- 注意：如果有 ECC，DI/DO 宽度为 32 位数据 + 7 位 ECC，WEB 宽度为 1。

DRAM 的读/写时序与 IRAM 相同 (Figure 15, Figure 16)

2.5.13 KMU RAM 接口

Table 24: KMU RAM 接口

Signal	Direction	Width	Description
o_kbuf_CK	Out	1	时钟，上升沿
o_kbuf_A	Out	8	地址
o_kbuf_CSB	Out	1	片选，低电平有效
o_kbuf_WEB	Out	1/4	写使能，低电平有效
o_kbuf_DI	Out	39/32	写入 KMU RAM 的数据
i_kbuf_DO	In	39/32	从 KMU RAM 读回的数据

- 注意：如果有 ECC，DI/DO 宽度为 32 位数据 + 7 位 ECC，WEB 宽度为 1。

KMU RAM 的读/写时序与 IRAM 相同 (Figure 15, Figure 16)

2.5.14 PKE RAM 接口

PKE HP 使用 4 个 RAM（双端口 RAM 或单端口 RAM）

Dual Port RAM 接口

PKE 不会同时读写同一地址.

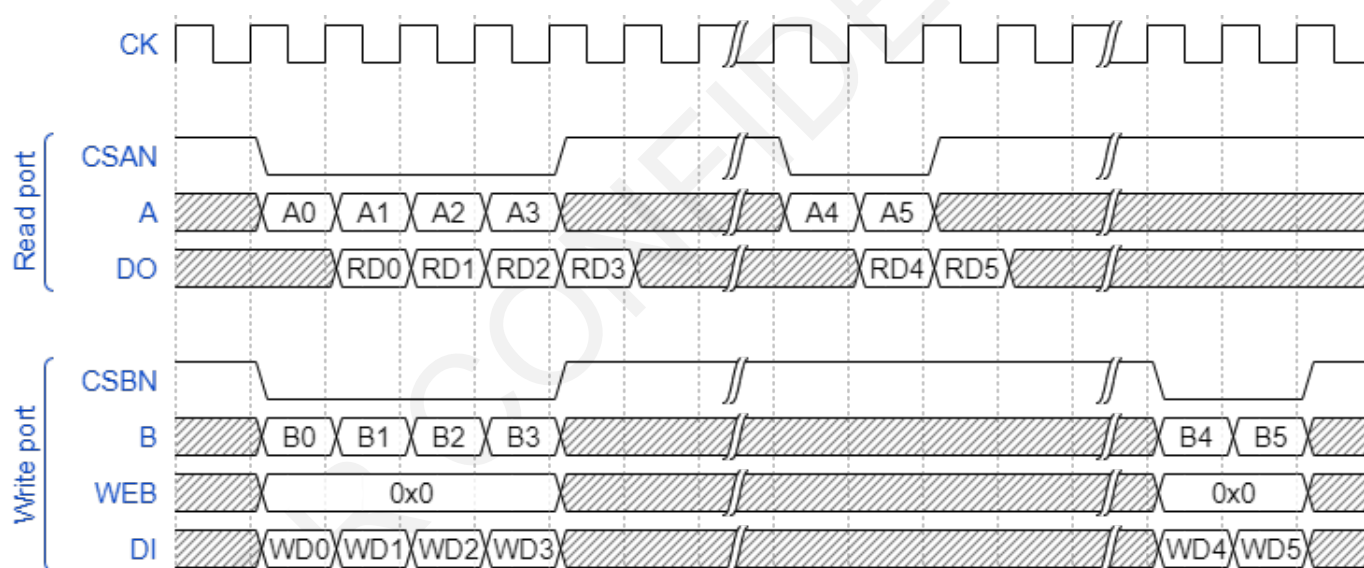
Table 25: PKE RAM 接口 (Dual Port RAM)

Signal	Direction	Width	Description
o_pke_sram*_CK	Out	1	时钟, 上升沿
o_pke_sram*_A	Out	9	地址 (端口 A)
o_pke_sram*_CSAN	Out	1	片选 (端口 A), 低电平有效
i_pke_sram*_DO	In	72/64	从 PKE RAM(port A) 读回的数据
o_pke_sram*_B	Out	9	地址 (端口 B)
o_pke_sram*_CSBN	Out	1	片选 (端口 B), 低电平有效
o_pke_sram*_DI	Out	72/64	写入 PKE RAM(port B) 的数据

Note: * = 0-3

下图示例了 PKE RAM 的读写时序。

Figure 17: eHSM PKE RAM 读写时序图



Single Port RAM 接口

Table 26: PKE RAM 接口 (Single Port RAM)

Signal	Direction	Width	Description
o_pke_sram*_CK	Out	1	时钟, 上升沿
o_pke_sram*_A	Out	9	地址
o_pke_sram*_CSB	Out	1	片选, 低电平有效
o_pke_sram*_WEB	Out	1/8	写使能, 低电平有效
o_pke_sram*_DI	Out	72/64	写入 PKE RAM 的数据
i_pke_sram*_DO	In	72/64	从 PKE RAM 读回的数据

- Notice : 如果使用 ECC, DI/DO 宽度为 64 位数据 +8 位 ECC, WEB 宽度为 1。

Note: * = 0-3

PKE 单端口 RAM 的读/写时序与 IRAM 相同 ([Figure 15](#), [Figure 16](#))

2.5.15 调试接口

Table 27: Debug 接口

Signal	Direction	Width	Description
i_trst_n	In	1	JTAG TRSTN, 低电平有效 (如果未使用, 固定为 1)
i_tck	In	1	JTAG TCK
i_tms	In	1	JTAG TMS
i_tdi	In	1	JTAG TDI
o_tdo_oe	Out	1	JTAG TDO 的输出使能, 高电平有效
o_tdo	Out	1	JTAG TDO

2.5.16 UART 接口

Table 28: UART 接口

Signal	Direction	Width	Description
i_uart_rxd	In	1	UART RXD
o_uart_txd_oe	Out	1	UART TXD 的输出使能, 高电平有效
o_uart_txd	Out	1	UART TXD

2.5.17 TRNG 测试接口

Table 29: TRNG 测试接口

Signal	Direction	Width	Description
o_trng_ro_clk	Out	4	TRNG RO 时钟输出
o_trng_ro_out	Out	4	TRNG RO 数据输出 (由 ro_clk 采样)
o_trng_rdy	Out	1	TRNG 就绪, 可以正常工作
o_trng_alarm	Out	1	TRNG 自检失败告警

Note: [o_trng_rdy](#) 与 [o_trng_alarm](#) 均为电平信号, 用于指示 TRNG 的状态。

2.5.18 DFT 接口

Table 30: DFT 接口

Signal	Direction	Width	Description
i_scan_mode	In	1	扫描模式使能
i_scan_icg_enable	In	1	扫描时钟使能, 参考 2.3.1.1

2.5.19 Mailbox 中断接口

Table 31: Mailbox 中断接口

Signal	Direction	Width	Description
o_mailbox_irq	Out	1-16	eHSM mailbox 中断请求

Note: [o_mailbox_irq](#) 信号的位宽取决于 mailbox 的个数。

2.6 内存映射

可编程内存（NVM, OTP）、数据内存和块寄存器都组织在同一个线性的 4G 字节地址空间内。

字节在内存中以小端格式编码。字中的最低编号字节被认为是字的最低有效字节，最高编号字节被认为是最高有效字节。

可寻址内存空间被划分为 16 个主分区，每个分区为 256 MB。

所有未分配的块都被视为 “Reserved”。

有关块寄存器的详细映射，请参见 [Table 32](#)。

Table 32: 块边界地址

Partitions	Boundary address	Blocks	Register map
CPU	0x0000_0000 - 0x0FFF_FFFF	Reserved for CPU	-
ILM : 0x1000_0000 0x1FFF_FFFF	0x1000_0000 - 0x100F_FFFF	IROM	-
	0x1040_0000 - 0x107F_FFFF	NVM	-
	0x1080_0000 - 0x108F_FFFF	IRAM	-
	Others	Reserved	-
DLM : 0x2000_0000 0x2FFF_FFFF	0x2000_0000 - 0x20FF_FFFF	DRAM	-
	0x2100_0000 - 0x21FF_FFFF	Reserved	-
	Others	Reserved	-
AHB : 0x3000_0000 0x3FFF_FFFF	0x3000_0000 - 0x300F_FFFF	System	Table 51
	0x3010_0000 - 0x301F_FFFF	EMU	Table 129
	0x3040_0000 - 0x304F_FFFF	Boot	Reserved
	0x3080_0000 - 0x308F_FFFF	KMU	Table 169
	0x30C0_0000 - 0x30CF_FFFF	Mailbox	Table 176
	0x3100_0000 - 0x310F_FFFF	TRNG	-
	0x3140_0000 - 0x314F_FFFF	HASH	-
	0x3180_0000 - 0x318F_FFFF	SKE	-
	0x31C0_0000 - 0x31CF_FFFF	PKE	-
	0x3300_0000 - 0x330F_FFFF	OTP	Table 34
	0x3340_0000 - 0x334F_FFFF	CFG(NVM/OTP/SENSOR controller)	-
	0x3F00_0000 - 0x3F0F_FFFF	Random Clock	Table 194
	0x3F10_0000 - 0x3F1F_FFFF	WDT	Table 199
	0x3F40_0000 - 0x3F4F_FFFF	UART	Table 207
	0x3F60_0000 - 0x3F6F_FFFF	CRC	Table 213
	0x3F80_0000 - 0x3F8F_FFFF	General Timer	Table 218
	Others	Reserved	-
Reserved	0x4000_0000 - 0x6FFF_FFFF	Reserved	-
Memory : 0x7000_0000 0x7FFF_FFFF	0x7000_0000 - 0x70FF_FFFF	NVM	-
	0x7800_0000 - 0x78FF_FFFF	IRAM	-
	Others	Reserved	-
SoC_Memory	0x8000_0000 - 0xFFFF_FFFF	SoC Memory	-

2.7 CPU 中断

CPU 支持 64 个中断。

下图展示了每个中断的定义。

Table 33: eHSM CPU 的中断源

CPU Interrupt Number	RTL Bit Index	Interrupt Source
95:61	63:29	<i>Reserved</i>
60	28	EMU
59	27	<i>Reserved</i>
58	26	Mailbox
57	25	<i>Reserved</i>
56	24	System
55	23	<i>Reserved</i>
54	22	<i>Reserved</i>
53	21	<i>Reserved</i>
52	20	<i>Reserved</i>
51	19	<i>Reserved</i>
50	18	<i>Reserved</i>
49	17	<i>Reserved</i>
48	16	General Timer1
47	15	General Timer0
46	14	<i>Reserved</i>
45	13	UART
44	12	<i>Reserved</i>
43	11	<i>Reserved</i>
42	10	WDT0
41	9	<i>Reserved</i>
40	8	CHACHA
39	7	<i>Reserved</i>
38	6	PKE
37	5	<i>Reserved</i>
36	4	SKE
35	3	<i>Reserved</i>
34	2	HASH
33	1	<i>Reserved</i>
32	0	TRNG

注：其它中断保留为特性

3 安全存储

eHSM 提供了一个完整的安全存储解决方案，用于存储密钥、敏感信息和固件镜像。

3.1 OTP

需要一个片上的 OTP，用于存储 eHSM 的敏感信息，包括：

1. 生命周期
2. UID
3. 硬件/固件控制信息
4. eHSM 错误响应控制信息
5. eHSM 固件版本计数器
6. SoC 固件版本计数器
7. OTP 密钥

OTP 可以是真正的 OTP/eFuse，也可以通过 NVM 模拟。

3.1.1 OTP 内存映射

Table 34: OTP 内存映射地址

Field	Offset	Size/Word	Multi-update	Decoded by
Life Cycle 4	0x000	1	yes	HW
UID	0x004	5	no	FW
HW Control Table 35	0x018	2	yes	HW
FW Control-eHSM	0x020	2	yes	HW
FW Control-SoC	0x028	2	yes	HW
Error Response Control Table 36	0x030	8	no	FW
eHSM Version Counter	0x050	4	yes	FW
SOC Version Counter	0x060	4	yes	FW
OTP Key N Attribute 3.4.3	0x070+ 0x4*0xA*N	1	yes	HW
OTP Key N 3.4.1	0x070+ 0x4*(0xA*N+0x1)	8	no	HW
OTP Key N CRC Table 38	0x070+ 0x4*(0xA*N+0x9)	1	no	HW

Note:

- OTP 中的每一位都可以单独编程。
- 请参阅 [OSR_eHSM_BootLoader_-TRM_vx.x.pdf](#) 以获取有关 [FW Control-eHSM](#) 和 [FW Control-SoC](#) 的更详细信息。

3.2 OTP 硬件控制字段

Table 35: OTP 硬件控制字段

Bits	Fields	Description
Hardware OTP 控制字段, 参见3.1.1		
63:48	<i>Reserved</i>	保留
47:40	TRNGTimeOut*	TRNG 超时周期是 $(TRNGTimeOut + 1) \times 2^{18}$ i_clk 周期。 例子： 如果 Fi_clk 是 100MHz (Ti_clk 是 10ns), TRNGTimeOut = 0x7, 那么， TRNG 超时周期是 $(7 + 1) \times 2^{18} \times 10ns = 21ms$ 。 TRNG 超时周期必须大于 20ms。
39:36	TRNGResetTimes*	当 TRNG 失败（超时或报警）时，有 <i>TRNGResetTimes</i> 次硬件重置机会。 如果 <i>TRNGResetTimes</i> = 0, 硬件在 TRNG 失败时不会复位 TRNG。
35:32	<i>Reserved</i>	保留
31:18	<i>Reserved</i>	保留
17:16	RandomClockEn*	Random Clock 0b01: 关闭 (*TEST_MODE 保持关闭) Others: 启用
15:14	<i>Reserved</i>	保留
13:12	KeyAlgSel	OTP 密钥的解密算法选择 0b01: AES128-ECB Others: SM4-ECB
11:8	<i>Reserved</i>	保留
7:6	TRNGSclkSel	TRNG 采样时钟源选择 0b01: 使用系统时钟 Others: 使用内部采样时钟
5:4	TrngSelfTestSkip	TRNG 硬件自测试 0b01: 跳过 其他: 不跳过
3:2	RAMCipherEn*	RAM 加密 0b01: 关闭 Others: 打开
1:0	BusCipherEn*	Bus 加密 0b01: 关闭 Others: 打开

Note: *: 仅在硬件的安全版本中可用

3.3 传感器响应

传感器响应字段定义了每个传感器报警响应。每个传感器报警有 8 位，每一位对应一个特定的响应动作，定义在

Table 36: OTP 传感器响应配置

Sensor Response Bits	Description
[7+N*8 : 0+N*8]	i_sensor Bits[N] 响应配置 bit 7: 保留 bit 6: 保留 bit 5: 复位 SoC bit 4: 复位 eHSM bit 3: 清除 OTP 密钥 bit 2: 清除 NVM 密钥 bit 1: 清除 RAM 密钥 bit 0: 停止加密服务 N = 0 - 31

3.4 密钥存储

加密密钥、其属性及 CRC 校验码存储于 OTP（防回滚 Flash）中。在 eHSM 硬件自动读取并校验 CRC 后，密钥将由专用的 SKE 解密模块解密，并存储在 KMU 中以备后用。

3.4.1 eHSM 密钥

eHSM 密钥按层级排列 (Chip RTL key -> Chip root key -> Device root key). **Chip/Device root key 是必需的。**

下表示例了每个 OTP 密钥的定义（客户可以根据需求增加或删除密钥）

Table 37: eHSM Key

Key Name	Level	Owner	Location	KMU Key ID	Description
Chip RTL Key	–	Chip	RTL	–	仅在启动期间由硬件使用
Chip RTL Kek eHSM	–	Chip	RTL	–	用于安装 eHSM 密钥
Chip RTL Kek SoC	–	Chip	RTL	–	用于安装 SoC 密钥
Chip Root Key	0	Chip	OTP-KMU	0	仅在启动期间由硬件使用
Device Root Key	1	Device	OTP-KMU	1	仅在启动期间由硬件使用
OTP Key 2	1	Chip	OTP-KMU	2	用户根密钥，如可用于在导入（或导出）用户密钥时进行解密（或加密）
OTP Key N	1/2	Chip/Device	OTP-KMU	FW mapping	由软件定义

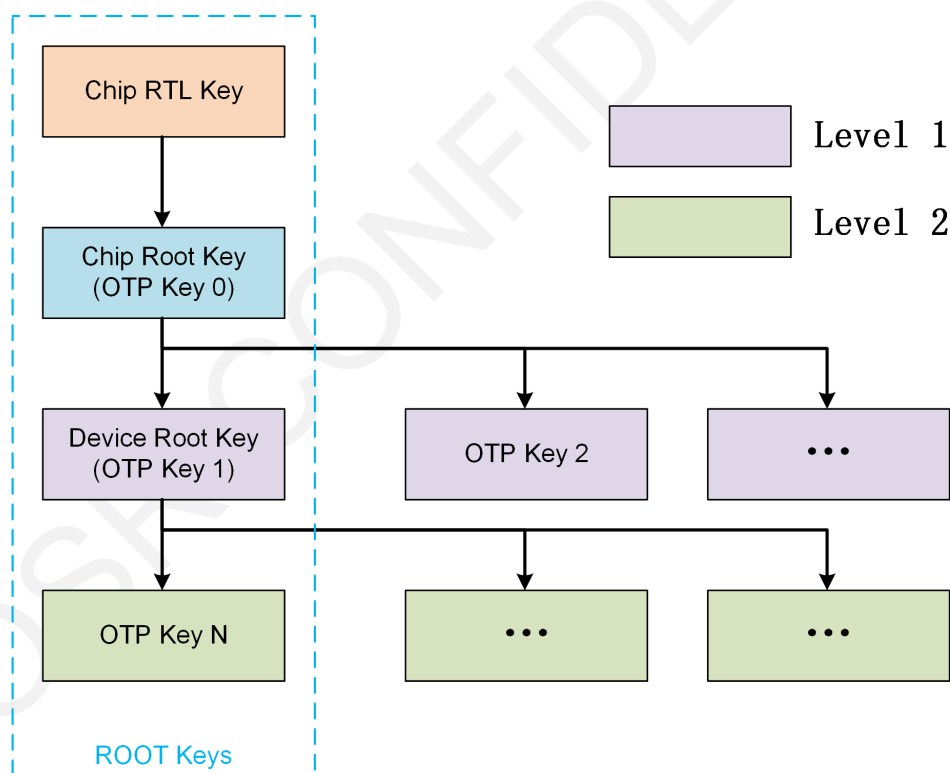
备注: OTP 中的密钥将自动解密到 KMU 直到识别到最后一个密钥的标识符。

Note:

- *KEK*: Key encrypt key, 用于安装 eHSM/SoC 密钥

- 密钥级别
 - 0: 由"Chip RTL Key" 加密
 - 1: 由"Chip ROOT Key" 加密
 - 2: 由"Device ROOT Key" 加密
- 密钥位置
 - "RTL": 密钥存储在 RTL 中
 - "OTP": 密钥存储在 OTP 中
 - "OTP-KMU": 密钥存储在 OTP 中, 并将被解密到 KMU
- 密钥映射: eHSM 固件支持密钥的映射, 密钥的 ID 取决于固件的映射。请参考[OSR_eHSM_Firmware_TRM_vx.x.pdf, 5.4.1\(vx.x: 版本号\)](#)
- 只有前三个 OTP 密钥是硬件需要的, 其余的密钥的使用由软件定义, 请参考[OSR_eHSM_Firmware_TRM_vx.x.pdf, 5.4\(vx.x: 版本号\)](#)

Figure 18: eHSM 密钥层级



3.4.2 OTP 密钥字段

每一个 OTP key 由三个部分构成: 密钥属性, 密钥值, 密钥的 CRC 值。

Table 38: OTP Key 字段划分

OTP Key Offset	Fields	Description
0x00	OTP Key N Attribute	OTP Key N Attribute
0x04	OTP Key N Value Word 0	OTP Key N Bits[31 :0]
0x08	OTP Key N Value Word 1	OTP Key N Bits[63 :32]
0x0C	OTP Key N Value Word 2	OTP Key N Bits[95 :64]
0x10	OTP Key N Value Word 3	OTP Key N Bits[127:96]
0x14	OTP Key N Value Word 4	OTP Key N Bits[159:128]
0x18	OTP Key N Value Word 5	OTP Key N Bits[191:160]
0x1C	OTP Key N Value Word 6	OTP Key N Bits[223:192]
0x20	OTP Key N Value Word 7	OTP Key N Bits[255:224]
0x24	OTP Key N CRC	OTP Key N CRC value

Note:

- N=0-24
- CRC 多项式为 $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1$

3.4.3 eHSM 密钥属性

密钥属性由 32bit 参数构成。

Table 39: eHSM 密钥属性（OTP 位的默认值为 1 的情形）

Bits	Field	Description
31:28	Last Key	最后一个密钥标志 0b1111/0b1000: 此密钥不是最后一个密钥 其他: 此密钥是最后一个密钥 硬件依次解密 OTP 密钥。当遇到具有“最后一个密钥”属性的密钥时，解密完成，随后的 OTP 密钥将不再被解密。
27:12	Reserved	保留以备将来使用
11:8	Key Type	密钥类型（One Hot） 0b1110: 对称密钥 0b1101: 非对称私钥（仅支持 SECP256R1/SM2） 0b1011: 非对称公钥的哈希 0b0111: 保留 其他: 非法
7:4	Key Level	密钥等级 0b1010: 1 级 0b0101: 2 级 其他: 非法
3:0	Life Cycle	密钥生命周期 0b1111: 未烧录，既不检查密钥 CRC 也不保存到 KMU 0b1110: 未使用，既不检查密钥 CRC 也不保存到 KMU 0b1100: 可用，检查密钥 CRC 并保存到 KMU 0b1000: 禁用，既不检查密钥 CRC 也不保存到 KMU 0b0000: 已销毁，既不检查密钥 CRC 也不保存到 KMU 其他: 非法，启动失败并报告标志

Table 40: eHSM 密钥属性（OTP 位的默认值为 0 的情形）

Bits	Field	Description
31:28	Last Key	最后一个密钥标志 0b0000/0b0111: 此密钥不是最后一个密钥 其他: 此密钥是最后一个密钥 硬件依次解密 OTP 密钥。当遇到具有“最后一个密钥”属性的密钥时，解密完成，随后的 OTP 密钥将不再被解密。
27:12	Reserved	保留以备将来使用
11:8	Key Type	密钥类型（One Hot） 0b0001: 对称密钥 0b0010: 非对称私钥（仅支持 SECP256R1/SM2） 0b0100: 非对称公钥的哈希 0b1000: 保留 其他: 非法
7:4	Key Level	密钥等级 0b0101: 1 级 0b1010: 2 级 其他: 非法
3:0	Life Cycle	密钥生命周期 0b0000: 未烧录（Unburned），既不检查密钥 CRC 也不保存到 KMU 0b0001: 未使用（Unused），既不检查密钥 CRC 也不保存到 KMU 0b0011: 可用（Available），检查密钥 CRC 并保存到 KMU 0b0111: 禁用（Disabled），既不检查密钥 CRC 也不保存到 KMU 0b1111: 已销毁（Distorted），既不检查密钥 CRC 也不保存到 KMU 其他: 非法（Illegal），启动失败并拉高 flag 标志

只有当密钥生命周期为“Available”时，才检查 CRC。如果 CRC 结果不符合预期，将设置 CRC_ERROR 信号，并且密钥被认为是无效的。下表描述了如何确定密钥的有效性：

Table 41: eHSM 密钥的有效性

Key	valid conditions
Chip Root Key	密钥生命周期为“Available” 密钥 CRC 正确
Device Root Key	密钥生命周期为“Available” 密钥 CRC 正确
Secret Key 0	密钥生命周期为“Available” 密钥 CRC 正确
Others	密钥生命周期为“Available” 密钥类型为“legal”（非“illegal”） 密钥级别为“legal”（非“illegal”） 密钥 CRC 正确

Notice:

- 只有有效的 OTP 密钥才会被解密并保存到 KMU，否则，保存到 KMU 的为无效数据。参见 Table 47。
- 在 TEST_MODE 生命周期下，即使没有满足有效条件，所有密钥也将被保存到 KMU。

3.4.4 OTP 密钥安装

强烈推荐使用硬件安全模块（**eHSM**）生成和维护所有 **OTP** 密钥。以下表格展示了使用 OSR eHSM 生成 OTP 密钥的提议。

Note:

- Chip root key 将被解码到 KMU，并用于在[测试模式](#)和[开发模式](#)下安装 1 级 OTP 密钥。
- Device root key 将被解码到 KMU，并用于在[测试模式](#)、[开发模式](#)和[制造模式](#)下安装 2 级 OTP 密钥。

Table 42: OTP 密钥安装

OTP Key Name	Type	Owner	Burned by	Key from	Time to Burn	Install Flow
Chip Root Key	Symmetric	Chip Vendor	Chip Vendor	eHSM TRNG	TEST/DEV	加密由 TRNG 生成的随机数，并烧录到 OTP 中。
Device Root Key	Symmetric	Device Vendor	Device Vendor	eHSM TRNG	MANU	加密由 TRNG 生成的随机数，并烧录到 OTP 中。
Secret Key 0	Symmetric	Chip Vendor	Chip Vendor	eHSM TRNG	TEST/DEV	加密由 TRNG 生成的随机数，并烧录到 OTP 中。
eHSM Debug Key	Asymmetric	Chip Vendor	Chip Vendor	External	TEST/DEV	导入公钥哈希值，由 Chip Root Key 加密，烧录到 OTP 中。
eHSM FW Verify Key	Asymmetric/ Symmetric	Chip Vendor	Chip Vendor	External	TEST/DEV	非对称：导入公钥哈希值， 由 Chip Root Key 加密，烧录到 OTP 中。 对称：直接将 TRNG 的随机数据烧录到 OTP 中
eHSM Encrypt Key	Symmetric	Chip Vendor	Chip Vendor	eHSM TRNG	TEST/DEV	加密由 TRNG 生成的随机数，并烧录到 OTP 中。
eHSM Upgrade Encrypt Key	Symmetric	Chip Vendor	Chip Vendor	External	TEST/DEV	导入密钥值，由 Chip Root Key 加密，烧录到 OTP 中。
eHSM Upgrade Verify Key	Asymmetric/ Symmetric	Chip Vendor	Chip Vendor	External	TEST/DEV	非对称：导入公钥哈希值， 由 Chip Root Key 加密，烧录到 OTP 中。 对称：直接将 TRNG 的随机数据烧录到 OTP 中
eHSM Private Key	Asymmetric	Chip Vendor	Chip Vendor	eHSM TRNG	TEST/DEV	加密由 PKE 算法生成的私钥，并烧录到 OTP 中。
SoC Debug Key	Asymmetric	Device Vendor	Device Vendor	External	MANU	导入公钥哈希值，由 Device Root Key 加密，烧录到 OTP 中。
SoC FW Verify Key	Asymmetric/ Symmetric	Device Vendor	Device Vendor	External	MANU	非对称：导入公钥哈希值， 由 Device Root Key 加密，烧录到 OTP 中。 对称：直接将 TRNG 的随机数据烧录到 OTP 中

Table 42: OTP 密钥安装

OTP Key Name	Type	Owner	Burned by	Key from	Time to Burn	Install Flow
SoC Encrypt Key	Symmetric	Device Vendor	Device Vendor	eHSM TRNG	MANU	加密由 TRNG 生成的随机数，并烧录到 OTP 中。
SoC Upgrade Encrypt Key	Symmetric	Device Vendor	Device Vendor	External	MANU	导入密钥值，由 Device Root Key，烧录到 OTP 中。
SoC Upgrade Verify Key	Asymmetric/ Symmetric	Device Vendor	Device Vendor	External	MANU	非对称：导入公钥哈希值， 由 Device Root Key 加密，烧录到 OTP 中。 对称：直接将 TRNG 的随机数据烧录到 OTP 中
SoC Private Key	Asymmetric	Device Vendor	Device Vendor	eHSM TRNG	MANU	加密由 PKE 算法生成的私钥，并烧录到 OTP 中。
User Auth Key	Asymmetric	User	Device Vendor	External	MANU	导入公钥哈希值，由 Device Root Key 加密，烧录到 OTP 中。

Note: 表格 [Table 42](#) 中的密钥仅用于参考。实际的 OTP 密钥安装，请参考 [OSR_eHSM_-Firmware_TRM_-vx.x.pdf, 5.4.1](#) (vx.x: 版本号)

3.5 敏感信息存储

控制 eHSM 系统配置的敏感数据存储在 OTP 或防回滚 Flash 中，并由硬件自动读取。

eHSM 的敏感信息请参见 [Table 35](#).

3.6 固件镜像存储

eHSM 固件存储在:

- 当具有内部 NVM 且无 IRAM 时
 - Bootloader 启动程序在 ROM 中
 - 内部 NVM 中的固件: 将在执行前由 Bootloader 验证
- 当具有内部 NVM 且有 IRAM 时
 - Bootloader 启动程序在 ROM 中
 - 内部 NVM 中的固件: 将在执行前由 Bootloader 验证并加载到 IRAM
- 当具有外部 NVM (IRAM 是必须的) 时:
 - Bootloader 启动程序在 ROM 中
 - 外部 NVM 中的固件: 将在执行前由 Bootloader 验证并加载到 IRAM
- 当只有 ROM (例如, 满足国密二级) 时:
 - 直接执行

Note: 启动程序和固件将被加密以满足国密二级要求。

- 内部 ROM/RAM/NVM 的内存加密
- 外部 NVM 中的固件镜像使用 AES/SM4 加密

4 安全生命周期

eHSM 生命周期定义了设备在其整个生命周期中的状态。设备安全生命周期中的每个安全状态定义了该状态的安全属性。访问和更新 eHSM 的内部资源以及硬件的错误处理取决于生命周期。

4.1 生命周期介绍

eHSM 有以下生命周期：

1. TEST_MODE (TEST)
2. DEVELOP_MODE (DEVE)
3. MANUFACTURE_MODE (MANU)
4. USER_MODE (USER)
5. DEBUG_MODE (DEBG)
6. DESTROY_MODE (DEST)

以下表格展示了 eHSM 中每个生命周期的详细信息。

Table 43: 生命周期概述

Life Cycle	Owner	Description
TEST_MODE	芯片供应商	芯片测试
DEVELOP_MODE	芯片供应商	固件开发
MANUFACTURE_MODE	设备供应商	设备开发/测试/制造
USER_MODE	用户	用户使用设备
DEBUG_MODE	芯片供应商	芯片/设备调试或诊断
DESTROY_MODE	—	芯片/设备不可用
UNDEFINE_MODE	—	与销毁模式相同

Table 44: 生命周期 (不带 ECC) 定义 (OTP 位的默认值为 1 的情形)

Life Cycle	Life Cycle Value
TEST_MODE	0xFFFFFFFF
DEVELOP_MODE	0xBD7E7BEB
MANUFACTURE_MODE	0xB93E5BE9
USER_MODE	0xA83E1369
DEBUG_MODE	0x283A0321
DESTROY_MODE	0x00000000
UNDEFINE_MODE	Others

Table 45: 生命周期 (不带 ECC) 定义 (OTP 位的默认值为 0 的情形)

Life Cycle	Life Cycle Value
TEST_MODE	0x00000000
DEVELOP_MODE	0x42818414
MANUFACTURE_MODE	0x46C1A416
USER_MODE	0x57C1EC96
DEBUG_MODE	0xD7C5FCDE
DESTROY_MODE	0xFFFFFFFF
UNDEFINE_MODE	Others

Note:

在测试生命周期、开发生命周期和调试生命周期下：无论发生任何硬件故障，“hw_boot_err”始终保持低。

4.2 生命周期定义

以下表格展示了 eHSM 生命周期控制的定义。

4.2.1 生命周期中的功能控制

Table 46: 生命周期中的功能控制

Function	TEST	DEVE	MANU	USER	DEBG	DEST
Secure Service	ON	ON	ON	ON	ON	OFF
SoC Secure Debug	ON	SW-OFF	SW-OFF	SW-ON	SW-ON	OFF
eHSM Secure Debug	ON	SW-OFF	SW-ON	SW-ON	SW-ON	OFF
TRNG Health-test	ON	OTP-OFF	OTP-OFF	OTP-OFF	OTP-OFF	OFF

Note:

- **ON** : 默认启用，无法通过软件禁用
- **OFF** : 默认禁用，无法通过软件启用
- **SW-ON** : 默认禁用，可以通过软件启用
- **SW-OFF** : 默认启用，可以通过软件禁用
- **OTP-ON** : 默认禁用，可以通过 OTP 可选控制位启用
- **OTP-OFF** : 默认启用，可以通过 OTP 可选控制位禁用
- 算法自检开关由软件控制，不在本文档中体现。

eHSM 的顶层端口 o_soc_dbg_en_128b 拓展了用于 SoC 的调试使能。

当 SoC Secure Debug 处于ON或者SW-OFF状态时，在硬件启动后，硬件的顶层端口 o_soc_dbg_en_128b 会被自动设置，自动设置的值可由宏定义进行控制，参考Table 232。其中SW-OFF状态表示在硬件设置后，可由软件修改对应寄存器Table 66 - Table 69进行关闭。相对的，SW-ON状态表示硬件启动后，硬件的顶层端口 o_soc_dbg_en_128b 不会被自动设置，保持输出为零，可由软件修改对应寄存器进行开启。

DFT 测试模式下，当i_scan_mode端口拉高时，o_soc_dbg_en_128b 端口会固定输出宏定义所设置的值Table 232，以防止 DFT 测试模式下影响 SoC 的调试功能。

o_soc_dbg_en_128b =

{OSR_SOC_DBG_EN_128B3,OSR_SOC_DBG_EN_128B2,

OSR_SOC_DBG_EN_128B1,OSR_SOC_DBG_EN_128B0};

请参考Table 13, Table 232

4.2.2 生命周期中的 OTP 密钥解密

下表说明了指定的 OTP 密钥是否会在每一个生命周期中被解密到 KMU。

Table 47: 生命周期中的 OTP 密钥解密

eHSM Keys	TEST	DEVE	MANU	USER	DEBG	DEST
Chip Root Key	ON	A	OFF	OFF	OFF	OFF
Device Root Key	ON	A	A	OFF	OFF	OFF
Secret Key 0	ON	A	AD	AD	A	OFF
eHSM Keys (key level=1)	ON	ABCD	ABCD	ABCD	ABCD	OFF
SoC Keys (level=2)	ON	ABCD	ABCD	ABCD	ABCD	OFF

Note:

当满足如下对应条件，密钥将被解密到 KMU，相关的密钥属性请参考Table 39, Table 40。

- ON : 密钥将无条件解密到 KMU
- OFF : 密钥将不被解密到 KMU
- A : 密钥 CRC 值正确
- B : 密钥类型不为非法
- C : 密钥级别不为非法
- D : 密钥生命周期不为非法

4.2.3 生命周期中的 KMU 密钥访问权限

Table 48: 生命周期中的 KMU 密钥访问权限

eHSM Keys	Key Type	TEST	DEVE	MANU	USER	DEBG	DEST
Chip Root Key	Symmetric key	OFF	OFF	OFF	OFF	OFF	OFF
Device Root Key	Symmetric key	OFF	OFF	OFF	OFF	OFF	OFF
Secret Key 0	Symmetric key	OFF	OFF	OFF	OFF	OFF	OFF
eHSM Keys (key level=1)	Symmetric key	RW	RW	OFF	OFF	WO	OFF
	Asymmetric key	RW	RW	RO	RO	RW	OFF
SoC Keys (level=2)	Symmetric key	RW	RW	RW	OFF	WO	OFF
	Asymmetric key	RW	RW	RW	RO	RW	OFF
User Keys	Symmetric key	RW	RW	RW	RW	RW	OFF
	Asymmetric key	RW	RW	RW	RW	RW	OFF

Note:

- **User Keys** : 即用户密钥。不属于 OTP 密钥，由 eHSM CPU 写入 KMU
- **RW** : 可以由 eHSM CPU 读取或写入
- **RO** : 仅可由 eHSM CPU 读取
- **WO** : 仅可由 eHSM CPU 写入
- **OFF** : 不能由 eHSM CPU 访问

如果 OTP 密钥（非 Root Key）的密钥生命周期为未烧录：其密钥槽可由 eHSM CPU 读取或写入。

4.2.4 生命周期中的 eHSM 硬件启动错误

以下表格展示了在每个生命周期中，当 eHSM 具有 KMU 模块时，硬件的启动错误（o_hsm_status.hw_boot_err）检测。

hw_boot_err 信号设置为指示在硬件启动期间检测到的错误，可以从状态寄存器（o_hsm_status）中读取。触发启动错误的条件包括：

- 检测到无效的 OTP 密钥
- Root Key 的密钥生命周期不为“Available”
- Boot 期间获取 TRNG 随机数失败（仅适用于国密二级）

无效的 OTP 密钥意味着它不满足所有“有效条件”（参见 Table 41）。当检测到无效密钥时，hw_boot_err 的触发取决于不同的生命周期，如下表所示。

Table 49: 生命周期中的 eHSM 硬件启动错误（由无效密钥触发）

Items	TEST	DEVE	MANU	USER	DEBG	DEST
无效的 Chip Root Key	OFF	OFF	ON	ON	OFF	NA
无效的 Device Root Key	OFF	OFF	OFF	ON	OFF	NA
无效的 Secret Key 0	OFF	OFF	ON	ON	OFF	NA
无效的 eHSM Key	OFF	OFF	ON	ON	OFF	NA
无效的 SoC Key	OFF	OFF	OFF	ON	OFF	NA

Note:

- **ON** : 将会触发 eHSM hardware boot error (o_ehsm_status.hw_boot_err)
- **OFF** : 将不会触发 eHSM hardware boot error (o_ehsm_status.hw_boot_err)
- **NA** : 将不再启动 Boot 阶段
- **eHSM key** : 密钥等级为 1 的密钥
- **SoC key** : 密钥等级为 2 的密钥

还有其他条件会触发启动错误，请参见 [Table 50](#).

Table 50: 生命周期中的 eHSM 硬件启动错误（由其他条件触发）

Items	TEST	DEVE	MANU	USER	DEBG	DEST
Chip Root Key 的密钥生命周期不为 “Available”	OFF	OFF	ON	ON	OFF	NA
Device Root Key 的密钥生命周期不为 “Available”	OFF	OFF	OFF	ON	OFF	NA
Secret Key 0 的密钥生命周期不为 “Available”	OFF	OFF	ON	ON	OFF	NA
Boot 阶段获取 TRNG 失败 (仅适用于国密二级)	OFF	OFF	ON	ON	OFF	NA

Note:

- **ON** : 将触发 eHSM 硬件启动错误 (o_ehsm_status.hw_boot_err)
- **OFF** : 不会触发 eHSM 硬件启动错误 (o_ehsm_status.hw_boot_err)
- **NA** : 将不再启动 Boot 阶段
- 密钥生命周期的相关说明请参考 [Table 39](#)

5 安全启动

5.1 启动需求

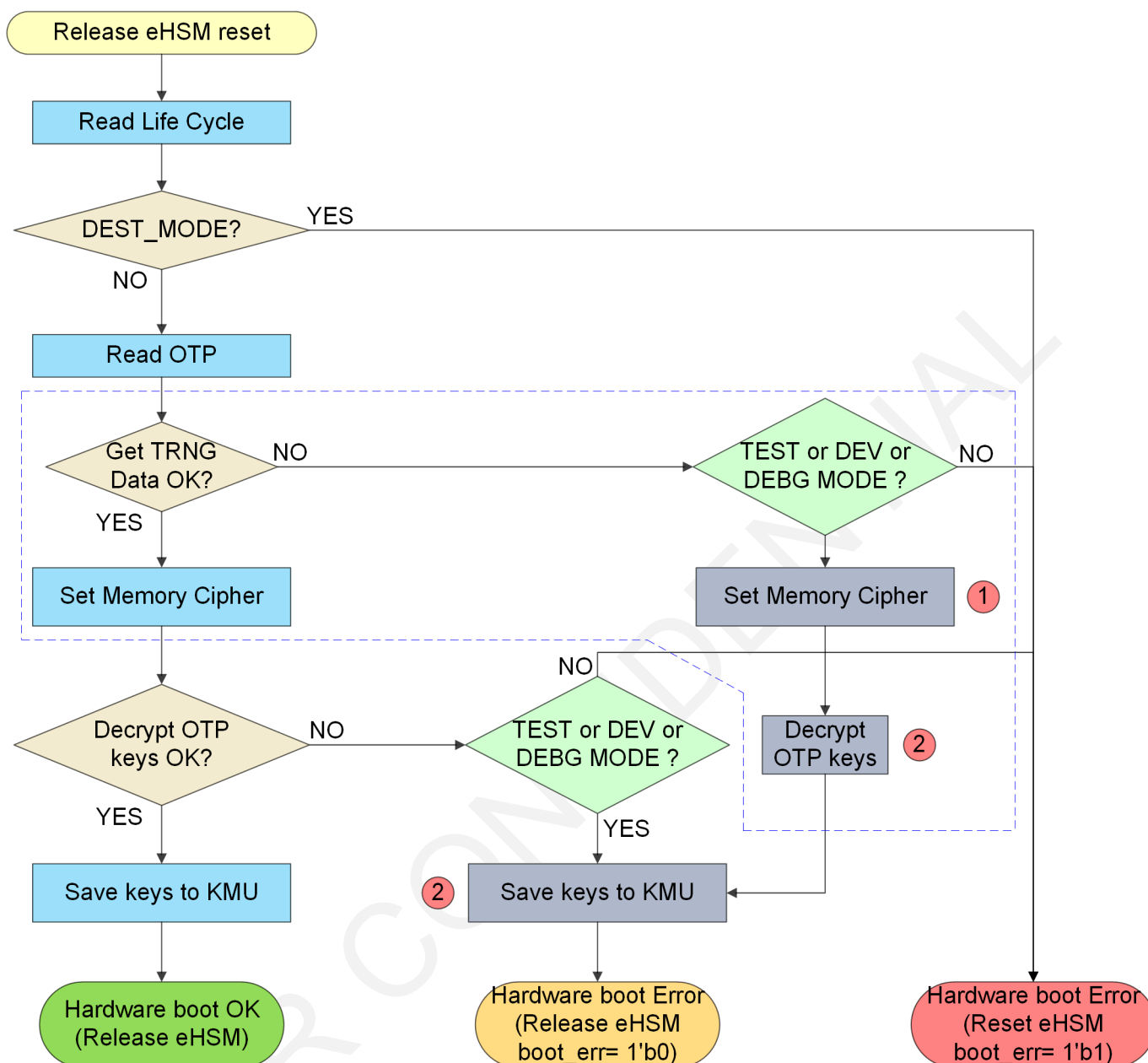
- OTP 密钥
- 芯片生命周期
- Boot 参数
- Boot ROM Code

5.2 硬件启动流程

下图 [Figure 19](#) 描述了 SoC 释放 eHSM 复位后的硬件启动流程。

- 硬件读取 OTP 字段 (refer to [Table 34](#)) 并将值保存到相应的 CSR (参见 [Table 120-Table 126](#)) 以供固件读取。
- 当 TRNG 失败时, 包括硬件自检失败或等待 RNG 数据超时, 启动控制器将重置 TRNG 直到重置次数超时, 参见 [Table 35](#) bit[47:36]。
- 硬件解密 OTP 密钥并将其保存到 KMU RAM 中, 参见 [Table 37](#)
- “Key Error” 包括 OTP 密钥 CRC 错误或 OTP 无效密钥错误, 参见 [4.2.4](#)

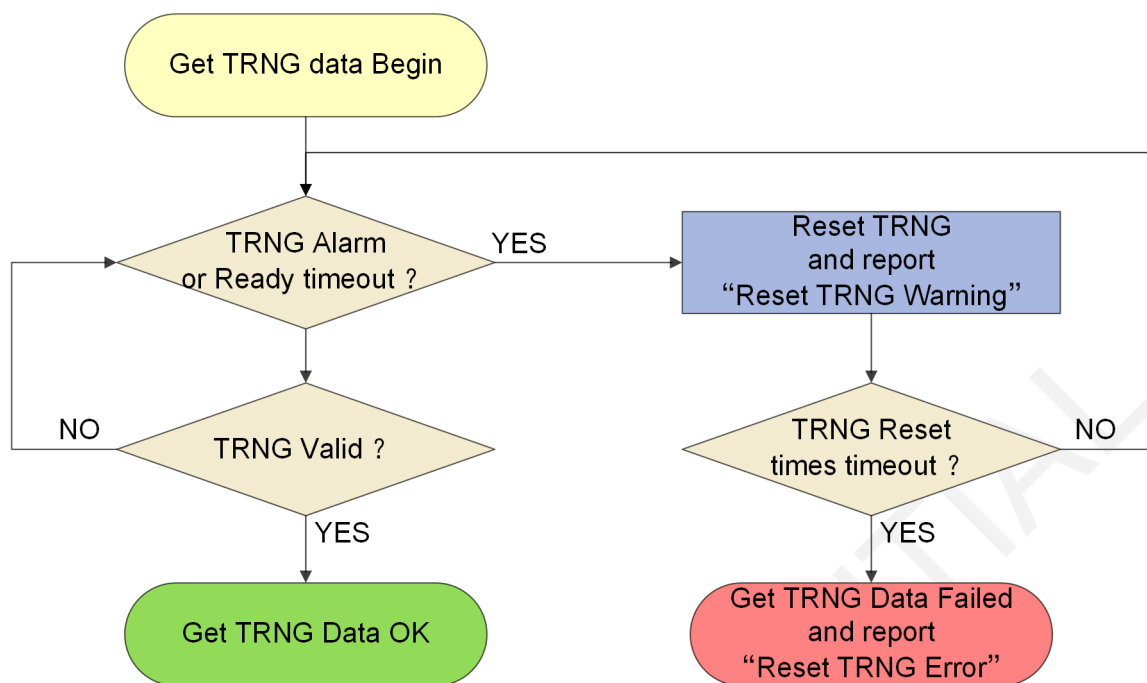
Figure 19: eHSM 硬件启动流程图



Note:

- ① 由于获取TRNG数据失败，使用默认nonce进行内存加密。
- ② 仅用于调试，由于某些密钥可能错误。
- 蓝框内的流程仅适用于国密二级；
否则，如国密一级: Read OTP → Hardware boot OK

Figure 20: eHSM 硬件获取随机数流程图

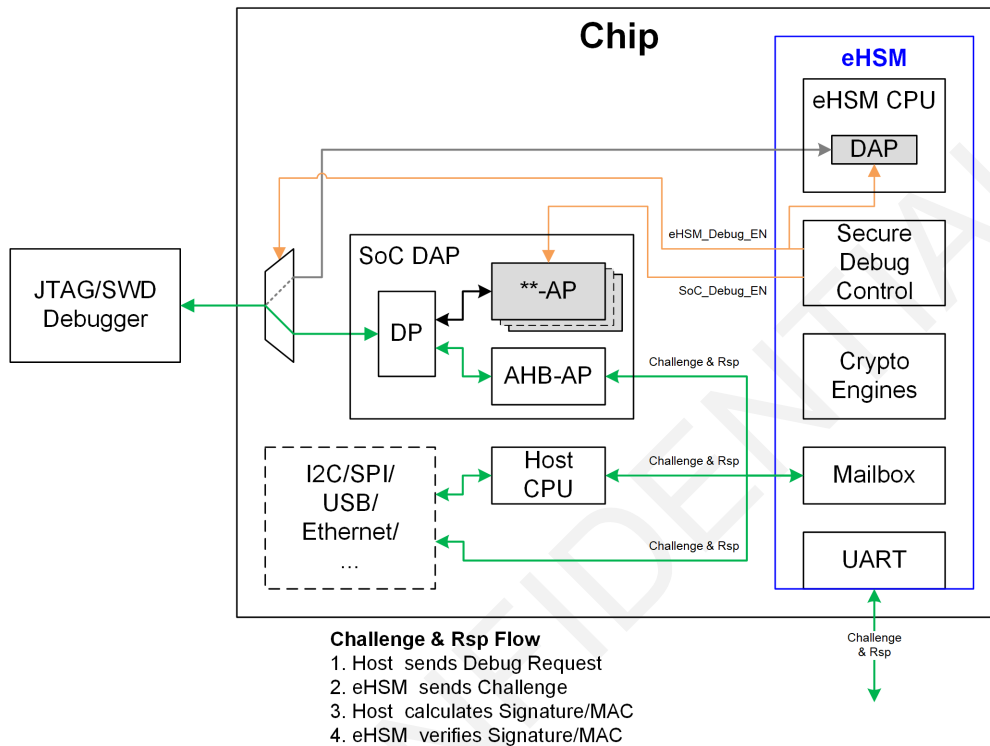


Note: 此流程仅适用于国密二级。

6 安全调试

eHSM 通过 `o_ehsm_status` 的第 17 位和第 16 位控制 eHSM 和 SoC 的调试端口（参见 [Table 10](#)）。eHSM 和 SoC 调试端口的控制取决于生命周期（参见 [Table 46](#)）

Figure 21: eHSM 安全调试架构图



当 eHSM 或 SoC 调试端口默认禁用时，需要进行鉴权才能启用。鉴权可以通过 UART 端口或 eHSM Mailbox 完成。

6.1 安全调试鉴权流程

安全调试鉴权支持非对称或对称算法。

1. 非对称算法: SHA256-SECP256R1, SM3-SM2
2. 对称算法: AES-CMAC, SM4-CMAC

安全调试流程是：1. 主机通过 UART 或 Mailbox 向 eHSM 发送调试请求。

2. eHSM 向主机发送挑战（UID + 随机数）。

3. 主机使用调试密钥签名挑战，并将其发送回 eHSM：

(a) 非对称算法：使用 ehsm 调试密钥/soc 调试密钥的相应私钥签名。

(b) 对称算法：使用 ehsm 调试密钥/soc 调试密钥计算 CMAC 值。

4. eHSM 使用 ehsm 调试密钥/soc 调试密钥验证签名。

5. 如果验证通过，eHSM 将设置 `o_ehsm_status` 相应位以使能 eHSM/SoC 调试端口。

7 密码服务

eHSM 算法包含如下：

- TRNG
- HASH
- SKE
- PKE

7.1 TRNG

TRNG 模块包含真随机数生成器（TRNG）和伪随机数生成器（PRNG）。

- 符合 NIST SP800-90 a/b/c 和 GM 0008-2012 标准
- 支持 CTR-AES-128、CTR-SM4、LFSR128 的 DRBG（根据 TRNG 参数设置选择）
- 生成的随机数可以通过 NIST SP800-22 和 GM 0005-2012 标准的测试
- 支持已生成随机数的在线健康检测
- 支持 AMBA 3 AHB-Lite Slave 接口
- 支持中断
- 可以关闭熵源以降低功耗

7.2 HASH

- 支持 SM3、MD5、SHA-1、SHA-2、SHA-3
- 支持 AMBA 3 AHB-Lite Slave 接口
- 支持硬件填充，包括字节或字填充
- 支持轮询和中断

7.3 SKE

对称密钥引擎（SKE）支持以下功能：

- 支持 AES-128/192/256、DES、3DES、SM4
- 支持 ECB、CBC、CFB、OFB、CTR、CMAC、CBC-MAC、GCM、CCM、XTS 模式
- 支持旁路模式（不加密或不解密）

- 支持 AMBA 3 AHB-Lite Slave 接口
- 支持轮询和中断
- 通过安全密钥端口导入密钥
- 支持抗测信道分析和抗故障注入攻击

7.4 PKE

- 支持 RSA（支持 CRT）：512 - 4096 位
- 支持 ECC（主域）：192, 224, 256, 384, 512, 521 位
- 支持 Ed25519/X25519
- 支持 SM2
- 支持 SM9
- 支持模操作，包括模加法、模减法、模乘法、模幂、模逆
- 支持椭圆曲线点操作，包括点加、点倍、点乘、快速点乘、验证点是否在曲线上
- 支持大数操作，包括加法、乘法和余数
- 支持 AMBA 3 AHB-Lite Slave 接口
- 支持轮询和中断
- 支持点乘器和模幂抗测信道分析和抗故障注入攻击
- 支持内存加密

8 自检

8.1 硬件自检

硬件自检包括生命周期和 OTP 密钥的自检。若自检出错，将触发 eHSM 硬件启动错误信号（o_ehsm_status.hw_boot_err），包括：

- 生命周期处于 DESTROY_MODE 或 UNDEFINE_MODE
- 在 MANUFACTURE_MODE 或 USER_MODE 中无法获取 TRNG 数据，参见 [Figure 20](#)
- 在 MANUFACTURE_MODE 或 USER_MODE 中的 OTP 无效密钥，参见 [4.2.4](#)

eHSM 硬件启动错误意味着：

- eHSM 将处于复位状态
- eHSM 不能提供安全服务
- 串行启动下，eHSM 不释放 SoC 复位（串行启动的说明请参考软件的安全启动文档）

8.2 固件自检

8.2.1 算法自检

eHSM 固件在以下情况下将触发算法自检：

- 上电复位后（POR）
- 接收到指定的自检指令后

在自检期间，固件将执行以下项目：

- TRNG
 - Poker 测试
- PKE
 - RSA 验证
 - RSA-CRT 验证
 - RSA-CRT 密钥生成
 - SM2 密钥生成
 - SM2 签名验签
 - SM2 密钥交换
 - ECDSA-SECP256 签名验签

- SKE
 - SM4 ECB/CBC/CFB/OFB/CTR/XTS/CBC-MAC/CMAC/GCM/CCM
 - AES ECB/CBC/CMAC/GCM
- Hash
 - SHA256
 - SM3

OSR CONFIDENTIAL

9 系统 CSR

9.1 系统 CSR 介绍

系统 CSR 包含控制和状态寄存器。

9.2 系统 CSR 特性

- SoC 至 eHSM 信息
 - 传感器报警
 - SoC 状态信息
- eHSM 至 SoC 信息
 - 启动状态
 - 生命周期信息
 - 自检结果
 - 硬件信息和版本
- eHSM 内部控制和状态
 - 复位控制
 - 时钟门控控制
 - RAM 加密控制
 - 总线加密控制
 - Memory ECC 控制和状态
 - 总线奇偶校验控制
 - AXI 控制信息
 - SoC AHB Master 接口地址偏移
 - 调试寄存器
 - 固件运行步骤信息
- 系统中断
 - 系统中断状态
 - 系统中断使能

9.3 系统 CSR 寄存器

9.3.1 系统 CSR 寄存器映射

系统 CSR 的基地址参见: [Table 32](#)

Table 51: 系统 CSR 寄存器映射

Register	Offset	Description
Hardware Information and Version		
SYS_HW_INF[N]	0x0000+N*0x4	硬件信息 N (N=0-5)
SYS_VER[N]	0x0080+N*0x4	硬件版本 N (N=0-1)
HSM to SOC Status		
SYS_HSM_STA[N]	0x1000+N*0x4	eHSM 状态寄存器 N (N=0-3)
SYS_WR_HSM_DBG_EN	0x1080	设置 eHSM 调试使能
SYS_WR_SOC_DBG_EN	0x1100	设置 SoC 调试使能
SYS_WR_SOC_DBG_EN_128B0	0x1104	设置 SoC 调试位 0
SYS_WR_SOC_DBG_EN_128B1	0x1108	设置 SoC 调试位 1
SYS_WR_SOC_DBG_EN_128B2	0x110C	设置 SoC 调试位 2
SYS_WR_SOC_DBG_EN_128B3	0x1110	设置 SoC 调试位 3
SYS_WR_SOC_CPU_RELEASE	0x1200	设置 SoC CPU 释放
SYS_WR_SOC_CPU_RESET	0x1240	设置 SoC CPU 复位
SYS_WR_SOC_RESET	0x1280	设置 SoC 复位
SOC to HSM Status		
SYS_SOC_STA	0x2000	SoC 状态
SYS_SOC_STA_INT	0x2800	SoC 状态中断
SYS_SOC_STA_INT_EN	0x2880	SoC 状态中断使能
System Control		
SYS_CLK_CTRL[N]	0x3000+N*0x4	通用时钟控制寄存器 N(N=0-1)
SYS_ALG_CLK_CTRL[N]	0x3100+N*0x4	算法模块时钟控制寄存器 N(N=0-4)
SYS_RST_CTRL[N]	0x3200+N*0x4	通用复位控制寄存器 N(N=0-1)
SYS_ALG_RST_CTRL[N]	0x3300+N*0x4	算法模块复位控制寄存器 N(N=0-4)
SYS_CIPHER_CTRL	0x3400	系统加密控制寄存器
SYS_SOC_MEM_BAL	0x3800	SoC Memory 访问基地址低位
SYS_SOC_MEM_BAH	0x3804	SoC Memory 访问基地址高位
SYS_TRNG_DRBG_ALG	0x3900	TRNG DRBG 算法选择
CPU Control		
SYS_CPU_CFG	0x4000	CPU 配置寄存器

Table 51: 系统 CSR 寄存器映射

Register	Offset	Description
Bus Control		
SYS_BUS_ERR_CFG	0x5000	总线响应错误配置寄存器
SYS_BUS_PR_CFG	0x5100	总线奇偶校验配置寄存器
DMA Control		
SYS_AHB_DMA_CFG	0x6000	AHB DMA 配置寄存器
SYS_AXI_DMA_CFG[N]	0x6800+N*0x4	AXI DMA 配置寄存器 N(N=0-1)
SYS_AXI_DMA_CH0_CFG[N]	0x6880+N*0x4	AXI DMA Channel0 配置寄存器 N(N=0-2)
SYS_AXI_DMA_CH1_CFG[N]	0x68A0+N*0x4	AXI DMA Channel1 配置寄存器 N(N=0-2)
SYS_AXI_DMA_CH2_CFG[N]	0x68C0+N*0x4	AXI DMA Channel2 配置寄存器 N(N=0-2)
SYS_AXI_DMA_CH3_CFG[N]	0x68E0+N*0x4	AXI DMA Channel3 配置寄存器 N(N=0-2)
ECC Control		
SYS_CPU_ECC_CTRL[N]	0x7000+N*0x4	ECC 控制寄存器 N(N=0-2)
SYS_KMU_ECC_CTRL	0x7100	ECC 控制寄存器
SYS_PKE_ECC_CTRL[N]	0x7200+N*0x4	ECC 控制寄存器 N(N=0-3)
OTP DATA		
LIFE_CYCLE	0x8000	生命周期
HW_CONTROL[N]	0x8080+N*0x4	HW 控制 N(N=0-1)
FW_CONTROL[N]	0x8100+N*0x4	FW 控制 N(N=0-3)
Extenal Signal		
SYS_GEN_REG[N]	0xF0800+N*0x4	通用寄存器 N(N=0-15)
SYS_RUN_STEP	0xF0F00	固件运行步骤寄存器

9.3.2 系统 CSR_SYS_HW_INF0

Table 52: 系统 CSR_SYS_HW_INF0 (Register offset: 0x0)

Bits	Name	Access Type	Default	Description
31	otp_polarity	RO	0	1: OTP 的默认值为 1 0: OTP 的默认值为 0
30	define_fpga	RO	0	1: "FPGA" 已定义 0: "FPGA" 未定义
29:26	rsv	RO	0	Reserved, read as 0
25:21	cpu_dram_width	RO	0	CPU DRAM 地址位宽
20:16	cpu_iram_width	RO	0	CPU IROM 地址位宽
15:11	cpu_iram_width	RO	0	CPU IROM 地址位宽
10	cpu_dram_byte	RO	0	具有 ECC 检查时 1: CPU DLM 支持字节访问 0: CPU DLM 不支持字节访问
9	cpu_iram_byte	RO	0	具有 ECC 检查时 1: CPU ILM(IRAM) 支持字节访问 0: CPU ILM(IRAM) 不支持字节访问
8	cpu_irom_byte	RO	0	具有 ECC 检查时 1: CPU ILM(IROM) 支持字节访问 0: CPU ILM(IROM) 不支持字节访问
7:2	rsv	RO	0	Reserved, read as 0
1	cpu_tmr	RO	0	1: CPU 具有内部定时器 0: CPU 没有内部定时器
0	cpu_dbg	RO	0	1: CPU 具有调试器 0: CPU 没有调试器

9.3.3 系统 CSR_SYS_HW_INF1

Table 53: 系统 CSR_SYS_HW_INF1 (Register offset: 0x4)

Bits	Name	Access Type	Default	Description
31:0	cpu_reset_vector	RO	0	CPU 复位向量

9.3.4 系统 CSR_SYS_HW_INF2

Table 54: 系统 CSR_SYS_HW_INF2 (Register offset: 0x8)

Bits	Name	Access Type	Default	Description
31:17	rsv	RO	0	Reserved, read as 0
16	kmu_ram_cipher	RO	0	1: KMU RAM 支持加密 0: KMU RAM 不支持加密
15:3	rsv	RO	0	Reserved, read as 0
2	cpu_dram_cipher	RO	0	1: CPU DRAM 支持加密 0: CPU DRAM 不支持加密
1	cpu_iram_cipher	RO	0	1: CPU IRAM 支持加密 0: CPU IRAM 不支持加密
0	cpu_irom_cipher	RO	0	1: CPU IROM 支持加密 0: CPU IROM 不支持加密

9.3.5 系统 CSR_SYS_HW_INF3

Table 55: 系统 CSR_SYS_HW_INF3 (Register offset: 0xC)

Bits	Name	Access Type	Default	Description
31:17	rsv	RO	0	Reserved, read as 0
16	kmu_ram_ecc	RO	0	1: KMU RAM 支持 ECC 0: KMU RAM 不支持 ECC
15:3	rsv	RO	0	Reserved, read as 0
2	cpu_dram_ecc	RO	0	1: CPU DRAM 支持 ECC 0: CPU DRAM 不支持 ECC
1	cpu_iram_ecc	RO	0	1: CPU IRAM 支持 ECC 0: CPU IRAM 不支持 ECC
0	cpu_irom_ecc	RO	0	1: CPU IROM 支持 ECC 0: CPU IROM 不支持 ECC

9.3.6 系统 CSR_SYS_HW_INF4

Table 56: 系统 CSR_SYS_HW_INF4 (Register offset: 0x10)

Bits	Name	Access Type	Default	Description
31:2	rsv	RO	0	Reserved, read as 0
1	pke_ecc_en	RO	0	1: 使能 PKE ECC 0: 关闭 PKE ECC
0	pke_cdc_en	RO	0	1: 使能 PKE CDC 0: 关闭 PKE CDC

9.3.7 系统 CSR SYS_HW_INF5

Table 57: 系统 CSR SYS_HW_INF5 (Register offset: 0x14)

Bits	Name	Access Type	Default	Description
31:8	rsv	RO	0	Reserved, read as 0
7:0	hw_rd_otp_end	RO	—	硬件读取的最后一个 OTP key 对应序号

9.3.8 系统 CSR SYS_VER0

Table 58: 系统 CSR SYS_VER0 (Register offset: 0x80)

Bits	Name	Access Type	Default	Description
31:28	rsv	RO	0	Reserved, read as 0
27:16	con_yq	RO	0	合同年份和季度 [YYQ] 例如 0x223 表示 2022Q3
15:4	rsv	RO	0	Reserved, read as 0
3:0	qnum	RO	0	此季度内的合同编号

9.3.9 系统 CSR SYS_VER1

Table 59: 系统 CSR SYS_VER1 (Register offset: 0x84)

Bits	Name	Access Type	Default	Description
31:16	client_code	RO	0	客户代码
15:0	git_tag	RO	0	不同版本信息的标签

9.3.10 系统 CSR SYS_HSM_STA0

这些状态将会通过 o_hsm_status 传输给 SoC，详情参见 [Table 10](#).

Table 60: 系统 CSR SYS_HSM_STA0 (Register offset: 0x1000)

Bits	Name	Access Type	Default	Description
31	hsm_fw_sta0	RWP	0	eHSM 状态，保留给固件进行配置，用于信息传递
30	hsm_lc_undef	RO	0	eHSM 处于 UNDEFINE_MODE 生命周期
29	hsm_lc_destroy	RO	0	eHSM 处于 DESTROY_MODE 生命周期
28	hsm_lc_debug	RO	0	eHSM 处于 DEBUG_MODE 生命周期
27	hsm_lc_user	RO	0	eHSM 处于 USER_MODE 生命周期
26	hsm_lc_manu	RO	0	eHSM 处于 MANUFACTURE_MODE 生命周期
25	hsm_lc_dev	RO	0	eHSM 处于 DEVELOP_MODE 生命周期
24	hsm_lc_test	RO	0	eHSM 处于 TEST_MODE 生命周期
23	soc_verify_err	RWP	0	SoC 引导程序加载和验签出错
22	soc_verify_done	RWP	0	SoC 引导程序加载和验签结束
21	firmware_err	RWP	0	eHSM 加载固件出错
20	firmware_done	RWP	0	eHSM 加载固件结束
19	bootloader_err	RWP	0	Bootloader 启动出错
18	bootloader_done	RWP	0	Bootloader 启动结束
17	hw_boot_err	RO	0	硬件启动出错
16	hw_boot_done	RO	0	硬件启动结束
15:8	rsv	RO	0	Reserved, read as 0
7:0	write_sta0_valid	W65T	0	0x65: 写入 SYS_WR_STA0 有效 其他: 写入 SYS_WR_STA0 无效

9.3.11 系统 CSR SYS_HSM_STA1

这些状态将会通过 o_hsm_status 传输给 SoC，详情参见 [Table 10](#).

Table 61: 系统 CSR SYS_HSM_STA1 (Register offset: 0x1004)

Bits	Name	Access Type	Default	Description
27	cpu_hart_halted	RO	0	eHSM CPU 停止运行
25	cpu_wfi	RO	0	eHSM CPU WFi（等待中断）状态，高电平有效
24:21	hsm_fw_sta1	RWP	0	eHSM 状态，为固件保留的功能
20	soc_cpu_release	RO	0	释放 SoC CPU
19	soc_cpu_reset	RO	0	复位 SoC CPU，高电平有效
18	soc_reset	RO	0	复位 SoC，高电平有效
17	soc_dbg_en	RO	0	使能 SoC 调试器，高电平有效。 参见 Table 46
16	hsm_dbg_en	RO	0	使能 eHSM 调试器，高电平有效。 参见 Table 46
15:8	rsv	RO	0	Reserved, read as 0
7:0	write_sta1_valid	W93T	0	0x93: 写入 SYS_WR_STA1 有效 其他: 写入 SYS_WR_STA1 无效

9.3.12 系统 CSR SYS_HSM_STA2

这些状态将会通过 o_hsm_status 传输给 SoC，详情参见 [Table 10](#).

Table 62: 系统 CSR SYS_HSM_STA2 (Register offset: 0x1008)

Bits	Name	Access Type	Default	Description
31:21	hsm_fw_sta2	RWP	0	eHSM 状态，保留给固件进行配置，用于信息传递
20	fw_upgrade_working	RWP	0	升级/备份工作状态，1 表示升级或备份正在进行
19	fw_backup_error	RWP	0	备份错误状态，0 表示备份成功，1 表示备份失败
18	fw_backup_done	RWP	0	备份完成，成功状态见 fw_backup_error
17	fw_upgrade_error	RWP	0	Single bank 升级错误状态，0 表示升级成功，1 表示升级失败
16	fw_upgrade_done	RWP	0	Single bank 升级完成，成功状态由 fw_upgrade_error 决定
15:8	rsv	RO	0	Reserved, read as 0
7:0	write_sta2_valid	WA6T	0	0xA6: 写入 SYS_WR_STA2 有效 其他: 写入 SYS_WR_STA2 无效

9.3.13 系统 CSR SYS_HSM_STA3

这些状态将会通过 o_hsm_status 传输给 SoC，详情参见 [Table 10](#).

Table 63: 系统 CSR SYS_HSM_STA3 (Register offset: 0x100C)

Bits	Name	Access Type	Default	Description
31:16	hsm_fw_sta3	RWP	0	eHSM 状态，保留给固件进行配置，用于信息传递
15:8	rsv	RO	0	Reserved, read as 0
7:0	write_sta3_valid	WC9T	0	0xC9: 写入 SYS_WR_STA3 有效 其他: 写入 SYS_WR_STA3 无效

9.3.14 系统 CSR SYS_WR_HSM_DBG_EN

Table 64: 系统 CSR SYS_WR_HSM_DBG_EN (Register offset: 0x1080)

Bits	Name	Access Type	Default	Description
31:0	wr_hsm_dbg_en	RW	0	0x265C1A93: 拉高 hsm_dbg_en 其他: 清除 hsm_dbg_en

9.3.15 系统 CSR SYS_WR_SOC_DBG_EN

Table 65: 系统 CSR SYS_WR_SOC_DBG_EN (Register offset: 0x1100)

Bits	Name	Access Type	Default	Description
31:0	wr_soc_dbg_en	RW	0	0x6F3C0A95: 拉高 soc_dbg_en 其他: 清除 soc_dbg_en

9.3.16 System CSR SYS_WR_SOC_DBG_EN_128B0

Table 66: 系统 CSR SYS_WR_SOC_DBG_EN_128B0 (Register offset: 0x1104)

Bits	Name	Access Type	Default	Description
31:0	wr_soc_dbg_en_128B0	RW	0	使能 SoC 调试器, 高有效。 soc_dbg_en[31:0]

9.3.17 System CSR SYS_WR_SOC_DBG_EN_128B1

Table 67: 系统 CSR SYS_WR_SOC_DBG_EN_128B1 (Register offset: 0x1108)

Bits	Name	Access Type	Default	Description
31:0	wr_soc_dbg_en_128B1	RW	0	使能 SoC 调试器, 高有效。 soc_dbg_en[63:32]

9.3.18 System CSR SYS_WR_SOC_DBG_EN_128B2

Table 68: 系统 CSR SYS_WR_SOC_DBG_EN_128B2 (Register offset: 0x110C)

Bits	Name	Access Type	Default	Description
31:0	wr_soc_dbg_en_128B2	RW	0	使能 SoC 调试器, 高有效。 soc_dbg_en[95:64]

9.3.19 System CSR SYS_WR_SOC_DBG_EN_128B3

Table 69: 系统 CSR SYS_WR_SOC_DBG_EN_128B3 (Register offset: 0x1110)

Bits	Name	Access Type	Default	Description
31:0	wr_soc_dbg_en_128B3	RW	0	使能 SoC 调试器, 高有效。 soc_dbg_en[127:96]

9.3.20 系统 CSR SYS_WR_SOC_CPU_RELEASE

Table 70: 系统 CSR SYS_WR_SOC_CPU_RELEASE (Register offset: 0x1200)

Bits	Name	Access Type	Default	Description
31:0	wr_soc_cpu_release	RW	0	0x59AC65A3: 拉高 soc_cpu_release 0x7F99A765: 清除 soc_cpu_release 其他: 不受影响

9.3.21 系统 CSR SYS_WR_SOC_CPU_RESET

Table 71: 系统 CSR SYS_WR_SOC_CPU_RESET (Register offset: 0x1240)

Bits	Name	Access Type	Default	Description
31:0	wr_soc_cpu_reset	RW	0	0xA93C65F4: 拉高 soc_cpu_reset 0xB0DC2ECA: 清除 soc_cpu_reset 其他: 不受影响

9.3.22 系统 CSR SYS_WR_SOC_RESET

Table 72: 系统 CSR SYS_WR_SOC_RESET (Register offset: 0x1280)

Bits	Name	Access Type	Default	Description
31:0	wr_soc_reset	RW	0	0x5C28639D: 拉高 soc_reset 0xAE1F05B7: 清除 soc_reset 其他: 不受影响

9.3.23 System CSR SYS_SOC_STA

Table 73: System CSR SYS_SOC_STA (Register offset: 0x2000)

Bits	Name	Access Type	Default	Description
31:0	sys_soc_sta	RO	0	对应 i_soc_status 输入状态, 参见 Table 8

9.3.24 System CSR SYS_SOC_STA_INT

Table 74: System CSR SYS_SOC_STA_INT (Register offset: 0x2800)

Bits	Name	Access Type	Default	Description
31:0	sys_soc_sta_int	RW1C	0	对应 i_soc_status[31:0] 中断状态, 参见 Table 8

9.3.25 System CSR SYS_SOC_STA_INT_EN

Table 75: System CSR SYS_SOC_STA_INT_EN (Register offset: 0x2880)

Bits	Name	Access Type	Default	Description
31:0	sys_soc_sta_int_en	RW	0	对应 i_soc_status 中断使能, 参见 Table 8

9.3.26 系统 CSR SYS_CLK_CTRL0

Table 76: 系统 CSR SYS_CLK_CTRL0 (Register offset: 0x3000)

Bits	Name	Access Type	Default	Description
31:27	rsv	RO	0	Reserved, read as 0
26	axi_dma_clk_en	RWP	1	1: 使能 AXI DMA 时钟 0: 关闭 AXI DMA 时钟
25	rsv	RO	0	Reserved, read as 0
24	ahb_dma_clk_en	RWP	1	1: 使能 AHB DMA 时钟 0: 关闭 AHB DMA 时钟
23	debug_mailbox_clk_en	RWP	1	1: 使能 Debug mailbox 时钟 0: 关闭 Debug mailbox 时钟
22:21	rsv	RO	0	Reserved, read as 0
20	mailbox_clk_en	RWP	1	1: 使能 Mailbox 时钟 0: 关闭 Mailbox 时钟
19:17	rsv	RO	0	Reserved, read as 0
16	kmu_clk_en	RWP	1	1: 使能 KMU 时钟 0: 关闭 KMU 时钟
15	irom_clk_en	RWP	1	1: 使能 IROM 时钟 0: 关闭 IROM 时钟
14:13	rsv	RO	0	Reserved, read as 0
12	emu_clk_en	RWP	1	1: 使能 EMU 时钟 0: 关闭 EMU 时钟
11:9	rsv	RO	0	Reserved, read as 0
8	boot_clk_en	RWP	1	1: 使能 BOOT 时钟 0: 关闭 BOOT 时钟
7:0	write_cc0_valid	W5AT	0	0x5A: 写入 SYS_CLK_CTRL0 有效 其他: 写入 SYS_CLK_CTRL0 无效

9.3.27 系统 CSR SYS_CLK_CTRL1

Table 77: 系统 CSR SYS_CLK_CTRL1 (Register offset: 0x3004)

Bits	Name	Access Type	Default	Description
31:17	rsv	RO	0	Reserved, read as 0
16	tim_clk_en	RWP	1	1: 使能 Timer 时钟 0: 关闭 Timer 时钟
15:14	rsv	RO	0	Reserved, read as 0
13	crc_clk_en	RWP	1	1: 使能 CRC 时钟 0: 关闭 CRC 时钟
12	uart_clk_en	RWP	1	1: 使能 UART 时钟 0: 关闭 UART 时钟
11:9	rsv	RO	0	Reserved, read as 0
8	wdt_clk_en	RWP	1	1: 使能 WDT 时钟 0: 关闭 WDT 时钟
7:0	write_cc1_valid	W5AT	0	0x5A: 写入 SYS_CLK_CTRL1 有效 其他: 写入 SYS_CLK_CTRL1 无效

9.3.28 系统 CSR_SYS_ALG_CLK_CTRL0

Table 78: 系统 CSR_SYS_ALG_CLK_CTRL0 (Register offset: 0x3100)

Bits	Name	Access Type	Default	Description
31:10	rsv	RO	0	Reserved, read as 0
9	hash_clk_dma_en	RWP	1	1: 使能 HASH DMA 时钟 0: 关闭 HASH DMA 时钟
8	hash_clk_en	RWP	1	1: 使能 HASH 时钟 0: 关闭 HASH 时钟
7:0	write_acc0_valid	W5AT	0	0x5A: 写入 SYS_ALG_CLK_CTRL0 有效 其他: 写入 SYS_ALG_CLK_CTRL0 无效

9.3.29 系统 CSR_SYS_ALG_CLK_CTRL1

Table 79: 系统 CSR_SYS_ALG_CLK_CTRL1 (Register offset: 0x3104)

Bits	Name	Access Type	Default	Description
31:10	rsv	RO	0	Reserved, read as 0
9	ske_clk_dma_en	RWP	1	1: 使能 SKE DMA 时钟 0: 关闭 SKE DMA 时钟
8	ske_clk_en	RWP	1	1: 使能 SKE 时钟 0: 关闭 SKE 时钟
7:0	write_acc1_valid	W5AT	0	0x5A: 写入 SYS_ALG_CLK_CTRL1 有效 其他: 写入 SYS_ALG_CLK_CTRL1 无效

9.3.30 系统 CSR_SYS_ALG_CLK_CTRL2

Table 80: 系统 CSR_SYS_ALG_CLK_CTRL2 (Register offset: 0x3108)

Bits	Name	Access Type	Default	Description
31:9	rsv	RO	0	Reserved, read as 0
8	pke_clk_en	RWP	1	1: 使能 PKE 时钟 0: 关闭 PKE 时钟
7:0	write_acc2_valid	W5AT	0	0x5A: 写入 SYS_ALG_CLK_CTRL2 有效 其他: 写入 SYS_ALG_CLK_CTRL2 无效

9.3.31 系统 CSR_SYS_ALG_CLK_CTRL3

Table 81: 系统 CSR_SYS_ALG_CLK_CTRL3 (Register offset: 0x310C)

Bits	Name	Access Type	Default	Description
31:9	rsv	RO	0	Reserved, read as 0
8	trng_clk_en	RWP	1	1: 使能 TRNG 时钟 0: 关闭 TRNG 时钟
7:0	write_acc3_valid	W5AT	0	0x5A: 写入 SYS_ALG_CLK_CTRL3 有效 其他: 写入 SYS_ALG_CLK_CTRL3 无效

9.3.32 系统 CSR SYS_ALG_CLK_CTRL4

Table 82: 系统 CSR SYS_ALG_CLK_CTRL4 (Register offset: 0x3110)

Bits	Name	Access Type	Default	Description
31:10	rsv	RO	0	Reserved, read as 0
9	chacha_clk_dma_en	RWP	1	1: 使能 ChaCha DMA 时钟 0: 关闭 ChaCha DMA 时钟
8	chacha_clk_en	RWP	1	1: 使能 ChaCha 时钟 0: 关闭 ChaCha 时钟
7:0	write_acc4_valid	W5AT	0	0x5A: 写入 SYS_ALG_CLK_CTRL4 有效 其他: 写入 SYS_ALG_CLK_CTRL4 无效

9.3.33 系统 CSR SYS_RST_CTRL0

Table 83: 系统 CSR SYS_RST_CTRL0 (Register offset: 0x3200)

Bits	Name	Access Type	Default	Description
31	hsm_with_boot_rst	RWP	1	eHSM 复位 (包含 Boot 模块), 低电平有效, 硬件自动释放
30	hsm_rst	RWP	1	eHSM 复位 (不包含 Boot 模块), 低电平有效, 硬件自动释放
29	cpu_rst	RWP	1	eHSM CPU 复位, 低电平有效, 硬件自动释放
28:27	rsv	RO	0	Reserved, read as 0
26	axi_dma_rst	RWP	1	AXI DMA 复位, 低电平有效
25	rsv	RO	0	Reserved, read as 0
24	ahb_dma_rst	RWP	1	AHB DMA 复位, 低电平有效
23	debug_mailbox_rst	RWP	1	Debug mailbox 复位, 低电平有效
22:21	rsv	RO	0	Reserved, read as 0
20	mailbox_rst	RWP	1	Mailbox 复位, 低电平有效
19:17	rsv	RO	0	Reserved, read as 0
16	kmu_rst	RWP	1	KMU 复位, 低电平有效
15	irom_rst	RWP	1	IROM 复位, 低电平有效
14:13	rsv	RO	0	Reserved, read as 0
12	emu_rst	RWP	1	EMU 复位, 低电平有效
11:10	rsv	RO	0	Reserved, read as 0
9	sys_rst	RWP	1	System 复位, 低电平有效, 硬件自动释放
8	boot_rst	RWP	1	Boot 复位, 低电平有效, 硬件自动释放
7:0	write_rc0_valid	W5AT	0	0x5A: 写入 SYS_RST_CTRL0 有效 其他: 写入 SYS_RST_CTRL0 无效

9.3.34 系统 CSR SYS_RST_CTRL1

Table 84: 系统 CSR SYS_RST_CTRL1 (Register offset: 0x3204)

Bits	Name	Access Type	Default	Description
31:17	rsv	RO	0	Reserved, read as 0
16	tim_rst	RWP	1	Timer 复位, 低电平有效
15:14	rsv	RO	0	Reserved, read as 0
13	crc_rst	RWP	1	CRC 复位, 低电平有效
12	uart_rst	RWP	1	UART 复位, 低电平有效
11:9	rsv	RO	0	Reserved, read as 0
8	wdt_rst	RWP	1	WDT 复位, 低电平有效
7:0	write_rc1_valid	W5AT	0	0x5A: 写入 SYS_RST_CTRL1 有效 其他: 写入 SYS_RST_CTRL1 无效

9.3.35 系统 CSR SYS_ALG_RST_CTRL0

Table 85: 系统 CSR SYS_ALG_RST_CTRL0 (Register offset: 0x3300)

Bits	Name	Access Type	Default	Description
31:10	rsv	RO	0	Reserved, read as 0
9	hash_dma_rst	RWP	1	HASH DMA 复位, 低电平有效
8	hash_rst	RWP	1	HASH 复位, 低电平有效
7:0	write_arc0_valid	W5AT	0	0x5A: 写入 SYS_ALG_RST_CTRL0 有效 其他: 写入 SYS_ALG_RST_CTRL0 无效

9.3.36 系统 CSR SYS_ALG_RST_CTRL1

Table 86: 系统 CSR SYS_ALG_RST_CTRL1 (Register offset: 0x3304)

Bits	Name	Access Type	Default	Description
31:10	rsv	RO	0	Reserved, read as 0
9	ske_dma_rst	RWP	1	SKE DMA 复位, 低电平有效
8	ske_rst	RWP	1	SKE 复位, 低电平有效
7:0	write_arc1_valid	W5AT	0	0x5A: 写入 SYS_ALG_RST_CTRL1 有效 其他: 写入 SYS_ALG_RST_CTRL1 无效

9.3.37 系统 CSR_SYS_ALG_RST_CTRL2

Table 87: 系统 CSR_SYS_ALG_RST_CTRL2 (Register offset: 0x3308)

Bits	Name	Access Type	Default	Description
31:9	rsv	RO	0	Reserved, read as 0
8	pke_rst	RWP	1	PKE 复位, 低电平有效
7:0	write_arc2_valid	W5AT	0	0x5A: 写入 SYS_ALG_RST_CTRL2 有效 其他: 写入 SYS_ALG_RST_CTRL2 无效

9.3.38 系统 CSR_SYS_ALG_RST_CTRL3

Table 88: 系统 CSR_SYS_ALG_RST_CTRL3 (Register offset: 0x330C)

Bits	Name	Access Type	Default	Description
31:9	rsv	RO	0	Reserved, read as 0
8	trng_rst	RWP	1	TRNG 复位, 低电平有效
7:0	write_arc3_valid	W5AT	0	0x5A: 写入 SYS_ALG_RST_CTRL3 有效 其他: 写入 SYS_ALG_RST_CTRL3 无效

9.3.39 系统 CSR_SYS_ALG_RST_CTRL4

Table 89: 系统 CSR_SYS_ALG_RST_CTRL4 (Register offset: 0x3310)

Bits	Name	Access Type	Default	Description
31:10	rsv	RO	0	Reserved, read as 0
9	chacha_dma_rst	RWP	1	ChaCha DMA 复位, 低电平有效
8	chacha_rst	RWP	1	ChaCha 复位, 低电平有效
7:0	write_arc4_valid	W5AT	0	0x5A: 写入 SYS_ALG_RST_CTRL4 有效 其他: 写入 SYS_ALG_RST_CTRL4 无效

9.3.40 系统 CSR_SYS_CIPHER_CTRL

Table 90: 系统 CSR_SYS_CIPHER_CTRL (Register offset: 0x3400)

Bits	Name	Access Type	Default	Description
31:2	rsv	RO	0	Reserved, read as 0
1	bus_cipher_en	RW1T	0	1: 使能 Bus Cipher 0: 无作用
0	ram_cipher_en	RW1T	0	1: 使能 RAM Cipher 0: 无作用

9.3.41 系统 CSR SYS_SOC_MEM_BAL

Table 91: 系统 CSR SYS_SOC_MEM_BAL (Register offset: 0x3800)

Bits	Name	Access Type	Default	Description
31:0	soc_mem_bal	RW	0	SoC Memory 访问基地址低 32 位

9.3.42 系统 CSR SYS_SOC_MEM_BAH

Table 92: 系统 CSR SYS_SOC_MEM_BAH (Register offset: 0x3804)

Bits	Name	Access Type	Default	Description
31:0	soc_mem_bah	RW	0	SoC Memory 访问基地址高 32 位 例如: 如果访问 SoC 空间 (参见 Figure 2) 总线地址位宽为 64 位: 总线地址 [63:30] = {soc_mem_bah[31:0], soc_mem_bal[31:30]} + {33'h0, haddrm[30]}; 总线地址 [29:0] = haddrm[29:0]; haddrm[31:0] 来自 eHSM 的 CPU

9.3.43 系统 CSR SYS_TRNG_DRBG_ALG

Table 93: 系统 CSR SYS_TRNG_DRBG_ALG (Register offset: 0x3900)

Bits	Name	Access Type	Default	Description
31:1	rsv	RO	0	Reserved, read as 0
0	TRNG_DRBG_ALG	RW	0	TRNG DRBG 算法选择 0x0: SM4 0x1: AES

9.3.44 系统 CSR SYS_CPU_CFG

Table 94: 系统 CSR SYS_CPU_CFG (Register offset: 0x4000)

Bits	Name	Access Type	Default	Description
31:26	rsv	RO	0	Reserved, read as 0
25:0	CPU_STCALIB_CFG	RW	0	CPU System Tick 校准配置

9.3.45 系统 CSR SYS_BUS_ERR_CFG

Table 95: 系统 CSR SYS_BUS_ERR_CFG (Register offset: 0x5000)

Bits	Name	Access Type	Default	Description
31:11	rsv	RO	0	Reserved, read as 0
10	ahb_cfg_rsp_err_en	RW	0	使能 AHB CFG 总线响应错误, 高电平有效。
9:7	rsv	RO	0	Reserved, read as 0
6	ahb_nvm_rsp_err_en	RW	0	使能 AHB NVM 总线响应错误, 高电平有效。
5	rsv	RO	0	Reserved, read as 0
4	ahb_otp_rsp_err_en	RW	0	使能 AHB OTP 总线响应错误, 高电平有效。
3:1	rsv	RO	0	Reserved, read as 0
0	ahb_soc_rsp_err_en	RW	0	使能 AHB SoC memory 总线响应错误, 高电平有效。

9.3.46 系统 CSR SYS_BUS_PR_CFG

Table 96: 系统 CSR SYS_BUS_PR_CFG (Register offset: 0x5100)

Bits	Name	Access Type	Default	Description
31:1	rsv	RO	0	Reserved, read as 0
0	ahb_bus_pr_en	RW	0	使能 AHB 总线 parity 错误告警产生, 高电平有效

9.3.47 系统 CSR SYS_AHB_DMA_CFG

Table 97: 系统 CSR SYS_AHB_DMA_CFG (Register offset: 0x6000)

Bits	Name	Access Type	Default	Description
31:16	rsv	RO	0	Reserved, read as 0
15:8	ahb_dma_rd_wr_to	RW	0	AHB DMA 读写总线的超时时钟周期数配置
7:4	rsv	RO	0	Reserved, read as 0
3	otp_ahb_if_sel	RW	0	OTP AHB 接口选择 1: Boot 0: AHB Bus Matrix
2:1	dma_wr_ctl	RW	3	00: 选择 AHB_DMA 写和 AXI_DMA 读 01: 选择 AHB_DMA 读和 AXI_DMA 写 10: 选择 AHB_DMA 读和写 11: 选择 AXI_DMA 读和写
0	dc_dma_sel	RW	0	用于解密固件的 DMA 选择 1: HASH AHB DMA 0: SKE AHB DMA

注意:

- **AHB_DMA_CFG 寄存器 [2:0]:** 只能用于需要在 DMA 模式下切换算法或切换 DMA 使用方式时设置, 如果在错误的时间设置 (例如: CPU 模式) 将会导致错误。

9.3.48 系统 CSR SYS_AXI_DMA_CFG0

Table 98: 系统 CSR SYS_AXI_DMA_CFG0 (Register offset: 0x6800)

Bits	Name	Access Type	Default	Description
31:24	axi_dma_wost_max	RO	0	AXI DMA write outstanding 的最大次数 (此配置依项目情况而定, 不能更改)
23:16	axi_dma_rost_max	RO	0	AXI DMA read outstanding 的最大次数 (此配置依项目情况而定, 不能更改)
15:8	axi_dma_wburst_max	RO	0	AXI DMA write burst 的最大次数 (此配置依项目情况而定, 不能更改)
7:0	axi_dma_rburst_max	RO	0	AXI DMA read burst 的最大次数 (此配置依项目情况而定, 不能更改)

9.3.49 系统 CSR SYS_AXI_DMA_CFG1

Table 99: 系统 CSR SYS_AXI_DMA_CFG1 (Register offset: 0x6804)

Bits	Name	Access Type	Default	Description
31:16	rsv	RO	0	Reserved, read as 0
15:0	axi_dma_rd_wr_to	RW	0	AXI DMA 读写总线的超时时钟数, 包括读写地址和数据总线

9.3.50 系统 CSR SYS_AXI_DMA_CH0_CFG0

Table 100: 系统 CSR SYS_AXI_DMA_CH0_CFG0 (Register offset: 0x6880)

Bits	Name	Access Type	Default	Description
31:16	axi_dma_ch0_rtrans_max	RO	0	AXI DMA read burst 的最大字节数 (通道 0) (此配置依项目情况而定, 不能更改)
15:0	axi_dma_ch0_wtrans_max	RO	0	AXI DMA write burst 的最大字节数 (通道 0) (此配置依项目情况而定, 不能更改)

9.3.51 系统 CSR SYS_AXI_DMA_CH0_CFG1

Table 101: 系统 CSR SYS_AXI_DMA_CH0_CFG1 (Register offset: 0x6884)

Bits	Name	Access Type	Default	Description
31:28	axi_dma_ch0_arsnoop	RW	0	AXI DMA ar snoop 配置（通道 0）
27:26	axi_dma_ch0_arbar	RW	0	AXI DMA arbar 配置（通道 0）
25:24	axi_dma_ch0_ardomain	RW	0	AXI DMA ardomain 配置（通道 0）
23:18	rsv	RO	0	Reserved, read as 0
17:16	axi_dma_ch0_arlock	RW	0	AXI DMA arlock 配置（通道 0）
15:12	axi_dma_ch0_arcache	RW	0	AXI DMA ar cache 配置（通道 0）
11	rsv	RO	0	Reserved, read as 0
10:8	axi_dma_ch0_arprot	RW	0	AXI DMA ar prot 配置（通道 0）
7:4	axi_dma_ch0_arqos	RW	0	AXI DMA ar qos 配置（通道 0）
3:0	axi_dma_ch0_arregion	RW	0	AXI DMA ar region 配置（通道 0）

9.3.52 系统 CSR SYS_AXI_DMA_CH0_CFG2

Table 102: 系统 CSR SYS_AXI_DMA_CH0_CFG2 (Register offset: 0x6888)

Bits	Name	Access Type	Default	Description
31	rsv	RO	0	Reserved, read as 0
30:28	axi_dma_ch0_awsnoop	RW	0	AXI DMA aw snoop 配置（通道 0）
27:26	axi_dma_ch0_awbar	RW	0	AXI DMA awbar 配置（通道 0）
25:24	axi_dma_ch0_awdomain	RW	0	AXI DMA awdomain 配置（通道 0）
23:18	rsv	RO	0	Reserved, read as 0
17:16	axi_dma_ch0_awlock	RW	0	AXI DMA awlock 配置（通道 0）
15:12	axi_dma_ch0_awcache	RW	0	AXI DMA aw cache 配置（通道 0）
11	rsv	RO	0	Reserved, read as 0
10:8	axi_dma_ch0_awprot	RW	0	AXI DMA aw prot 配置（通道 0）
7:4	axi_dma_ch0_awqos	RW	0	AXI DMA aw qos 配置（通道 0）
3:0	axi_dma_ch0_awregion	RW	0	AXI DMA aw region 配置（通道 0）

9.3.53 系统 CSR SYS_AXI_DMA_CH1_CFG0

Table 103: 系统 CSR SYS_AXI_DMA_CH1_CFG0 (Register offset: 0x68A0)

Bits	Name	Access Type	Default	Description
31:16	axi_dma_ch1_rtrans_max	RO	0	AXI DMA read burst 的最大字节数（通道 1） (此配置依项目情况而定，不能更改)
15:0	axi_dma_ch1_wtrans_max	RO	0	AXI DMA write burst 的最大字节数（通道 1） (此配置依项目情况而定，不能更改)

9.3.54 系统 CSR SYS_AXI_DMA_CH1_CFG1

Table 104: 系统 CSR SYS_AXI_DMA_CH1_CFG1 (Register offset: 0x68A4)

Bits	Name	Access Type	Default	Description
31:28	axi_dma_ch1_arsnoop	RW	0	AXI DMA ar snoop 配置（通道 1）
27:26	axi_dma_ch1_arbar	RW	0	AXI DMA arbar 配置（通道 1）
25:24	axi_dma_ch1_ardomain	RW	0	AXI DMA ardomain 配置（通道 1）
23:18	rsv	RO	0	Reserved, read as 0
17:16	axi_dma_ch1_arlock	RW	0	AXI DMA arlock 配置（通道 1）
15:12	axi_dma_ch1_arcache	RW	0	AXI DMA ar cache 配置（通道 1）
11	rsv	RO	0	Reserved, read as 0
10:8	axi_dma_ch1_arprot	RW	0	AXI DMA ar prot 配置（通道 1）
7:4	axi_dma_ch1_arqos	RW	0	AXI DMA ar qos 配置（通道 1）
3:0	axi_dma_ch1_arregion	RW	0	AXI DMA ar region 配置（通道 1）

9.3.55 系统 CSR SYS_AXI_DMA_CH1_CFG2

Table 105: 系统 CSR SYS_AXI_DMA_CH1_CFG2 (Register offset: 0x68A8)

Bits	Name	Access Type	Default	Description
31	rsv	RO	0	Reserved, read as 0
30:28	axi_dma_ch1_awsnoop	RW	0	AXI DMA aw snoop 配置（通道 1）
27:26	axi_dma_ch1_awbar	RW	0	AXI DMA awbar 配置（通道 1）
25:24	axi_dma_ch1_awdomain	RW	0	AXI DMA awdomain 配置（通道 1）
23:18	rsv	RO	0	Reserved, read as 0
17:16	axi_dma_ch1_awlock	RW	0	AXI DMA awlock 配置（通道 1）
15:12	axi_dma_ch1_awcache	RW	0	AXI DMA aw cache 配置（通道 1）
11	rsv	RO	0	Reserved, read as 0
10:8	axi_dma_ch1_awprot	RW	0	AXI DMA aw prot 配置（通道 1）
7:4	axi_dma_ch1_awqos	RW	0	AXI DMA aw qos 配置（通道 1）
3:0	axi_dma_ch1_awregion	RW	0	AXI DMA aw region 配置（通道 1）

9.3.56 系统 CSR SYS_AXI_DMA_CH2_CFG0

Table 106: 系统 CSR SYS_AXI_DMA_CH2_CFG0 (Register offset: 0x68C0)

Bits	Name	Access Type	Default	Description
31:16	axi_dma_ch2_rtrans_max	RO	0	AXI DMA read burst 的最大字节数（通道 2） (此配置依项目情况而定，不能更改)
15:0	axi_dma_ch2_wtrans_max	RO	0	AXI DMA write burst 的最大字节数（通道 2） (此配置依项目情况而定，不能更改)

9.3.57 系统 CSR_SYS_AXI_DMA_CH2_CFG1

Table 107: 系统 CSR_SYS_AXI_DMA_CH2_CFG1 (Register offset: 0x68C4)

Bits	Name	Access Type	Default	Description
31:28	axi_dma_ch2_arsnoop	RW	0	AXI DMA ar snoop 配置（通道 2）
27:26	axi_dma_ch2_arbar	RW	0	AXI DMA arbar 配置（通道 2）
25:24	axi_dma_ch2_ardomain	RW	0	AXI DMA ardomain 配置（通道 2）
23:18	rsv	RO	0	Reserved, read as 0
17:16	axi_dma_ch2_arlock	RW	0	AXI DMA arlock 配置（通道 2）
15:12	axi_dma_ch2_arcache	RW	0	AXI DMA ar cache 配置（通道 2）
11	rsv	RO	0	Reserved, read as 0
10:8	axi_dma_ch2_arprot	RW	0	AXI DMA ar prot 配置（通道 2）
7:4	axi_dma_ch2_arqos	RW	0	AXI DMA ar qos 配置（通道 2）
3:0	axi_dma_ch2_arregion	RW	0	AXI DMA ar region 配置（通道 2）

9.3.58 系统 CSR_SYS_AXI_DMA_CH2_CFG2

Table 108: 系统 CSR_SYS_AXI_DMA_CH2_CFG2 (Register offset: 0x68C8)

Bits	Name	Access Type	Default	Description
31	rsv	RO	0	Reserved, read as 0
30:28	axi_dma_ch2_awsnoop	RW	0	AXI DMA aw snoop 配置（通道 2）
27:26	axi_dma_ch2_awbar	RW	0	AXI DMA awbar 配置（通道 2）
25:24	axi_dma_ch2_awdomain	RW	0	AXI DMA awdomain 配置（通道 2）
23:18	rsv	RO	0	Reserved, read as 0
17:16	axi_dma_ch2_awlock	RW	0	AXI DMA awlock 配置（通道 2）
15:12	axi_dma_ch2_awcache	RW	0	AXI DMA aw cache 配置（通道 2）
11	rsv	RO	0	Reserved, read as 0
10:8	axi_dma_ch2_awprot	RW	0	AXI DMA aw prot 配置（通道 2）
7:4	axi_dma_ch2_awqos	RW	0	AXI DMA aw qos 配置（通道 2）
3:0	axi_dma_ch2_awregion	RW	0	AXI DMA aw region 配置（通道 2）

9.3.59 系统 CSR_SYS_AXI_DMA_CH3_CFG0

Table 109: 系统 CSR_SYS_AXI_DMA_CH3_CFG0 (Register offset: 0x68E0)

Bits	Name	Access Type	Default	Description
31:16	axi_dma_ch3_rtrans_max	RO	0	AXI DMA read burst 的最大字节数（通道 3） (此配置依项目情况而定, 不能更改)
15:0	axi_dma_ch3_wtrans_max	RO	0	AXI DMA write burst 的最大字节数（通道 3） (此配置依项目情况而定, 不能更改)

9.3.60 系统 CSR_SYS_AXI_DMA_CH3_CFG1

Table 110: 系统 CSR_SYS_AXI_DMA_CH3_CFG1 (Register offset: 0x68E4)

Bits	Name	Access Type	Default	Description
31:28	axi_dma_ch3_arsnoop	RW	0	AXI DMA ar snoop 配置（通道 3）
27:26	axi_dma_ch3_arbar	RW	0	AXI DMA arbar 配置（通道 3）
25:24	axi_dma_ch3_ardomain	RW	0	AXI DMA ardomain 配置（通道 3）
23:18	rsv	RO	0	Reserved, read as 0
17:16	axi_dma_ch3_arlock	RW	0	AXI DMA arlock 配置（通道 3）
15:12	axi_dma_ch3_arcache	RW	0	AXI DMA ar cache 配置（通道 3）
11	rsv	RO	0	Reserved, read as 0
10:8	axi_dma_ch3_arprot	RW	0	AXI DMA ar prot 配置（通道 3）
7:4	axi_dma_ch3_arqos	RW	0	AXI DMA ar qos 配置（通道 3）
3:0	axi_dma_ch3_arregion	RW	0	AXI DMA ar region 配置（通道 3）

9.3.61 系统 CSR_SYS_AXI_DMA_CH3_CFG2

Table 111: 系统 CSR_SYS_AXI_DMA_CH3_CFG2 (Register offset: 0x68E8)

Bits	Name	Access Type	Default	Description
31	rsv	RO	0	Reserved, read as 0
30:28	axi_dma_ch3_awsnoop	RW	0	AXI DMA aw snoop 配置（通道 3）
27:26	axi_dma_ch3_awbar	RW	0	AXI DMA awbar 配置（通道 3）
25:24	axi_dma_ch3_awdomain	RW	0	AXI DMA awdomain 配置（通道 3）
23:18	rsv	RO	0	Reserved, read as 0
17:16	axi_dma_ch3_awlock	RW	0	AXI DMA awlock 配置（通道 3）
15:12	axi_dma_ch3_awcache	RW	0	AXI DMA aw cache 配置（通道 3）
11	rsv	RO	0	Reserved, read as 0
10:8	axi_dma_ch3_awprot	RW	0	AXI DMA aw prot 配置（通道 3）
7:4	axi_dma_ch3_awqos	RW	0	AXI DMA aw qos 配置（通道 3）
3:0	axi_dma_ch3_awregion	RW	0	AXI DMA aw region 配置（通道 3）

注意：

- **axlock**：当 lock=1 时，AXI 总线启动原子访问，用户必须确保地址与突发长度对齐，以符合 AXI 协议。

9.3.62 系统 CSR SYS_CPU_ECC_CTRL0(CPU IROM)

Table 112: 系统 CSR SYS_CPU_ECC_CTRL0(CPU IROM) (Register offset: 0x7000)

Bits	Name	Access Type	Default	Description
31:24	cpu_irom_ecc_ckbits	RW	0	CPU IROM ECC 测试检查位
23:16	cpu_irom_ecc_tm_en	RW	0	0x5A: 使能 CPU IROM ECC 测试模式 其他: 关闭 CPU IROM ECC 测试模式
15:8	cpu_irom_ecc_err_rsp_en	RW	0x5A	0x5A: 关闭 CPU IROM ECC 检查错误响应 其他: 使能 CPU IROM ECC 检查错误响应
7:0	cpu_irom_ecc_ck_en	RW	0	0x5A: 关闭 CPU IROM ECC 检查 其他: 使能 CPU IROM ECC 检查

注意:

- 对于 irom ECC 检查, 'ckbits' 和 'tm_en' 只在 FPGA 模式下有效。使用实际 ROM 时, 只有 'err_rsp' 和 'ck_en' 控制有效。

9.3.63 系统 CSR SYS_CPU_ECC_CTRL1(CPU IRAM)

Table 113: 系统 CSR SYS_CPU_ECC_CTRL1(CPU IRAM) (Register offset: 0x7004)

Bits	Name	Access Type	Default	Description
31:24	cpu_iram_ecc_ckbits	RW	0	CPU IRAM ECC 测试检查位
23:16	cpu_iram_ecc_tm_en	RW	0	0x5A: 使能 CPU IRAM ECC 测试模式 其他: 关闭 CPU IRAM ECC 测试模式
15:8	cpu_iram_ecc_err_rsp_en	RW	0x5A	0x5A: 关闭 CPU IRAM ECC 检查错误响应 其他: 使能 CPU IRAM ECC 检查错误响应
7:0	cpu_iram_ecc_ck_en	RW	0	0x5A: 关闭 CPU IRAM ECC 检查 其他: 使能 CPU IRAM ECC 检查

9.3.64 系统 CSR SYS_CPU_ECC_CTRL2(CPU DRAM)

Table 114: 系统 CSR SYS_CPU_ECC_CTRL2(CPU DRAM) (Register offset: 0x7008)

Bits	Name	Access Type	Default	Description
31:24	cpu_dram_ecc_ckbits	RW	0	CPU DRAM ECC 测试检查位
23:16	cpu_dram_ecc_tm_en	RW	0	0x5A: 使能 CPU DRAM ECC 测试模式 其他: 关闭 CPU DRAM ECC 测试模式
15:8	cpu_dram_ecc_err_rsp_en	RW	0x5A	0x5A: 关闭 CPU DRAM ECC 检查错误响应 其他: 使能 CPU DRAM ECC 检查错误响应
7:0	cpu_dram_ecc_ck_en	RW	0	0x5A: 关闭 CPU DRAM ECC 检查 其他: 使能 CPU DRAM ECC 检查

9.3.65 系统 CSR SYS_KMU_ECC_CTRL(KMU RAM)

Table 115: 系统 CSR SYS_KMU_ECC_CTRL(KMU RAM) (Register offset: 0x7100)

Bits	Name	Access Type	Default	Description
31:26	rsv	RO	0	Reserved, read as 0
25:16	kmu_ram_ecc_ckbits	RW	0	KMU RAM ECC 测试检查位
15:8	kmu_ram_ecc_tm_en	RW	0	0x5A: 使能 KMU RAM ECC 测试模式 其他: 关闭 KMU RAM ECC 测试模式
7:0	kmu_ram_ecc_ck_en	RW	0	0x5A: 关闭 KMU RAM ECC 检查 其他: 使能 KMU RAM ECC 检查

9.3.66 系统 CSR SYS_PKE_ECC_CTRL0(PKE RAM0)

Table 116: 系统 CSR SYS_PKE_ECC_CTRL0(PKE RAM0) (Register offset: 0x7200)

Bits	Name	Access Type	Default	Description
31:26	rsv	RO	0	Reserved, read as 0
25:16	pke_ram0_ecc_ckbits	RW	0	PKE RAM0 ECC 测试检查位
15:8	pke_ram0_ecc_tm_en	RW	0	0x5A: 使能 PKE RAM0 ECC 测试模式 其他: 关闭 PKE RAM0 ECC 测试模式
7:0	pke_ram0_ecc_ck_en	RW	0	0x5A: 关闭 PKE RAM0 ECC 检查 其他: 使能 PKE RAM0 ECC 检查

9.3.67 系统 CSR SYS_PKE_ECC_CTRL1(PKE RAM1)

Table 117: 系统 CSR SYS_PKE_ECC_CTRL1(PKE RAM1) (Register offset: 0x7204)

Bits	Name	Access Type	Default	Description
31:26	rsv	RO	0	Reserved, read as 0
25:16	pke_ram1_ecc_ckbits	RW	0	PKE RAM1 ECC 测试检查位
15:8	pke_ram1_ecc_tm_en	RW	0	0x5A: 使能 PKE RAM1 ECC 测试模式 其他: 关闭 PKE RAM1 ECC 测试模式
7:0	pke_ram1_ecc_ck_en	RW	0	0x5A: 关闭 PKE RAM1 ECC 检查 其他: 使能 PKE RAM1 ECC 检查

9.3.68 系统 CSR SYS_PKE_ECC_CTRL2(PKE RAM2)

Table 118: 系统 CSR SYS_PKE_ECC_CTRL2(PKE RAM2) (Register offset: 0x7208)

Bits	Name	Access Type	Default	Description
31:26	rsv	RO	0	Reserved, read as 0
25:16	pke_ram2_ecc_ckbits	RW	0	PKE RAM2 ECC 测试检查位
15:8	pke_ram2_ecc_tm_en	RW	0	0x5A: 使能 PKE RAM2 ECC 测试模式 其他: 关闭 PKE RAM2 ECC 测试模式
7:0	pke_ram2_ecc_ck_en	RW	0	0x5A: 关闭 PKE RAM2 ECC 检查 其他: 使能 PKE RAM2 ECC 检查

9.3.69 系统 CSR SYS_PKE_ECC_CTRL3(PKE RAM3)

Table 119: 系统 CSR SYS_PKE_ECC_CTRL3(PKE RAM3) (Register offset: 0x720C)

Bits	Name	Access Type	Default	Description
31:26	rsv	RO	0	Reserved, read as 0
25:16	pke_ram3_ecc_ckbits	RW	0	PKE RAM3 ECC 测试检查位
15:8	pke_ram3_ecc_tm_en	RW	0	0x5A: 使能 PKE RAM3 ECC 测试模式 其他: 关闭 PKE RAM3 ECC 测试模式
7:0	pke_ram3_ecc_ck_en	RW	0	0x5A: 关闭 PKE RAM3 ECC 检查 其他: 使能 PKE RAM3 ECC 检查

ECC 测试用户工作流程:

1. 在'ecc_ckbits' 寄存器中设置测试检查位。
2. 在'ecc_tm_en' 寄存器中使能测试模式。
3. 向 RAM 写入数据然后读出, 将有一个 ECC 错误标志发送到 EMU。
4. 在'ecc_tm_en' 寄存器中关闭测试模式。
5. 向 RAM 写入数据然后读出, 没有 ECC 错误标志发送到 EMU 表明 ECC 功能正常运行。

9.3.70 系统 CSR LIFE_CYCLE

Table 120: 系统 CSR LIFE_CYCLE (Register offset: 0x8000)

Bits	Name	Access Type	Default	Description
31:0	life_cycle	RO	0	生命周期 参见4

9.3.71 系统 CSR HW_CONTROL0

Table 121: 系统 CSR HW_CONTROL0 (Register offset: 0x8080)

Bits	Name	Access Type	Default	Description
31:0	hw_control0	RO	0	OTP 硬件控制域: Bits[31:0] 参见 Table 35

9.3.72 系统 CSR HW_CONTROL1

Table 122: 系统 CSR HW_CONTROL1 (Register offset: 0x8084)

Bits	Name	Access Type	Default	Description
31:0	hw_control1	RO	0	OTP 硬件控制域: Bits[63:32]

9.3.73 系统 CSR FW_CONTROL0

Table 123: 系统 CSR FW_CONTROL0 (Register offset: 0x8100)

Bits	Name	Access Type	Default	Description
31:0	fw_control0	RO	0	OTP 固件控制域: Bits[31:0]

9.3.74 系统 CSR FW_CONTROL1

Table 124: 系统 CSR FW_CONTROL1 (Register offset: 0x8104)

Bits	Name	Access Type	Default	Description
31:0	fw_control1	RO	0	OTP 固件控制域: Bits[63:32]

9.3.75 系统 CSR FW_CONTROL2

Table 125: 系统 CSR FW_CONTROL2 (Register offset: 0x8108)

Bits	Name	Access Type	Default	Description
31:0	fw_control2	RO	0	OTP 固件控制域: Bits[95:64]

9.3.76 系统 CSR FW_CONTROL3

Table 126: 系统 CSR FW_CONTROL3 (Register offset: 0x810C)

Bits	Name	Access Type	Default	Description
31:0	fw_control3	RO	0	OTP 固件控制域: Bits[127:96]

9.3.77 系统 CSR SYS_GEN_REG[N]

Table 127: 系统 CSR SYS_GEN_REG[N] (Register offset: 0xF0800)

Bits	Name	Access Type	Default	Description
31:0	gen_reg[N]	RW	0	通用寄存器 N(0-15)

9.3.78 系统 CSR SYS_RUN_STEP

Table 128: 系统 CSR SYS_RUN_STEP (Register offset: 0xF0F00)

Bits	Name	Access Type	Default	Description
31:0	sys_run_step	RW	0	固件运行步骤

10 错误管理单元

10.1 EMU 介绍

EMU 从硬件 (HW) 和固件 (FW) 接收错误信息，并将其发送到 SoC。每个硬件错误事件都将触发一个中断到 eHSM CPU。

- Memory ECC 错误
 - 一位可纠正警告
 - 两位不可纠正错误
- WDT 超时
- 传感器报警
- TRNG 健康测试失败
- 算法自检失败
- SoC 总线响应错误
- OTP 密钥错误
- 安全启动失败
- 总线奇偶校验错误
- DMA 控制器总线超时

10.2 EMU 特性

1. 具有一个 AHB Slave 接口
2. 对于硬件支持 64 位错误事件输入
 - (a) 每个错误事件将触发一个中断
3. 对于固件支持 64 位错误事件输入
4. 所有硬件/固件错误信息将被发送到 SoC
 - (a) 硬件错误将直接发送到 SoC
 - (b) 固件错误通过读写内部 CSR 发送到 SoC

10.3 EMU 寄存器

10.3.1 EMU 寄存器映射

EMU 的基地址参见: [Table 32](#)

OSR CONFIDENTIAL

Table 129: EMU 寄存器映射

Register	Offset	Description
ERR_SENSOR	0x0000	传感器错误
ERR_SOC_ERR	0x0004	SoC 错误
ERR_HW_0	0x0008	硬件错误低位
ERR_HW_1	0x000C	硬件错误高位
ERR_FW_0	0x0010	固件错误低位
ERR_FW_1	0x0014	固件错误高位
INT_SENSOR	0x0020	传感器错误中断
INT_SOC_ERR	0x0024	SoC 错误中断
INT_HW_0	0x0028	硬件错误低位中断
INT_HW_1	0x002C	硬件错误高位中断
INTEN_SENSOR	0x0040	传感器或错误中断使能
INTEN_SOC_ERR	0x0044	SoC 错误中断使能
INTEN_HW_0	0x0048	硬件错误低位中断使能
INTEN_HW_1	0x004C	硬件错误高位中断使能
MEMECC_ADDR_0	0x0080	IRAM 存储器 ECC 地址
MEMECC_ADDR_1	0x0084	IROM 存储器 ECC 地址
MEMECC_ADDR_2	0x0088	DRAM 存储器 ECC 地址
MEMECC_ADDR_3	0x008C	KMU RAM 存储器 ECC 地址
MEMECC_ADDR_4	0x0090	PKE RAM 0 存储器 ECC 地址
MEMECC_ADDR_5	0x0094	PKE RAM 1 存储器 ECC 地址
MEMECC_ADDR_6	0x0098	PKE RAM 2 存储器 ECC 地址
MEMECC_ADDR_7	0x009C	PKE RAM 3 存储器 ECC 地址
RST_ALL_SENSOR	0x0100	传感器错误的 eHSM 复位使能
RST_ALL_SOC_ERR	0x0104	SoC 错误的 eHSM 复位使能
RST_ALL_HW_0	0x0108	硬件错误低位的 eHSM 复位使能
RST_ALL_HW_1	0x010C	硬件错误高位的 eHSM 复位使能
RST_NBT_SENSOR	0x0110	传感器错误的 eHSM 复位使能（不包括 Boot 模块）
RST_NBT_SOC_ERR	0x0114	SoC 错误的 eHSM 复位使能（不包括 Boot 模块）
RST_NBT_HW_0	0x0118	硬件错误低位的 eHSM 复位使能（不包括 Boot 模块）
RST_NBT_HW_1	0x011C	硬件错误高位的 eHSM 复位使能（不包括 Boot 模块）
RST_CPU_SENSOR	0x0120	传感器错误的 CPU 复位使能
RST_CPU_SOC_ERR	0x0124	SoC 错误的 CPU 复位使能
RST_CPU_HW_0	0x0128	硬件错误低位的 CPU 复位使能
RST_CPU_HW_1	0x012C	硬件错误高位的 CPU 复位使能
RST_ENABLE	0x0130	EMU 复位使能
RST_MODE_FLAG	0x0134	EMU 复位标志
ERR_HW_TRIG_0	0x0140	硬件错误低位触发
ERR_HW_TRIG_1	0x0144	硬件错误高位触发
ERR_TRIG_LOCK	0x0180	硬件错误触发寄存器锁

10.3.2 EMU_ERR_SENSOR 寄存器

Table 130: EMU_ERR_SENSOR 寄存器

Bits	Name	Access Type	Default	Description
31:0	err_sensor	RO	0x0	传感器错误, 参见 Table 7

10.3.3 EMU_ERR_SOC_ERR 寄存器

Table 131: EMU_ERR_SOC_ERR 寄存器

Bits	Name	Access Type	Default	Description
31:0	err_soc_err	RO	0x0	SoC 错误, refer to Table 9

10.3.4 EMU_ERR_HW_0 寄存器

Table 132: EMU_ERR_HW_0 寄存器

Bits	Name	Access Type	Default	Description
31:0	err_hw_0	RO	0x0	硬件错误 (bit 31-0), 参见 Table 11

10.3.5 EMU_ERR_HW_1 寄存器

Table 133: EMU_ERR_HW_1 寄存器

Bits	Name	Access Type	Default	Description
31:0	err_hw_1	RO	0x0	硬件错误 (bit 63-32), 参见 Table 11

10.3.6 EMU_ERR_FW_0 寄存器

Table 134: EMU_ERR_FW_0 寄存器

Bits	Name	Access Type	Default	Description
31:0	err_fw_0	RW	0x0	固件错误 (bit 31-0), 参见 Table 12

10.3.7 EMU_ERR_FW_1 寄存器

Table 135: EMU_ERR_FW_1 寄存器

Bits	Name	Access Type	Default	Description
31:0	err_fw_1	RW	0x0	固件错误 (bit 63-32), 参见 Table 12

10.3.8 EMU INT_SENSOR 寄存器

Table 136: EMU INT_SENSOR 寄存器

Bits	Name	Access Type	Default	Description
31:0	int_sensor	RW1C	0x0	传感器错误中断

10.3.9 EMU INT_SOC_ERR 寄存器

Table 137: EMU INT_SOC_ERR 寄存器

Bits	Name	Access Type	Default	Description
31:0	int_soc_err	RW1C	0x0	SoC 错误中断

10.3.10 EMU INT_HW_0 寄存器

Table 138: EMU INT_HW_0 寄存器

Bits	Name	Access Type	Default	Description
31:0	int_hw_0	RW1C	0x0	硬件错误（bit 31-0）中断

10.3.11 EMU INT_HW_1 寄存器

Table 139: EMU INT_HW_1 寄存器

Bits	Name	Access Type	Default	Description
31:0	int_hw_1	RW1C	0x0	硬件错误（bit 63-32）中断

10.3.12 EMU INTEN_SENSOR 寄存器

Table 140: EMU INTEN_SENSOR 寄存器

Bits	Name	Access Type	Default	Description
31:0	inten_sensor	RW	0x0	传感器错误中断使能, 高电平有效

10.3.13 EMU INTEN_SOC_ERR 寄存器

Table 141: EMU INTEN_SOC_ERR 寄存器

Bits	Name	Access Type	Default	Description
31:0	inten_soc_err	RW	0x0	SoC 错误中断使能, 高电平有效

10.3.14 EMU INTEN_HW_0 寄存器

Table 142: EMU INTEN_HW_0 寄存器

Bits	Name	Access Type	Default	Description
31:0	inten_hw_0	RW	0x0	硬件错误 (bit 31-0) 中断使能, 高电平有效

10.3.15 EMU INTEN_HW_1 寄存器

Table 143: EMU INTEN_HW_1 寄存器

Bits	Name	Access Type	Default	Description
31:0	inten_hw_1	RW	0x0	硬件错误 (bit 63-32) 中断使能, 高电平有效

10.3.16 EMU MEMECC_ADDR_0 寄存器

Table 144: EMU MEMECC_ADDR_0 寄存器

Bits	Name	Access Type	Default	Description
31:17	rsv	RO	0	Reserved, read as 0
16:0	memecc_addr_0	RO	0x0	IROM memory ECC 地址, 在遇到 1bit 或者多 bit 错误时更新

10.3.17 EMU MEMECC_ADDR_1 寄存器

Table 145: EMU MEMECC_ADDR_1 寄存器

Bits	Name	Access Type	Default	Description
31:17	rsv	RO	0	Reserved, read as 0
16:0	memecc_addr_1	RO	0x0	IRAM memory ECC 地址, 在遇到 1bit 或者多 bit 错误时更新

10.3.18 EMU MEMECC_ADDR_2 寄存器

Table 146: EMU MEMECC_ADDR_2 寄存器

Bits	Name	Access Type	Default	Description
31:17	rsv	RO	0	Reserved, read as 0
16:0	memecc_addr_2	RO	0x0	DRAM memory ECC 地址, 在遇到 1bit 或者多 bit 错误时更新

10.3.19 EMU MEMECC_ADDR_3 寄存器

Table 147: EMU MEMECC_ADDR_3 寄存器

Bits	Name	Access Type	Default	Description
31:17	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
16:0	memecc_addr_3	RO	0x0	KMU RAM memory ECC 地址, 在遇到 1bit 或者多 bit 错误时更新

10.3.20 EMU MEMECC_ADDR_4 寄存器

Table 148: EMU MEMECC_ADDR_4 寄存器

Bits	Name	Access Type	Default	Description
31:17	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
16:0	memecc_addr_4	RO	0x0	PKE RAM 0 memory ECC 地址, 在遇到 1bit 或者多 bit 错误时更新

10.3.21 EMU MEMECC_ADDR_5 寄存器

Table 149: EMU MEMECC_ADDR_5 寄存器

Bits	Name	Access Type	Default	Description
31:17	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
16:0	memecc_addr_5	RO	0x0	PKE RAM 1 memory ECC 地址, 在遇到 1bit 或者多 bit 错误时更新

10.3.22 EMU MEMECC_ADDR_6 寄存器

Table 150: EMU MEMECC_ADDR_6 寄存器

Bits	Name	Access Type	Default	Description
31:17	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
16:0	memecc_addr_6	RO	0x0	PKE RAM 2 memory ECC 地址, 在遇到 1bit 或者多 bit 错误时更新

10.3.23 EMU MEMECC_ADDR_7 寄存器

Table 151: EMU MEMECC_ADDR_7 寄存器

Bits	Name	Access Type	Default	Description
31:17	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
16:0	memecc_addr_7	RO	0x0	PKE RAM 3 memory ECC 地址, 在遇到 1bit 或者多 bit 错误时更新

10.3.24 EMU_RST_ALL_SENSOR 寄存器

Table 152: EMU_RST_ALL_SENSOR 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_all_sensor	RW	0x0	传感器错误的 eHSM 复位使能，高电平有效，参见 Table 7 获取每一个位的定义

10.3.25 EMU_RST_ALL_SOC_ERR 寄存器

Table 153: EMU_RST_ALL_SOC_ERR 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_all_soc_err	RW	0x0	SoC 错误的 eHSM 复位使能，高电平有效，参见 Table 9 获取每一个位的定义

10.3.26 EMU_RST_ALL_HW_0 寄存器

Table 154: EMU_RST_ALL_HW_0 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_all_hw_0	RW	0x0	当出现 eHSM 硬件错误（bit 31-0）的 eHSM 复位使能，高电平有效，参见 Table 11 获取每一个位的定义

10.3.27 EMU_RST_ALL_HW_1 寄存器

Table 155: EMU_RST_ALL_HW_1 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_all_hw_1	RW	0x0	当出现 eHSM 硬件错误（bit 63-32）的 eHSM 复位使能，高电平有效，参见 Table 11 获取每一个位的定义

10.3.28 EMU_RST_NBT_SENSOR 寄存器

Table 156: EMU_RST_NBT_SENSOR 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_nbt_sensor	RW	0x0	传感器错误的 eHSM 复位使能（不包括 Boot 模块），高电平有效，参见 Table 7 获取每一个位的定义

10.3.29 EMU_RST_NBT_SOC_ERR 寄存器

Table 157: EMU_RST_NBT_SOC_ERR 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_nbt_soc_err	RW	0x0	SoC 错误的 eHSM 复位使能（不包括 Boot 模块），高电平有效, 参见 Table 9 获取每一个位的定义

10.3.30 EMU_RST_NBT_HW_0 寄存器

Table 158: EMU_RST_NBT_HW_0 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_nbt_hw_0	RW	0x0	当出现 eHSM 硬件错误（bit 31-0）的 eHSM 复位使能（不包括 Boot 模块），高电平有效, 参见 Table 11 获取每一个位的定义

10.3.31 EMU_RST_NBT_HW_1 寄存器

Table 159: EMU_RST_NBT_HW_1 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_nbt_hw_1	RW	0x0	当出现 eHSM 硬件错误（bit 63-32）的 eHSM 复位使能（不包括 Boot 模块），高电平有效, 参见 Table 11 获取每一个位的定义

10.3.32 EMU_RST_CPU_SENSOR 寄存器

Table 160: EMU_RST_CPU_SENSOR 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_cpu_sensor	RW	0x0	传感器错误的 eHSM CPU 复位使能，高电平有效, 参见 Table 7 获取每一个位的定义

10.3.33 EMU_RST_CPU_SOC_ERR 寄存器

Table 161: EMU_RST_CPU_SOC_ERR 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_cpu_soc_err	RW	0x0	SoC 错误的 eHSM CPU 复位使能，高电平有效, 参见 Table 9 获取每一个位的定义

10.3.34 EMU RST_CPU_HW_0 寄存器

Table 162: EMU RST_CPU_HW_0 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_cpu_hw_0	RW	0x0	当出现, eHSM 硬件错误 (bit 31-0) 的 eHSM CPU 复位使能, 高电平有效, 参见 Table 11 获取每一个位的定义

10.3.35 EMU RST_CPU_HW_1 寄存器

Table 163: EMU RST_CPU_HW_1 寄存器

Bits	Name	Access Type	Default	Description
31:0	rst_cpu_hw_1	RW	0x0	当出现, eHSM 硬件错误 (bit 63-32) 的 eHSM CPU 复位使能, 高电平有效, 参见 Table 11 获取每一个位的定义

10.3.36 EMU RST_ENABLE 寄存器

Table 164: EMU RST_ENABLE 寄存器

Bits	Name	Access Type	Default	Description
31:17	rsv	RO	0	Reserved, read as 0
16	rst_cpu_enable	RW	0x0	CPU 复位使能, 高电平有效
15:9	rsv	RO	0	Reserved, read as 0
8	rst_noboot_enable	RW	0x0	全局复位使能 (不包括 Boot 模块), 高电平有效
7:1	rsv	RO	0	Reserved, read as 0
0	rst_all_enable	RW	0x0	全局复位使能, 高电平有效

10.3.37 EMU_RST_MODE_FLAG 寄存器

Table 165: EMU_RST_MODE_FLAG 寄存器

Bits	Name	Access Type	Default	Description
31:18	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
17	sys_soft_rst_cpu_flag	RW1C	0x0	系统 CPU 软复位标志 1: 复位发生 0: 无复位
16	emu_rst_cpu_flag	RW1C	0x0	EMU CPU 复位标志 1: 复位发生 0: 无复位
15:11	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
10	soc_rst_noboot_flag	RW1C	0x0	SoC 全局复位标志（不包括 Boot 模块） 1: 复位发生 0: 无复位
9	sys_soft_rst_noboot_flag	RW1C	0x0	系统全局软复位标志（不包括 Boot 模块） 1: 复位发生 0: 无复位
8	emu_rst_noboot_flag	RW1C	0x0	EMU 全局复位标志（不包括 Boot 模块） 1: 复位发生 0: 无复位
7:1	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
1	sys_soft_rst_all_flag	RW1C	0x0	系统全局软复位标志 1: 复位发生 0: 无复位
0	emu_rst_all_flag	RW1C	0x0	EMU 全局所有复位标志 1: 复位发生 0: 无复位

10.3.38 EMU_ERR_HW_TRIG_0 寄存器

Table 166: EMU_ERR_HW_TRIG_0 寄存器

Bits	Name	Access Type	Default	Description
31:0	err_hw_trig_0	W1T	0x0	写 1 触发硬件错误 (31-0 位), 每位的定义与 ERR_HW_0 寄存器相同, 参见 Table 11

10.3.39 EMU_ERR_HW_TRIG_1 寄存器

Table 167: EMU_ERR_HW_TRIG_1 寄存器

Bits	Name	Access Type	Default	Description
31:0	err_hw_trig_1	W1T	0x0	写 1 触发硬件错误 (63-32 位), 每位的定义与 ERR_HW_1 寄存器相同, 参见 Table 11

10.3.40 EMU_ERR_TRIG_LOCK 寄存器

Table 168: EMU_ERR_TRIG_LOCK 寄存器

Bits	Name	Access Type	Default	Description
31:0	err_trig_lock	RW	0x0	只有在该寄存器写入 0x965F_0CBA 后，ERR_HW_TRIG_0 和 ERR_HW_TRIG_1 寄存器才能被写入；否则无法写入 ERR_HW_TRIG_0 和 ERR_HW_TRIG_1 寄存器。

注意：

- 外部 AXI 总线主机：如果使能复位功能，当发生错误导致 CPU 复位，外部 AXI 总线主机也应复位。否则可能发生 AXI 总线握手失败或奇偶校验失败。
- 外部 AHB 总线主机：如果使能复位功能，当发生错误导致 CPU 复位，外部 AHB 总线主机也应复位。否则可能发生 AHB 总线握手失败或奇偶校验失败。
- 外部 AHB 总线从机：如果使能复位功能，当发生错误导致 CPU 复位，外部 AHB 总线从机也应复位。否则可能发生 AHB 总线握手失败或奇偶校验失败。
- EMU_RST_ENABLE (Table 164) 中的寄存器是每种复位的全局复位使能，包括 CPU 复位、不包含 Boot 模块的 eHSM 复位和 eHSM 全局复位。内部 CPU 可以配置这些寄存器为 1 来启用复位使能。每次复位发生时，EMU_RST_ENABLE 中的相关使能寄存器会复位为 0，内部 CPU 可以在重新配置使能寄存器之前检查 EMU_RST_MODE_FLAG 中的复位标志。

10.4 EMU 固件流程

10.4.1 示例：使能 WDT 复位 CPU

此处展示了一个在 WDT 超时复位 CPU 的示例（WDT 位于硬件的第 40 位）。

- Write "EMU_RST_ALL_HW_1.rst_all_hw_1[8]" = 0 (WDT 不会导致 eHSM 全局复位)
- Write "EMU_RST_NBT_HW_1.rst_nbt_hw_1[8]" = 0 (WDT 不会导致 eHSM 全局非启动复位)
- Write "EMU_RST_CPU_HW_1.rst_cpu_hw_1[8]" = 1 and Write "EMU_RST_ENABLE.rst_cpu_enable" = 0 (WDT 不会导致 eHSM CPU 复位)
- Write "EMU_RST_CPU_HW_1.rst_cpu_hw_1[8]" = 1 and Write "EMU_RST_ENABLE.rst_cpu_enable" = 1 (WDT 将导致 eHSM CPU 复位)

11 密钥管理单元

11.1 KMU 介绍

KMU（密钥管理单元）在 eHSM 运行时临时存储密钥。KMU 中存储的密钥来源：

- 在 eHSM 启动时由硬件从 OTP 解密到 KMU
- 通过密钥导入指令从外部导入到 KMU

11.2 KMU 特性

- RAM 加密是可选的，用于加密密钥存储
- Memory ECC 是可选的，用于保护密钥完整性
- 根据芯片生命周期对解密的 OTP 密钥进行访问控制

11.3 KMU 密钥访问

- 256 深度、32 位宽的 RAM 用于存储密钥；最大可存储 25 个密钥 $\lceil \text{int}(256/10) = 25 \rceil$ ，实际支持的密钥个数取决于客户配置。
- 在 eHSM 启动期间，硬件解密 OTP 密钥并根据 [Table 47](#) 将其存储在 KMU RAM 中。
- eHSM CPU 对 KMU RAM 的访问权限由 [Table 48](#) 决定。

11.4 KMU RAM 密钥槽

参考 [Table 37](#)。

Figure 22: KMU RAM 密钥槽

Field	Size/Word	Offset
Key 0	8	0x000
Key 1	8	0x020
...	...	...
Key 0 Attribute	1	0x320
Key 1 Attribute	1	0x324
...	...	...

11.5 KMU 寄存器

11.5.1 KMU 寄存器映射

KMU 的基地址参见：[Table 32](#)

Table 169: KMU 寄存器映射

Register	Offset	Description
KMU_CTRL	0x0000	KMU 控制寄存器
KMU_EX_CH	0x000C	KMU 外部安全端口通道选择
KMU_BUF[N]	0x2000+N*0x20	KMU 密钥
KMU_ATT[N]	0x4000+N*0x4	KMU 密钥属性

Note:

KMU_BUF[N] 的地址对应的是第 N 个 KMU key 的首地址，每个 KMU key 由 8 个 word 组成。

11.5.2 KMU KMU_CTRL

Table 170: KMU KMU_CTRL

Bits	Name	Access Type	Default	Description
31	key_destination_sel	RW	0	0: 内部密钥，用于 eHSM 1: 外部密钥，通过安全端口传输到 SoC
30:25	rsv	RO	0	Reserved, read as 0
24	key_swap	RW	0	0: 无作用 1: 交换密钥的高 128 位和低 128 位
23:16	key_sel	RW	0	选择要传输的密钥 0xFF: 选择 OSR_RTL_KEY_INSTALL_KEK_EHSM 0xFE: 选择 OSR_RTL_KEY_INSTALL_KEK_SOC 其他: 选择 KMU RAM 中的密钥
15:8	rsv	RO	0	Reserved, read as 0
7:4	in_port_sel	RW	0	内部安全端口选择。 0: 将密钥传输到 SKE0 1: 将密钥传输到 Hash0 其他: 保留
3:1	rsv	RO	0	Reserved, read as 0
0	start_key_transport	W1T	0	写入 1 启动安全端口密钥传输

Note:

使用上需注意，要求先配置完毕 KMU_CTRL 寄存器中 start_key_transport 以外的传输密钥的相关控制信息后，才能往 start_key_transport 寄存器写入 1 以启动安全端口的密钥传输。

11.5.3 KMU KMU_EX_CH

Table 171: KMU KMU_EX_CH

Bits	Name	Access Type	Default	Description
31:16	rsv	RO	0	Reserved, read as 0
15:0	kmu_ex_ch_sel	RW	0	KMU 外部安全端口通道选择

11.5.4 KMU KMU_KEY_EN

Table 172: KMU KMU_KEY_EN

Bits	Name	Access Type	Default	Description
31:0	kmu_key_en	RW0C	0xFFFFFFFF	此寄存器每一位控制对应 ID 的 KMU 密钥的安全端口传输使能。 0: 对应密钥无法通过安全端口传输 1: 对应密钥可通过安全端口传输

Note:

该寄存器的每一位默认值为高电平，即可通过安全端口传输该密钥，当写为低电平后，将无法再次拉高。

11.5.5 KMU KMU_BUF[N]

Table 173: KMU KMU_BUF[N]

Bits	Name	Access Type	Default	Description
31:0	kmu_buf[N]	RW	0	此寄存器包含了存储在 KMU RAM 中的密钥值

11.5.6 KMU KMU_ATT[N]

Table 174: KMU KMU_ATT[N]

Bits	Name	Access Type	Default	Description
31:0	kmu_att[N]	RW	0	此寄存器包含了存储在 KMU RAM 中的密钥属性。 参见 Table 39 Table 40

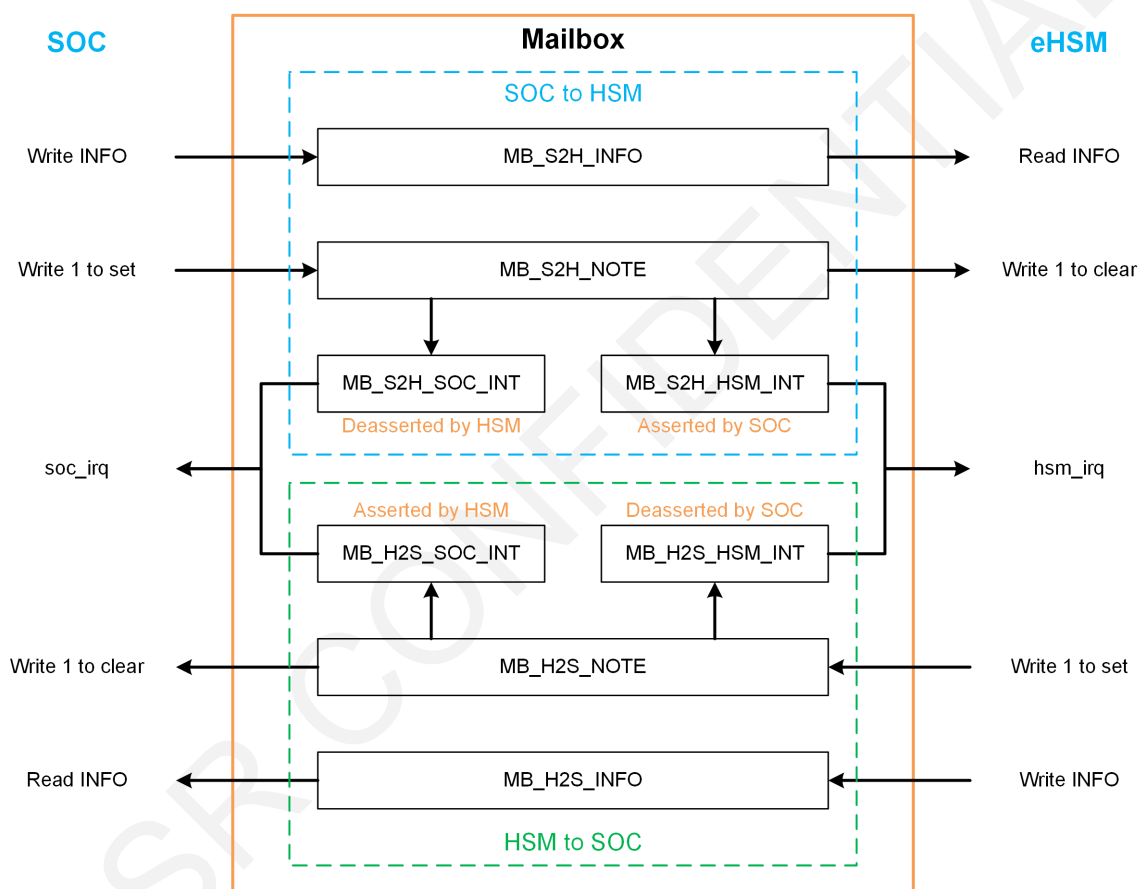
12 Mailbox

12.1 Mailbox 介绍

Mailbox 用于设置 SoC 和 eHSM 之间的通信，包括：

1. SoC 向 eHSM 发送信息，例如命令。
2. eHSM 向 SoC 发送信息，例如响应。

Figure 23: Mailbox 模块框图



12.2 Mailbox 特性

1. 支持两个 AHB Slave 接口，一个用于 SoC，另一个用于 eHSM
2. 支持中断输出，最大支持 16 个 Mailbox，对应 16 位中断用于 SoC，1 位中断用于 eHSM
3. 从 SoC 到 eHSM
 - (a) 支持最多 2 个 word 长度的命令
 - (b) 支持 1 个 SoC 到 eHSM 的通知
 - (c) 当 SoC 拉高 MB_S2H_NOTE 通知时，eHSM 的中断将被拉高
 - (d) 当 eHSM 拉低 MB_S2H_NOTE 通知时，SoC 的中断将被拉高

4. 从 eHSM 到 SoC

- (a) 支持最多 2 个 word 长度的 response 响应
- (b) 支持 1 个 eHSM 到 SoC 的通知
- (c) 当 eHSM 拉高 MB_H2S_NOTE 通知时，SoC 的中断将被拉高
- (d) 当 SoC 拉低 MB_H2S_NOTE 通知时，eHSM 的中断将被拉高

5. 当 SoC 和 eHSM 同时拉高/拉低相同的通知位时，会产生冲突中断

12.3 Mailbox 寄存器

Mailbox 有多个寄存器，用于支持 SoC 和 eHSM 之间的通信，包括：

1. SoC 到 eHSM

- (a) 命令
- (b) 中断
 - i. 到 SoC: eHSM 成功接收命令
 - ii. 到 eHSM: SoC 成功发送命令

2. eHSM 到 SoC 的响应

- (a) 响应
- (b) 中断
 - i. 到 SoC: eHSM 发送响应
 - ii. 到 eHSM: SoC 接收响应

12.3.1 Mailbox 寄存器映射

Mailbox 基地址: [Table 32](#)

Table 175: Mailbox 选择

Address	Value	Description
i_soc_haddr[15:12] 或 i_se_haddr[15:12]	N*0x1	选择 Mailbox[N] (N=0-F)

Note:

- 1. 实际的 Mailbox 的个数取决于客户需求，eHSM 最大支持 16 个 mailbox。
- 2. 当访问特定 Mailbox 时，使用 Mailbox AHB 总线的 haddr[15:12] 选择需要访问的 Mailbox。
- 3. 当访问特定 Mailbox 时，使用 Mailbox AHB 总线的 haddr[11:0] 进行内部寄存器的访问，每个 Mailbox 的寄存器参见 [Table 176](#)

Table 176: Mailbox 寄存器列表

Register	Offset	R/W		Description
		to SoC	to eHSM	
MB_S2H_INFO[N]	0x000+N*0x4	RW	RO	SoC 到 eHSM 信息寄存器 N (N=0-1)
MB_H2S_INFO[N]	0x080+N*0x4	RO	RW	eHSM 到 SoC 信息寄存器 N (N=0-1)
MB_S2H_NOTE	0x100	RW1S	RW1C	SoC 到 eHSM 通知
MB_H2S_NOTE	0x104	RW1C	RW1S	eHSM 到 SoC 通知
MB_S2H_SOC_INT	0x110	RW1C	RO	SoC 到 eHSM, SoC 的中断
MB_S2H_SOC_INT_EN	0x114	RW	RO	SoC 到 eHSM, SoC 的中断使能
MB_S2H_HSM_INT	0x118	RO	RW1C	SoC 到 eHSM, eHSM 的中断
MB_S2H_HSM_INT_EN	0x11C	RO	RW	SoC 到 eHSM, eHSM 的中断使能
MB_H2S_SOC_INT	0x120	RW1C	RO	eHSM 到 SoC, SoC 的中断
MB_H2S_SOC_INT_EN	0x124	RW	RO	eHSM 到 SoC, SoC 的中断使能
MB_H2S_HSM_INT	0x128	RO	RW1C	eHSM 到 SoC, eHSM 的中断
MB_H2S_HSM_INT_EN	0x12C	RO	RW	eHSM 到 SoC, eHSM 的中断使能
eHSM_STATUS_0	0x400	RO	RO	eHSM 状态低 32 位, 与 o_hsm_status[31:0] 相同
eHSM_STATUS_1	0x404	RO	RO	eHSM 状态高 32 位, 与 o_hsm_status[63:32] 相同
MB_INT_ALL	0x480	RO	RO	SoC 一侧: 读出所有 mailbox 的 SoC 中断 eHSM 一侧: 读出所有 mailbox 的 eHSM 中断
MB_HSM_CFG0	0x4F8	RO	RO	eHSM cfg[31:0]
MB_HSM_CFG1	0x4FC	RO	RO	eHSM cfg[63:32]

12.3.2 Mailbox MB_S2H_INFO_D[N] 寄存器

Mailbox MB_S2H_INFO_D 寄存器用于存储从 SoC 传输到 eHSM 的信息, 例如指令。

Table 177: Mailbox MB_S2H_INFO_D[N] 寄存器

Bits	Name	Access Type	Default	Description
31:0	s2h_info_d[N]	RW/RO	0	SoC 到 eHSM 的信息 N (N=0-1)

12.3.3 Mailbox MB_H2S_INFO_D[N] 寄存器

Mailbox MB_H2S_INFO_N 寄存器用于存储从 eHSM 传输到 SoC 的信息, 例如响应。

Table 178: Mailbox MB_H2S_INFO_D[N] 寄存器

Bits	Name	Access Type	Default	Description
31:0	h2s_info_d[N]	RO/RW	0	eHSM 到 SoC 的信息 N (N=0-1)

12.3.4 Mailbox MB_S2H_NOTE 寄存器

Mailbox MB_S2H_NOTE 寄存器用于 SoC 通知 eHSM 已发送命令。

Table 179: Mailbox MB_S2H_NOTE 寄存器

Bits	Name	Access Type	Default	Description
31:1	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
0	s2h_note	RW1S/ RW1C	0	SoC 到 eHSM 的通知

12.3.5 Mailbox MB_H2S_NOTE 寄存器

Mailbox MB_H2S_NOTE 寄存器用于 eHSM 通知 SoC 已发送响应。

Table 180: Mailbox MB_H2S_NOTE 寄存器

Bits	Name	Access Type	Default	Description
31:1	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
0	h2s_note	RW1C/ RW1S	0	eHSM 到 SoC 的通知

12.3.6 Mailbox MB_S2H_SOC_INT 寄存器

Mailbox MB_S2H_SOC_INT 寄存器在 Mailbox MB_S2H_NOTE 寄存器中相应位被 eHSM 清除时触发

Table 181: Mailbox MB_S2H_SOC_INT 寄存器

Bits	Name	Access Type	Default	Description
31	s2h_soc_col_int	RW1C/ RO	0	冲突中断，当 SoC 拉高且同时 eHSM 清除同一位 MB_S2H_NOTE 时拉高
30:1	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
0	s2h_soc_int	RW1C/ RO	0	当 MB_S2H_NOTE 被 eHSM 清除时拉高

12.3.7 Mailbox MB_S2H_SOC_INT_EN 寄存器

Table 182: Mailbox MB_S2H_SOC_INT_EN 寄存器

Bits	Name	Access Type	Default	Description
31	s2h_soc_col_int_en	RW/RO	0	中断使能，高电平有效
30:1	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
0	s2h_soc_int_en	RW/RO	0	中断使能，高电平有效

12.3.8 Mailbox MB_S2H_HSM_INT 寄存器

Mailbox MB_S2H_HSM_INT 寄存器在 Mailbox MB_S2H_NOTE 寄存器中相应位被 SoC 拉高时触发

Table 183: Mailbox MB_S2H_HSM_INT 寄存器

Bits	Name	Access Type	Default	Description
31	s2h_hsm_col_int	RO/ RW1C	0	冲突中断，当 SoC 拉高且同时 eHSM 清除同一位 MB_S2H_NOTE 时拉高
30:1	rsv	RO	0	Reserved, read as 0
0	s2h_hsm_int	RO/ RW1C	0	当 MB_S2H_NOTE 被 SoC 拉高时触发

12.3.9 Mailbox MB_S2H_HSM_INT_EN 寄存器

Table 184: Mailbox MB_S2H_HSM_INT_EN 寄存器

Bits	Name	Access Type	Default	Description
31	s2h_hsm_col_int_en	RO/RW	0	中断使能，高电平有效
30:1	rsv	RO	0	Reserved, read as 0
0	s2h_hsm_int_en	RO/RW	0	中断使能，高电平有效

12.3.10 Mailbox MB_H2S_SOC_INT 寄存器

Mailbox MB_H2S_SOC_INT 寄存器在 Mailbox MB_H2S_NOTE 寄存器中相应位被 eHSM 拉高时触发

Table 185: Mailbox MB_H2S_SOC_INT 寄存器

Bits	Name	Access Type	Default	Description
31	h2s_soc_col_int	RW1C/ RO	0	冲突中断，当 eHSM 拉高且同时 SoC 清除同一位 MB_H2S_NOTE 时拉高
30:1	rsv	RO	0	Reserved, read as 0
0	h2s_soc_int	RW1C/ RO	0	当 MB_H2S_NOTE 被 eHSM 拉高时触发

12.3.11 Mailbox MB_H2S_SOC_INT_EN 寄存器

Table 186: Mailbox MB_H2S_SOC_INT_EN 寄存器

Bits	Name	Access Type	Default	Description
31	h2s_soc_col_int_en	RW/RO	0	中断使能，高电平有效
30:1	rsv	RO	0	Reserved, read as 0
0	h2s_soc_int_en	RW/RO	0	中断使能，高电平有效

12.3.12 Mailbox MB_H2S_HSM_INT 寄存器

Mailbox MB_H2S_HSM_INT 寄存器在 Mailbox MB_H2S_NOTE 寄存器中相应位被 SoC 清除时拉高

Table 187: Mailbox MB_H2S_HSM_INT 寄存器

Bits	Name	Access Type	Default	Description
31	h2s_hsm_col_int	RO/ RW1C	0	冲突中断，当 eHSM 拉高且同时 SoC 清除同一位 MB_H2S_NOTE 时拉高
30:1	rsv	RO	0	Reserved, read as 0
0	h2s_hsm_int	RO/ RW1C	0	当 MB_H2S_NOTE 被 SoC 清除时拉高

12.3.13 Mailbox MB_H2S_HSM_INT_EN 寄存器

Table 188: Mailbox MB_H2S_HSM_INT_EN 寄存器

Bits	Name	Access Type	Default	Description
31	h2s_hsm_col_int_en	RO/RW	0	中断使能，高电平有效
30:1	rsv	RO	0	Reserved, read as 0
0	h2s_hsm_int_en	RO/RW	0	中断使能，高电平有效

12.3.14 Mailbox eHSM_STATUS_0 寄存器

Table 189: Mailbox eHSM_STATUS_0 寄存器

Bits	Name	Access Type	Default	Description
31:0	ehsm_status_0	RO	0	此寄存器用于 SoC 检查 eHSM 的状态，它的值与 o_hsm_status[31:0] 相同 Table 10

12.3.15 Mailbox eHSM_STATUS_1 寄存器

Table 190: Mailbox eHSM_STATUS_1 寄存器

Bits	Name	Access Type	Default	Description
31:0	ehsm_status_1	RO	0	此寄存器用于 SoC 检查 eHSM 的状态，它的值与 o_hsm_status[63:32] 相同 Table 10

12.3.16 Mailbox MB_INT_ALL 寄存器

Table 191: Mailbox MB_INT_ALL 寄存器

Bits	Name	Access Type	Default	Description
31: 16	rsv	RO	0	Reserved, read as 0
15:0	int_all	RO	0	SOC 一侧: 读出所有 mailbox 的 SoC 中断 eHSM 一侧: 读出所有 mailbox 的 eHSM 中断

12.3.17 Mailbox MB_HSM_CFG0 寄存器

Table 192: Mailbox MB_HSM_CFG0 寄存器

Bits	Name	Access Type	Default	Description
31:0	soc_hsm_cfg0	RO	0	hsm cfg[31:0], 读出为对应的 Mailbox ID

12.3.18 Mailbox MB_HSM_CFG1 寄存器

Table 193: Mailbox MB_HSM_CFG1 寄存器

Bits	Name	Access Type	Default	Description
31:0	soc_hsm_cfg1	RO	0	hsm cfg[63:32], 未使用, 保留为后续特性

12.4 Mailbox 软件使用流程

- 主机向 eHSM 发送命令
 - [MB_S2H_INFO_D\[0-1\]](#): 命令数据
 - [MB_S2H_NOTE](#) bit 0: 命令通知
- eHSM 向主机发送响应:
 - [MB_H2S_INFO_D\[0-1\]](#): 响应数据
 - [MB_H2S_NOTE](#) bit 0: 响应通知

12.4.1 主机向 eHSM 发送命令流程

1. 主机准备命令数据
 - 主机将命令数据写入 [MB_S2H_INFO_D\[0-1\]](#)
2. 主机拉高 [MB_S2H_NOTE](#) 第 0 位
 - [MB_S2H_HSM_INT](#) 第 0 位将被拉高, 通知 eHSM: 主机已拉高 [MB_S2H_NOTE](#) 第 0 位
3. eHSM CPU 接收 Mailbox 中断, 并读出 [MB_S2H_HSM_INT](#) 第 0 位被拉高
 - eHSM 读取 [MB_S2H_INFO_D\[0-1\]](#) 获取命令数据
 - eHSM 写入 0x1 到 [MB_S2H_HSM_INT](#) 以清除第 0 位中断
 - eHSM 写入 0x1 到 [MB_S2H_NOTE](#) 以清除第 0 位
 - [MB_S2H_SOC_INT](#) 第 0 位将被拉高, 通知主机: eHSM 已接收到命令
4. 主机接收 Mailbox 中断, 并读出 [MB_S2H_SOC_INT](#) 第 0 位被拉高
 - 主机写入 0x1 到 [MB_S2H_SOC_INT](#) 以清除第 0 位中断

此处命令已成功从主机发送到 eHSM, 主机可以重复上述步骤以发送新命令。

12.4.2 eHSM 向主机发送响应流程

该流程与主机向 eHSM 发送命令流程几乎相同。唯一的区别是涉及 MB_H2S_* 寄存器以支持相反方向的操作。

1. eHSM 准备响应数据

- eHSM 将响应数据写入 MB_H2S_INFO_D[0-1]

2. eHSM 拉高 MB_H2S_NOTE 第 0 位

- MB_H2S_SOC_INT 第 0 位将被拉高，通知主机: eHSM 已拉高 MB_H2S_NOTE 第 0 位

3. 主机 CPU 接收 Mailbox 中断，并读出 MB_H2S_SOC_INT 第 0 位被拉高

- 主机读取 MB_H2S_INFO_D[0-1] 获取响应数据
- 主机写入 0x1 到 MB_H2S_SOC_INT 以清除第 0 位中断
- 主机写入 0x1 到 MB_H2S_NOTE 以清除第 0 位
- MB_H2S_SOC_INT 第 0 位将被拉高，通知 eHSM: 主机已接收到响应

4. eHSM 接收 Mailbox 中断，并读出 MB_H2S_HSM_INT 第 0 位被拉高

- eHSM 写入 0x1 到 MB_H2S_HSM_INT 以清除第 0 位中断

现在响应已成功从 eHSM 发送到主机，eHSM 可以重复上述步骤以发送新的响应。

13 随机时钟

随机时钟模块可以为 CPU 和加密算法引擎生成随机时钟，并且时钟门控强度是可选的：

- 无时钟门控
- 在 32 个周期内随机关闭 1 个时钟周期
- 在 16 个周期内随机关闭 1 个时钟周期
- 在 8 个周期内随机关闭 1 个时钟周期

13.1 随机时钟寄存器

13.1.1 随机时钟寄存器映射

Table 194: 随机时钟寄存器映射

Register	Offset	Description
RANCLK_SEED	0x00	随机时钟种子寄存器
RANCLK_MODE	0x04	随机时钟模式寄存器
RANCLK_INT	0x08	随机时钟中断寄存器
RANCLK_INT_EN	0x0C	随机时钟中断使能寄存器

13.1.2 随机时钟种子寄存器

Table 195: 随机时钟种子寄存器

Bits	Name	Access Type	Default	Description
31:0	SEED	WO	0x0	随机种子

13.1.3 随机时钟模式寄存器

Table 196: 随机时钟模式寄存器

Bits	Name	Access Type	Default	Description
31:2	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
1:0	Mode	RW	0x0	随机时钟模式配置 0x0: 无门控时钟 0x1: 在 32 个周期内随机关闭 1 个时钟周期 0x2: 在 16 个周期内随机关闭 1 个时钟周期 0x3: 在 8 个周期内随机关闭 1 个时钟周期

13.1.4 随机时钟中断寄存器

Table 197: 随机时钟中断寄存器

Bits	Name	Access Type	Default	Description
31:1	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
0	INT	RO	0x0	LFSR 全 0 中断

13.1.5 随机时钟中断使能寄存器

Table 198: 随机时钟中断使能寄存器

Bits	Name	Access Type	Default	Description
31:1	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
0	INT_EN	RW	0x0	中断使能，高电平有效

14 看门狗定时器

- 16 位定时器
- 当定时器超时时触发中断/复位（计数到 0x0）

14.1 看门狗定时器用户工作流程

看门狗定时器的典型工作流程如下：

1. 在 WDT_WPT 寄存器中设置写入寄存器写使能。
2. 在 WDT_PSR 寄存器中设置时钟分频比。
3. 在 WDT_CTRL.CLKSEL 寄存器中设置时钟源。
4. 在 WDT_LDR 寄存器中设置看门狗定时器的初始值。
5. 在 WDT_RCR 寄存器中设置复位信号有效的时钟周期数。
6. 在 WDT_CTRL.EN 寄存器中设置看门狗定时器使能。
7. 在 WDT_ISR 寄存器中写 1 以清除中断。

Notice:

- 用户可以设置 WDT_CTRL.INT_EN 和 RST_EN 来使能中断和复位。（可选）
- 用户可以设置 WDT_CTRL.DIV 和 WDT_PSR 来调整超时时钟周期。（可选）
- 用户应更新 WDT_LDR 寄存器中的值以避免发生超时。
- 如果发生超时且中断未清除，看门狗定时器将重新加载 WDT_LDR 寄存器以开始下一次计数。

14.2 看门狗定时器寄存器

14.2.1 看门狗定时器寄存器映射

Table 199: 看门狗定时器寄存器映射

Register	Offset	Description
WDT_CTRL	0x00	看门狗定时器控制寄存器
WDT_PSR	0x04	看门狗定时器预分频寄存器
WDT_LDR	0x08	看门狗定时器初始值寄存器
WDT_VDR	0x0C	看门狗定时器计数寄存器
WDT_ISR	0x10	看门狗定时器中断状态寄存器
WDT_RCR	0x14	看门狗定时器复位周期寄存器
WDT_WPT	0x18	看门狗定时器写保护寄存器

14.2.2 看门狗定时器 WDT_CTRL 寄存器

Table 200: 看门狗定时器 WDT_CTRL 寄存器

Bits	Name	Access Type	Default	Description
31:6	rsv	RO	0	Reserved, read as 0
5:4	DIV	RWP	0x0	时钟分频比 0x0: 16 分频 0x1: 32 分频 0x2: 64 分频 0x3: 128 分频
3	CLKSEL	RWP	0x0	看门狗定时器时钟源选择 0: Pre-scale * DIV 1: Pre-scale
2	INT_EN	RWP	0x0	中断使能, 高电平有效
1	RST_EN	RWP	0x0	看门狗定时器超时复位使能, 高电平有效
0	En	RWP	0x0	看门狗定时器开关, 高电平有效

14.2.3 看门狗定时器 WDT_PSR 寄存器

Table 201: 看门狗定时器 WDT_PSR 寄存器

Bits	Name	Access Type	Default	Description
31:16	rsv	RO	0	Reserved, read as 0
15:0	Pre-scale	RWP	0x0	APB 时钟分频比 0x0: 不分频, 直接使用 pclk 其他: pclk 分频 Pre-scale+1 (0x1 至 0xFFFF)

14.2.4 看门狗定时器 WDT_LDR 寄存器

Table 202: 看门狗定时器 WDT_LDR 寄存器

Bits	Name	Access Type	Default	Description
31:16	rsv	RO	0	Reserved, read as 0
15:0	LD_VAL	RWP	0x0	看门狗定时器初始值

14.2.5 看门狗定时器 WDT_VDR 寄存器

Table 203: 看门狗定时器 WDT_VDR 寄存器

Bits	Name	Access Type	Default	Description
31:16	rsv	RO	0	Reserved, read as 0
15:0	LD_VAL	RO	0x0	看门狗定时器当前值

14.2.6 看门狗定时器 WDT_ISR 寄存器

Table 204: 看门狗定时器 WDT_ISR 寄存器

Bits	Name	Access Type	Default	Description
31:1	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
0	OV_PND	RW0C	0x0	看门狗定时器超时中断

14.2.7 看门狗定时器 WDT_RCR 寄存器

Table 205: 看门狗定时器 WDT_RCR 寄存器

Bits	Name	Access Type	Default	Description
31:16	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
15:0	RST_PRD	RWP	0x0	看门狗定时器超时的时钟周期数

14.2.8 看门狗定时器 WDT_WPT 寄存器

Table 206: 看门狗定时器 WDT_WPT 寄存器

Bits	Name	Access Type	Default	Description
31:0	WR_PRT	WO	0x0	上述信号: RWP/RW1C 的写保护信号, 信号值为: 0x7b3e8c5d

15 UART

- 20 位波特率分频器
- 8 位发送数据缓冲区，8 位接收数据缓冲区
- 中断：发送/接收/发送超限/接收超限

15.1 UART 用户工作流程

UART 的典型工作流程如下：

1. 在 UART_BAUD_DIV 寄存器中设置波特率。
2. 在 UART_CTRL.rx_en 和 UART_CTRL.tx_en 寄存器中设置 TX 和 RX 使能。
3. 在 UART_STATUS 寄存器中检查状态。
4. 如果 UART_STATUS.tx_buf_empty 被设置为 1，用户可以在 UART_DATA 寄存器中发送数据。
5. 如果 UART_STATUS.rx_buf_full 被设置为 1，用户可以在 UART_DATA 寄存器中接收数据。

15.2 UART 寄存器

TX_* 和 RX_* 信号在 eHSM 的方向上，没有主机端。

15.2.1 UART 寄存器映射

Table 207: UART 寄存器映射

Register	Offset	Description
UART_DATA	0x00	UART 数据寄存器
UART_STATUS	0x04	UART 状态寄存器
UART_CTRL	0x08	UART 控制寄存器
UART_INT_STA	0x0C	UART 中断状态寄存器
UART_BAUD_DIV	0x10	UART 波特率寄存器

15.2.2 UART 数据寄存器

Table 208: UART_DATA 寄存器

Bits	Name	Access Type	Default	Description
31:8	rsv	RO	0	Reserved, read as 0
7:0	data	RW	0x0	UART 发送数据写入；接收数据读取

15.2.3 UART 状态寄存器

Table 209: UART_STATUS 寄存器

Bits	Name	Access Type	Default	Description
31:4	rsv	RO	0	Reserved, read as 0
3	rx_buf_oflow	RW1C	0x0	Rx 缓存区溢出
2	tx_buf_oflow	RW1C	0x0	Tx 缓存区溢出
1	rx_buf_full	RO	0x0	Rx 缓存区满
0	tx_buf_full	RO	0x0	Tx 缓存区满

15.2.4 UART 控制寄存器

Table 210: UART_CTRL 寄存器

Bits	Name	Access Type	Default	Description
31:7	rsv	RO	0	Reserved, read as 0
6	fast_mode_en	RW	0x0	Fast 模式使能, 高有效
5	rx_oflow_int_en	RW	0x0	Rx 缓存区溢出中断使能, 高有效
4	tx_oflow_int_en	RW	0x0	Tx 缓存区溢出中断使能, 高有效
3	rx_int_en	RW	0x0	Rx 数据中断使能, 高有效
2	tx_int_en	RW	0x0	Tx 数据中断使能, 高有效
1	rx_en	RW	0x0	Rx 数据使能, 高有效
0	tx_en	RW	0x0	Tx 数据使能, 高有效

15.2.5 UART 中断状态寄存器

Table 211: UART_INT_STA 寄存器

Bits	Name	Access Type	Default	Description
31:4	rsv	RO	0	Reserved, read as 0
3	rx_oflow_int	RW1C	0x0	Rx 缓存区溢出中断
2	tx_oflow_int	RW1C	0x0	Tx 缓存区溢出中断
1	rx_int	RW1C	0x0	Rx 数据中断
0	tx_int	RW1C	0x0	Tx 数据中断

15.2.6 UART 速率寄存器

Table 212: UART_BAUD_DIV 寄存器

Bits	Name	Access Type	Default	Description
31:20	rsv	RO	0	Reserved, read as 0
19:0	baud_div	RW	–	UART 波特率分频 (最小值为 16)

Note: BAUD_DIV 的默认值 = 向下取整 (UART_CLK/BAUD_RATE+0.5)

16 CRC

- 支持 CRC16 和 CRC32

- CRC16 多项式为 $x^{16} + x^{15} + x^2 + 1$
- CRC32 多项式为 $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1$

16.1 CRC 用户工作流程

CRC 的典型工作流程如下：

- 在 CRC_ALG_CFG 寄存器中设置 CRC 启用、初始值和类型，用于首次计算。
- 在 CRC_DATA_IO[7:0] 寄存器中设置数据。（每个周期输入一个字节）
- 在输入所有数据后，用户可以在下一个周期从 CRC_DATA_IO 寄存器中读取 CRC 数据。
- 在 CRC_ALG_CFG 寄存器中设置 CRC 初始值，以进行下一次 CRC 计算。

LFSR 的典型工作流程如下：

- 在 LFSR_SEED_DAT 寄存器中设置 LFSR 初始种子。
- 在 CRC_ALG_CFG 寄存器中设置 LFSR 模型使能和类型。
- 如果设置为自动运行模式，用户可以立即在 CRC_DATA_IO 寄存器中读取 LFSR 数据。
- 如果设置为一次触发模式，用户可以在 LFSR_KICK 寄存器中设置任意值后，在 CRC_DATA_IO 寄存器中读取 LFSR 数据。

16.2 CRC 寄存器

16.2.1 CRC 寄存器映射

Table 213: CRC 寄存器映射

Register	Offset	Description
CRC_ALG_CFG	0x00	算法配置寄存器
CRC_DATA_IO	0x04	CRC 数据输入输出寄存器
LFSR_SEED_DATA	0x08	LFSR 种子输入寄存器
LFSR_KICK	0x0C	LFSR 单次触发寄存器

16.2.2 CRC 算法配置寄存器

Table 214: CRC_Alg_Config 寄存器

Bits	Name	Access Type	Default	Description
31:7	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
6	LFSR_KICK_MODE	RW	0x0	LFSR 模式 1: 单次触发模式 0: 自动运行模式
5	LFSR_EN	RW	0x0	1: LFSR 启用 0: LFSR 禁用
4	LFSR_32EN	RW	0x0	LFSR 32bit 启用 1: LFSR 32bit 启用 0: LFSR 16bit 启用
3	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
2	CRC_32EN	RW	0x0	CRC 32bit 启用 1: CRC 32bit 启用 0: CRC 16bit 启用
1	CRC_INIT_EN	RW	0x0	CRC 初始化启用 1: CRC 初始化启用 0: CRC 初始化禁用 用户应在 CRC 计算前进行初始化
0	CRC_EN	RW	0x0	CRC 启用 1: CRC 启用 0: CRC 禁用

16.2.3 CRC 数据 IQ 寄存器

Table 215: CRC_IO_DATA 寄存器

Bits	Name	Access Type	Default	Description
31:0	CRC_IN_OUT_DATA	WR*	0x0	WRITE: data[7:0] 用户数据输入 READ: 计算结果输出 CRC32 : data[31:0] 有效 CRC16 : data[15:0] 有效

16.2.4 LFSR 种子数据寄存器

Table 216: LFSR_SEED_DATA 寄存器

Bits	Name	Access Type	Default	Description
31:0	LFSR_SEED_DATA	WR*	0x0	WRITE: seed[31:0] 用户种子输入 READ: lfsr 数据结果输出 LFSR32: data[31:0] 有效 LFSR16 : data[15:0] 有效

16.2.5 LFSR KICK 寄存器

Table 217: LFSR_KICK 寄存器

Bits	Name	Access Type	Default	Description
31:0	LFSR_KICK	WO	0x0	用户可以输入任何值来单次触发 必须在单次触发模式下使用

17 通用定时器

- 有两个 24 位定时器
- 每个定时器可以触发中断（脉冲或电平取决于寄存器配置）

17.1 通用定时器工作流程

定时器的典型工作流程如下：

1. 配置预分频寄存器以生成定时器的计数时钟。
2. 配置初始值寄存器以设置计数器的周期。
3. 配置状态寄存器的 bit1 以设置中断信号的类型。
4. 同时配置控制寄存器的 bit2/1/0 以启用中断/定时器模式/定时器启用。
5. 产生配置好的定时器的中断信号。
6. 配置状态寄存器的 bit0 以清除中断信号。

17.2 通用定时器寄存器

17.2.1 通用定时器寄存器

Table 218: 通用定时器寄存器映射

Register	Offset	Description
TIM_CTL0	0x00	Timer 0 控制寄存器
TIM_CTL1	0x04	Timer 1 控制寄存器
TIM_PRS0	0x10	Timer 0 预分频寄存器
TIM_PRS1	0x14	Timer 1 预分频寄存器
TIM_INIT0	0x20	Timer 0 初始值寄存器
TIM_INIT1	0x24	Timer 1 初始值寄存器
TIM_CNT0	0x30	Timer 0 计数寄存器
TIM_CNT1	0x34	Timer 1 计数寄存器
TIM_STA0	0x40	Timer 0 状态寄存器
TIM_STA1	0x44	Timer 1 状态寄存器

17.2.2 通用定时器 TIM_CTL0 寄存器

Table 219: 通用定时器 TIM_CTL0 寄存器

Bits	Name	Access Type	Default	Description
31:4	rsv	RO	0	Reserved, read as 0
3	rsv	RW	0x0	–
2	INT_EN	RW	0x0	Timer0 中断使能, 高电平有效
1	Mode	RW	0x0	Timer0 模式 0: 单次触发模式 1: 重复触发模式
0	En	RW	0x0	Timer0 使能, 高电平有效

17.2.3 通用定时器 TIM_CTL1 寄存器

Table 220: 通用定时器 TIM_CTL1 寄存器

Bits	Name	Access Type	Default	Description
31:3	rsv	RO	0	Reserved, read as 0
2	INT_EN	RW	0x0	Timer1 中断使能, 高电平有效
1	Mode	RW	0x0	Timer1 模式 0: 单次触发模式 1: 重复触发模式
0	En	RW	0x0	Timer1 使能, 高电平有效

17.2.4 通用定时器 TIM_PRS0 寄存器

Table 221: 通用定时器 TIM_PRS0 寄存器

Bits	Name	Access Type	Default	Description
31:8	rsv	RO	0	Reserved, read as 0
7:0	Pre-scale	RW	0x0	Timer0 预分频时钟分频设置, 实际工作时钟为/(pre-scale+1)

17.2.5 通用定时器 TIM_PRS1 寄存器

Table 222: 通用定时器 TIM_PRS1 寄存器

Bits	Name	Access Type	Default	Description
31:8	rsv	RO	0	Reserved, read as 0
7:0	Pre-scale	RW	0x0	Timer1 预分频时钟分频设置, 实际工作时钟为/(pre-scale+1)

17.2.6 通用定时器 TIM_INIT0 寄存器

Table 223: 通用定时器 TIM_INIT0 寄存器

Bits	Name	Access Type	Default	Description
31:0	INIT_CNT	RW	0xFFFFFFFF	Timer0 初始值

17.2.7 通用定时器 TIM_INIT1 寄存器

Table 224: 通用定时器 TIM_INIT1 寄存器

Bits	Name	Access Type	Default	Description
31:0	INIT_CNT	RW	0xFFFFFFFF	Timer1 初始值

17.2.8 通用定时器 TIM_CNT0 寄存器

Table 225: 通用定时器 TIM_CNT0 寄存器

Bits	Name	Access Type	Default	Description
31:0	CNT	RW	0x0	Timer0 当前值

17.2.9 通用定时器 TIM_CNT1 寄存器

Table 226: 通用定时器 TIM_CNT1 寄存器

Bits	Name	Access Type	Default	Description
31:0	CNT	RW	0x0	Timer1 当前值

17.2.10 通用定时器 TIM_STA0 寄存器

Table 227: 通用定时器 TIM_STA0 寄存器

Bits	Name	Access Type	Default	Description
31:2	rsv	RO	0	Reserved, read as 0
1	INT_TYPE	RW	0x0	Timer0 中断信号类型 0: 脉冲 1: 电平
0	OV_PND	RW	0x0	Timer0 溢出 由硬件拉高 通过写入 0x0 清除

17.2.11 通用定时器 TIM_STA1 寄存器

Table 228: 通用定时器 TIM_STA1 寄存器

Bits	Name	Access Type	Default	Description
31:2	<i>rsv</i>	<i>RO</i>	<i>0</i>	<i>Reserved, read as 0</i>
1	INT_TYPE	RW	0x0	Timer1 中断信号类型 0: 脉冲 1: 电平
0	OV_PND	RW	0x0	Timer1 溢出 由硬件拉高 通过写入 0x0 清除

18 集成

18.1 发布数据库

Table 229: eHSM Database

Files	Description
└─ doc	eHSM Doc
└─ TRM	TRM doc
└─ OSR_eHSM_HP_Technical_Reference_Manual_CN.pdf	eHSM 技术参考手册
└─ OSR_eHSM_HP_Custom_Design_Technical_Manual.pdf	Custom design spec doc
└─ OSR_TRNG_RO_Implementation_Guide.pdf	TRNG RO 实施指南
└─ OSR_TRNG_FPGA_Implementation_Guide.pdf	TRNG constraint guide
└─ VR	验证文档
└─ OSR_eHSM_TC.pdf	验证用例报告
└─ OSR_eHSM_TP.pdf	验证需求报告
└─ OSR_eHSM_VR.pdf	验证报告
└─ CPU	CPU doc
└─ Wing-M130_Integration_Manual.pdf	CPU 集成手册
└─ Wing-M130_Technical_Reference_Manual_v1p0.pdf	CPU 技术参考手册
└─ eHSM 仿真环境说明.pdf	eHSM 仿真环境说明文档
└─ filelist	eHSM RTL filelist
└─ hsm_rtl.f	
└─ fpga	eHSM FPGA demo
└─ src	TB host source code
└─ sw	eHSM FW
└─ rom_bootloader	bootloader
└─ ehsm_fw	Firmware
└─ romcode	eHSM ROM code
└─ hsm_bootloader.romcode	Bootloader ROM code
└─ hsm_firmware.romcode	Firmware ROM code
└─ rtl	eHSM RTL
└─ osr_cell_wrapper.v	依赖于工艺的单元
└─ osr_ehsm_top.v	eHSM RTL 顶层文件
└─ osr_define_cfg.v	eHSM 可配置宏
└─ osr_define_key.v	eHSM 可配置密钥参数
└─ osr_define_sim.v	eHSM 仿真配置宏

Table 229: eHSM Database

Files	Description
└─ sim	eHSM 仿真文件
└─ romcode	用于 OSR 仿真的 ROM 代码
└─ run	sim 运行（日志/报告/波形）
└─ run_regression	回归脚本
└─ tb	Testbench
└─ tc	Testcase
└─ syn	eHSM 综合文件
└─ runsyn	综合脚本
└─ scripts	DC 脚本
└─ compile.tcl	DC compile 脚本
└─ constraint.tcl	时序约束文件
└─ dc_setup.tcl	DC 设置

18.2 eHSM 宏定义

eHSM 支持以下宏定义：

- RTL Key
- 仿真参数
- 功能配置

18.2.1 eHSM RTL Key

eHSM RTL Key 在 rtl/osr_define_key.v 文件中定义。

用户必须更改这些 RTL Key（值必须是十六进制的）。

Table 230: eHSM RTL Key 宏

Macro	Release Example Value	Description
RTL Key		
OSR_RTL_KEY	128'hFDB97531_ECA86420_01234567_89ABCDEF	Chip RTL Key
OSR_RTL_KEY_INSTALL_KEK_EHSM	128'hFEDCBA98_76543210_DBF73159_86AC204E	Chip RTL KEK Key 用于 eHSM OTP 密钥安装
OSR_RTL_KEY_INSTALL_KEK_SOC	128'hFEDCBA98_76543210_02468ACE_13579BEF	Chip RTL KEK Key 用于 SoC OTP 密钥安装
IROM Cipher		
OSR_ROM_CIPHER_KEY	64'hFEDCBA98_76543210	IROM 加密密钥
OSR_ROM_CIPHER_DNONCE	32'h89ABCDEF	IROM 加密数据随机数
OSR_ROM_CIPHER_ANONCE	32'h01234567	IROM 加密地址随机数
DFT KEY		
OSR_SCAN_RTL_KEY	128'h0	由用户定义具体值，用于取代 DFT 模式下的三个 RTL Key 密钥的值，以防止 DFT 模式下的真实 RTL Key 密钥泄露

18.2.2 eHSM 仿真宏

eHSM 仿真相关宏定义在 rtl/osr_define_sim.v 文件中

这些宏仅用于前端 RTL 仿真。

Table 231: eHSM 仿真宏

Macro	Release Example Value	Description
OSR_SIM_CLK_FREQ	500_000_000	TB 仿真使用的 eHSM i_clk 时钟频率, 单位: Hz

18.2.3 eHSM 配置宏

eHSM 配置宏定义在 rtl/osr_define_cfg.v 文件中

必须根据实际设计更新 UART 波特率和时钟的宏

Table 232: eHSM 配置宏

Macro	Release Example Value	Description
时钟频率和 UART 波特率配置		
OSR_FREQ_I_CLK	100_000_000	eHSM i_clk 时钟频率, 单位: Hz
OSR_BUAD_RATE	115_200	UART 默认波特率
UTC Timer 分频		
OSR_UTC_TIM_SEC_DIV	8_000_000	UTC clock frequency, Hz
SoC Debug Enable 参数默认值设置		
OSR_SOC_DBG_EN_128B0	0x0000_0001	参见 Table 46
OSR_SOC_DBG_EN_128B1	0x0000_0000	参见 Table 46
OSR_SOC_DBG_EN_128B2	0x0000_0000	参见 Table 46
OSR_SOC_DBG_EN_128B3	0x0000_0000	参见 Table 46
TRNG 仿真设置		
OSR_TRNG_FPGA	0	仿真环境使用伪随机数生成器代替 RO 环
时序优化选项		
OSR_SOC_AHB_BUS_TIMING_ISOLATION	1	针对 SOC_AHB_BUS 的时序优化
OSR_NVM_AHB_BUS_TIMING_ISOLATION	0	针对 NVM_AHB_BUS 的时序优化
OSR_OTP_AHB_BUS_TIMING_ISOLATION	0	针对 OTP_AHB_BUS 的时序优化
OSR_IROM_AHB_BUS_TIMING_ISOLATION	0	针对 IROM_AHB_BUS 的时序优化
OSR_IRAM_AHB_BUS_TIMING_ISOLATION	0	针对 IRAM_AHB_BUS 的时序优化
OSR_DRAM_AHB_BUS_TIMING_ISOLATION	0	针对 DRAM_AHB_BUS 的时序优化
OSR_ABUS_AHB_BUS_TIMING_ISOLATION	1	针对 AHB_BUS 的时序优化
OSR_AHB_DMA_TIMING_ISOLATION	0	针对 AHB_DMA 的时序优化

18.3 ROM Code

The Bootloader ROM Code 位于:

[romcode/hsm_bootloader.romcode](#)

The Firmware ROM Code 位于:

[romcode/hsm_firmware.romcode](#)

Note: 注意: [romcode/](#) 文件夹中的 ROM Code 是官方发布的。[sim/romcode/](#) 文件夹中的 ROM Code 仅用于 OSR 环境中的仿真。

18.3.1 生成 ROM Code

如果更改了 eHSM 密钥宏或 eHSM 固件, 则必须重新生成 ROM Code。

如下展示了如何从 “.hex” 文件生成 ROM Code。

1. 使用 [riscv32-wing-elf-objcopy.exe](#) 工具从 elf 文件生成 hex 文件

2. `cd romcode`
3. Usage: `./mk_romcode_hex [xx.vhex] [-l (the storage space of instruction memory (unit: kByte))] [-k xx_key.v] [-o output_name] [-64]`

示例:

```
riscv32-wing-elf-objcopy.exe -O verilog ehsm_fw.elf ehsm_fw.vhex  
./mk_romcode ehsm_fw.vhex -l 512 -k ../rtl/osr_define_key.v -o firmware
```

Note:

1. `[-k xx_key.v]` 是可选的。如果没有 ROM 加密，不需要输入此参数。
2. `[-o xxx]` 是不可缺少的，用于定义输出 `romcode` 的文件名。
3. `[-64]` 是可选的。如果 ROM Code 是 64 位宽，需要输入此参数。
4. hex 长度示例：
 - (a) `-l 256`: 256kByte
 - (b) `-l 128`: 128kByte
 - (c) `-l 32`: 32kByte

下面显示了如何从 “.bin” 文件为 eHSM 生成 ROM Code。

1. `cd romcode`
2. Usage: `./mk_romcode_bin [xx.bin] [-l (the length of the "xx.bin" file)] [-k xx_key.v] [-o output_name] [-64]`

示例:

```
./mk_romcode ../fw/ehsm.bin -l 40000 -k osr_define_key.v -o firmware
```

Note:

1. `[-k xx_key.v]` 是可选的。如果没有 ROM 加密，不需要输入此参数。
2. `[-64]` 是可选的。如果 ROM Code 是 64 位宽，需要输入此参数。
3. bin 长度示例：
 - (a) `-l 40000`: 256kByte
 - (b) `-l 20000`: 128kByte
 - (c) `-l 8000`: 32kByte

18.4 标准单元库

18.4.1 存储器

eHSM 中使用的存储器列在表 [Table 3](#) 中

18.4.2 标准单元

eHSM 中使用的所有标准单元列在：

[rtl/osr_cell_wrapper.v](#)

[rtl/osr_sync_level_2.v](#)

[rtl/osr_sync_level_3.v](#)

18.5 仿真

eHSM TB 仿真按以下方式运行：

1. [cd sim](#)
2. 编译: [make com](#)
3. 运行 [case](#)
 - (a) 运行 regression: [./run_regression](#)
 - (b) 运行单个 case: [make com](#); [make sim waveswitch=off TC=TC_NAME](#)
 - i. [waveswitch=on/off](#): 启用/禁用波形文件

注意：

- eHSM TB 仿真平台和 TB case 的介绍请参考[eHSM 仿真环境说明.pdf](#)。
- [sim](#) 目录中的文件仅可用于仿真。

18.6 TRNG 仿真

在使用 TRNG 进行仿真时，有一些配置需要注意：

对于在非芯片环境（如 FPGA 和仿真器）上的 [OSR_TRNG](#) 仿真，用户可以手动添加 `< 'define OSR_TRNG_FPGA>` 以用伪随机数生成器替换 RO-Loop。

伪随机数生成器可用于在没有时序延迟的平台上进行仿真。它也可以用于需要绕过 RO-Loop 的平台，或更侧重于其他设计功能的仿真。

18.7 Tape-out 检查清单

Table 233: Tape-out 检查清单

Item	Description
standard cell	确保 18.4.2 中的标准单元正确替换（功能相同）
memory timing	确保存储器 IO 时序正确。参见 2.5.10 和 2.5.12 以及 2.5.14
RTL key define	指派高权限人员修改密钥的宏，参考??
UART baud rate define	确保 UART 波特率设置与 i_clk 频率匹配，参考 18.2.3
FPGA & Simulation define	不定义 “FPGA” 和 “OSR_SIM_”
RTL version	确保每个 Verilog 文件中的文件 ID 是最新的，以避免使用错误版本
TB cases Simulation	确保所有 TB 案例在 SoC 仿真环境中通过
FPGA verification	最好在 FPGA 平台上验证所有或至少一些关键功能测试，使用真实的 eHSM 固件和主机 API
TRNG RO Implementation	TRNG 熵源的 RO 时钟频率目标满足要求，请参考 OSR_TRNG_RO_Implementation_Guide.pdf 了解详情

19 历史版本

Table 234: 历史版本

Version	Date	Changes
4.0	June 17, 2025	初始发布版本
4.1	July 4, 2025	增加 KMU 章节中关于 KMU_CTRL 寄存器使用的补充说明 Table 170
4.2	July 23, 2025	修改系统 CSR 章节内关于 DBG_EN 寄存器的访问类型描述, 参见 Table 64 - Table 72
4.3	Aug 5, 2025	修改密钥管理单元章节中 KMU_BUF 的描述, 参见 Table 169
4.4	Aug 14, 2025	增加 IROM 时钟和复位控制寄存器的描述, 参见 Table 76 Table 83
4.5	Sep 16, 2025	移除车规相关模块的说明
4.6	Sep 17, 2025	修改 EMU 寄存器描述, 参见 Table 165 Table 166 修改错误状态接口电平类型, 参见 Table 11
4.7	Sep 26, 2025	更新 ROM Code 章节, 增加 bin 文件生成 romcode 脚本的使用描述 18.3
4.8	Oct 31, 2025	更新复位同步模块示意图 8
4.9	Dec 3, 2025	修改 Mailbox 特性章节中断输出个数说明 12.2
4.10	Dec 17, 2025	1. 修改 OTP 密钥安装章节 3.4.4 , 移除软件密钥相关使用方式说明, 具体使用请参考软件使用文档。 2. 修改 OTP 控制字段 KeyAlgSel 说明, 补充密钥解密算法 Table 35
4.11	Dec 23, 2025	修改硬件错误表格描述及处理方式说明 Table 11
4.12	Jan 8, 2026	删除冗余的仿真宏定义配置说明 Table 231
4.13	Jan 30, 2026	1. 更新顶层端口 o_soc_dbg_en_128b 相关说明, 参见 Table 46 2. 修改 OTP 密钥解密表格 Secret Key 0 的描述 Table 47 3. 修改看门狗定时器 WDT_PSR 寄存器描述 Table 201 4. 增加 TRNG 测试端口信号描述 Table 29
4.14	Apr 21, 2026	修正部分文字描述错误
4.15	May 22, 2026	1. 修改集成章节, 增加 OSR_SCAN_RTL_KEY 的宏定义说明 Table 230 2. 增加 o_soc_dbg_en_128b 信号在 DFT 测试模式下的说明 4.2.1